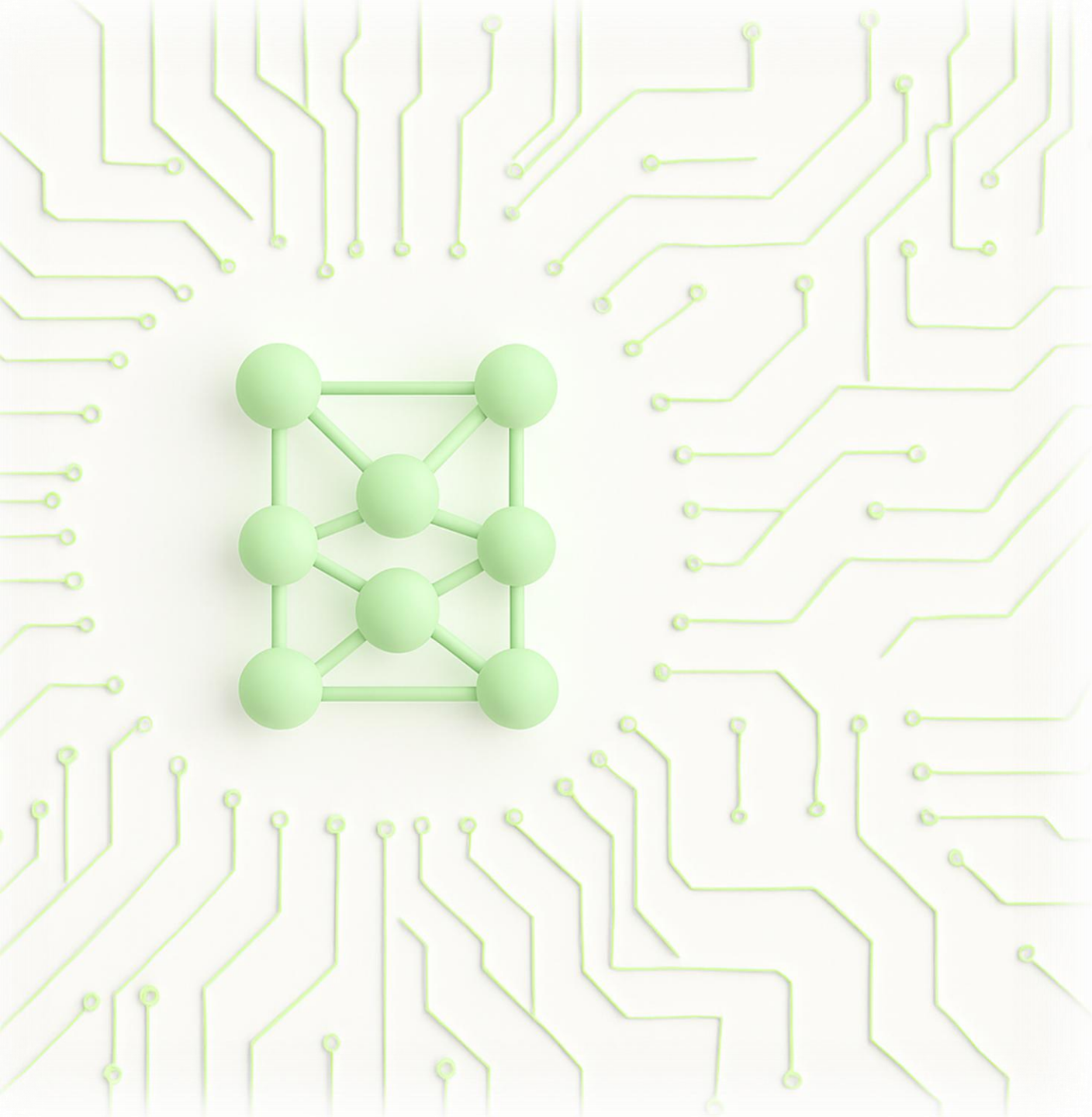




Deep Learning

Lakshmi Narasimhan
IIT Madras

Outline

1. Recap of Machine learning fundamentals

- Linear algebra and probability
- Supervised learning: Classification and Regression
- Loss and errors
- Model selection: over/under fitting, Bias-Variance tradeoff

Outline

2. Multilayer perceptron

- Function approximation theorems
- Feedforward neural networks
- Activation functions
- Width, depth and expressivity

Outline

3. Training neural networks

- Backpropagation
- Computational graph
- Optimization: Stochastic gradient descent and variants
- Avoiding overfitting: regularization, dropout, data augmentation
- Gradient instability, skip connection, initializations, early stop
- Normalization: input, layer, batch
- Hyperparameter tuning, curriculum training, transfer learning

Outline

4. Convolutional neural networks

- Multi-channel inputs
- Convolution and pooling
- Padding, striding, flattening
- Example architectures: AlexNet, GoogleNet, VGGNet, ResNet, U-Net

Outline

5. Recurrent neural networks

- Recursive neural networks
- Backpropagation through time
- Neural nets with memory: LSTM and GRU
- Sequence learning and bidirectional LSTM
- Optimizing RNN: Teacher forcing, leaky units, skipping through time

Outline

6. Attention mechanism

- Self attention and multi-head attention
- Query-key-value
- Attention pooling and scoring
- Embedding and encoding
- Transformer architecture

Outline

7. Autoencoders

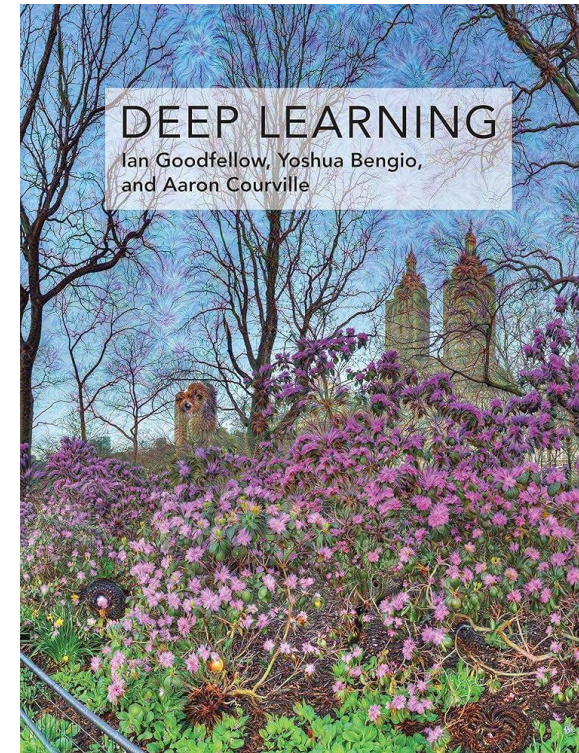
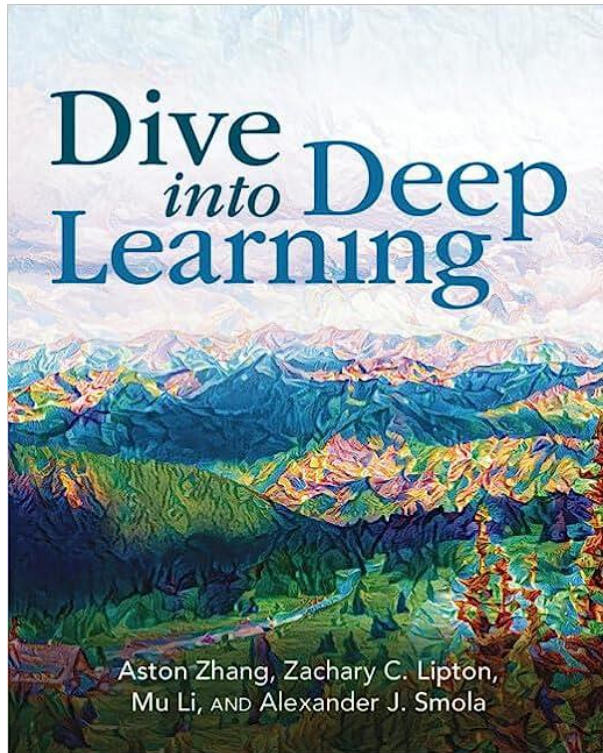
- Bottleneck architecture for encoder and decoders
- PCA/sparse decomposition with autoencoders
- Contractive and denoising autoencoders
- Variational autoencoder

Outline

8. State-of-the-art

- Graph neural networks
- Coperesheaf topological neural networks
- Physics informed neural networks
- Kolmogorov Arnold neural networks
- Diffusion and generative models

Reference materials



Logistics

- Lectures: 5.30 PM to 7 PM on Mon & Thu (Jan 5th to Apr 10th)
- Tutorials: On alternate weeks
- Assignments: One per two weeks
 - Theory: 50%
 - Programming: 50%
- Head TAs
 - Manoj Kumar (da24s018@smail.iitm.ac.in)
 - Yuvaram Singh (da24s015@smail.iitm.ac.in)

Grading

- 25% - Mid term, 22 Feb
- 25% - End term, 25 Apr
- 20% - Assignments (4 best out of 6)
- 30% - Project
 - Finalize topic by Feb 28
 - Deadline: April 22
 - Submit: Report + codes

Sample Project Ideas

- Sentence Sentiment Classification

- <https://www.kaggle.com/datasets/jp797498e/twitter-entity-sentiment-analysis>
- <https://www.kaggle.com/datasets/kazanova/sentiment140>

- Image Super Resolution

- <https://www.kaggle.com/datasets/adityachandrasekhar/image-super-resolution>
- <https://www.kaggle.com/datasets/jesucristo/super-resolution-benchmarks>

- GenAI with Variational Autoencoders

- <https://www.kaggle.com/code/basu369victor/generate-music-with-variational-autoencoder>
- <https://www.kaggle.com/code/averkij/variational-autoencoder-and-faces-generation>

- Product Recommender System

- <https://www.kaggle.com/datasets/shivamb/fashion-clothing-products-catalog>
- <https://www.kaggle.com/datasets/lokeshparab/amazon-products-dataset>

Project Report Format

- Introduction (1 mark)
- Background and state-of-the-art (1 mark)
- Problem Statement (4 marks)
- Dataset Description (2 marks)
- Solution/Methodology/Neural Net Architecture (4 marks)
- Theoretical Analysis (2 marks)
- Data Preprocessing and Training Methodology (3 marks)
- Experimental Results (5 marks for plots & description + 5 marks for attached code)
- Inferences/Insights Obtained/Difficulties faced (3 marks)
- Template - <https://www.overleaf.com/read/cbbbwmpqctbv#397b3a>

Linear algebra and probability

Vectorspaces

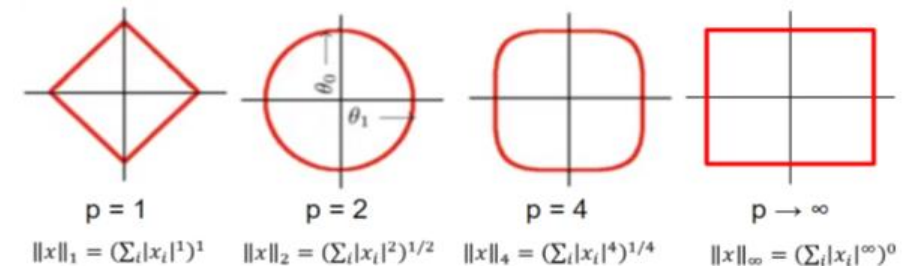
- A set $V : \forall \mathbf{x}, \mathbf{y} \in V, a\mathbf{x} + b\mathbf{y} \in V$, where a and b are any scalars
 - Closed under addition and scalar multiplication
- **Subspace**: a subset of V that is closed under addition and scalar multiplication
- Any element of this vectorspace is a **vector**
 - For finite-dimensional vectorspaces (e.g., $\mathbb{R}^n$): $n \times 1$ array of numbers
 - In our study, most data samples will be represented by a vector

Orthogonality & Norms

- An inner product on $\mathbb{R}^n$: $\mathbf{x}^T \mathbf{y}$
 - Measure of component of $\mathbf{x}$ on $\mathbf{y}$ (and vice versa)
 - **Orthogonal**: $\mathbf{x}^T \mathbf{y} = 0$ (perpendicular vectors)
 - $\mathbf{x}^T \mathbf{y} > 0$: $\mathbf{x}$ and $\mathbf{y}$ have components along the same direction
 - $\mathbf{x}^T \mathbf{y} < 0$: $\mathbf{x}$ and $\mathbf{y}$ have components along opposite directions

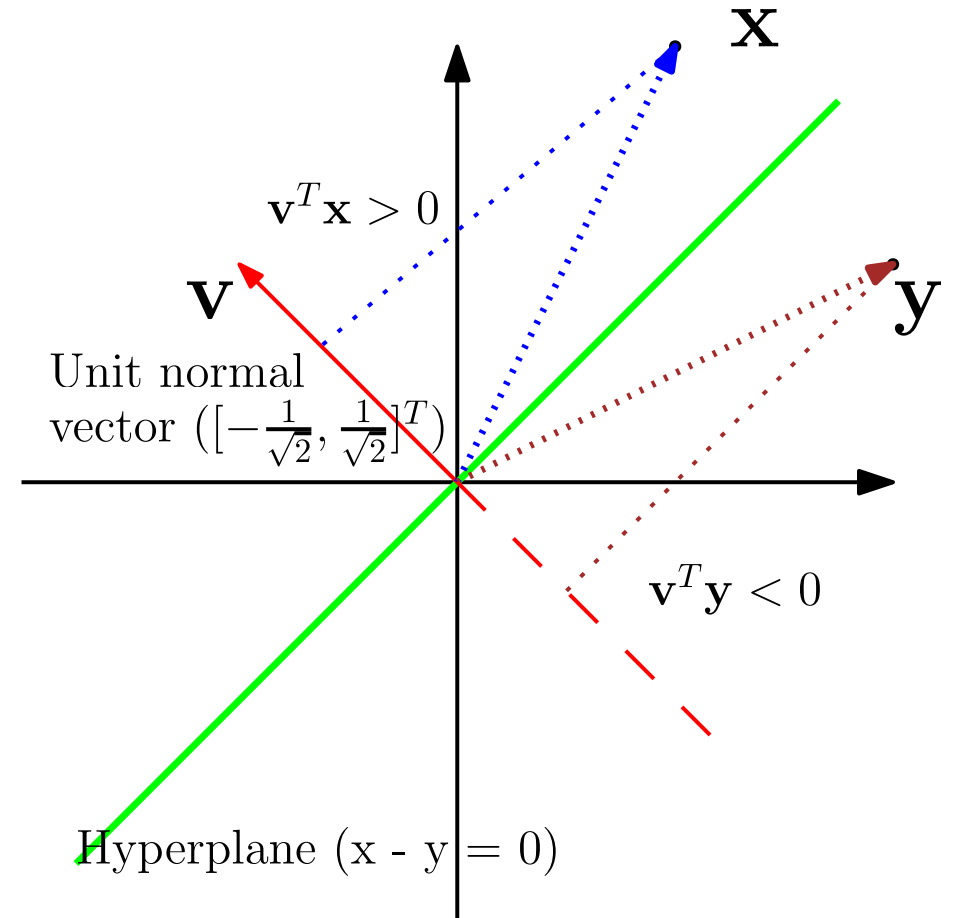
- **p - norm** ($p \geq 1$) : $\|\mathbf{x}\|_p := \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}$

- **Euclidean norm**: $\|\mathbf{x}\|^2 = \mathbf{x}^T \mathbf{x}$



Hyperplanes

- **Linear hyperplane**
 - An $n - 1$ dimensional subspace of $\mathbb{R}^n$
 - Passes through the origin
 - Divides $\mathbb{R}^n$ into two halves
 - Represented by a normal vector
- **Normal vector:**
 - A vector orthogonal to all vectors on the hyperplane
 - Distance of $\mathbf{x}$ from hyperplane = $\mathbf{x}^T \mathbf{v} / \|\mathbf{v}\|$



Singular Value Decomposition

- Singular value decomposition : $\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$
 - Exists for any matrix
 - $\mathbf{U}, \mathbf{V}$ are orthogonal: left and right **singular vectors**
 - $\mathbf{\Sigma}$ is PSD diagonal: The diagonal elements are **singular values**
 - The diagonal elements of $\mathbf{\Sigma}$ are in ascending order : $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n \geq 0$
- **Pseudoinverse**: $\mathbf{V}\mathbf{\Sigma}_r^{-1}\mathbf{U}^T$, where r is the rank of $\mathbf{M}$
- **Matrix norm**: $\|A\|_p = \sup_{x \neq 0} \frac{\|Ax\|_p}{\|x\|_p}$ $\|A\|_2 = \sigma_{\max}(A)$

Random Variable

- Almost all data collected in real life are affected by noise
 - This measurement is an outcome of the data collection experiment
 - The outcome may change every time the experiment is performed
 - However, certain *statistical* properties of these outcomes are deterministic
- **Probability space:** (Ω, F, P) – quantifying randomness of measurements
 - Ω : Sample space; F : Event space – all possible subsets of Ω
 - P : a measure for all elements of F – e.g., proportion of occurrence of any $u \in F$
- Random variable: a mapping from Ω to $\mathbb{R}$ (real numbers)

PMF vs PDF

Discrete random variables	Continuous random variables
Ω is countable all $w \in \Omega$ has $0 \leq P(w) \leq 1$	Ω is uncountable $u \in F$ has $0 \leq P(u) \leq 1$, $w \in \Omega$ has $P(w) = 0$
$X(w)$ is a real-discrete-valued r.v. $P(w)$ is its probability <i>mass</i>	$X(w)$ is a real-continuous-valued r.v. $P(w)$ is its probability <i>density</i>
Probability mass function (PMF) $P(X = x) = P(w)$ for $X(w) = x$	Probability density function (PDF) $P(X \in [a, b]) = \int_a^b P(x) dx = P(u)$, $u = \{X^{-1}(x), x \in [a, b]\}$
Law of total probability $\sum_i P(X = x_i) = P(\Omega) = 1$	Law of total probability $\int P(x) dx = P(\Omega) = 1$

Conditional Probability & Independence

- $P(X | Y)$: probability of occurrence of X when Y has occurred
 - The **sample space is reduced** by conditioning
 - Creates a new probability space and a valid PMF/PDF – $P(X = x | Y)$
 - Probability can increase or decrease upon conditioning
 - $P(X | Y) = P(X, Y) / P(Y)$
 - $P(X | Y = y)$ is a function of x , but $P(X | Y)$ is a function of both x and y
- **Independent** r.v.s: X and Y do not influence each other
 - $P(X, Y) = P(X) P(Y) \implies P(X | Y) = P(X)$

Bayes Theorem

- $P(X | Y) = P(Y | X) P(X) / P(Y)$
 - Given that Y is observed and a dependent hidden variable X exists, we can infer the probability of the hidden variable
- $P(X)$: Prior, statistics known before any observation
- $P(X | Y)$: Posterior, computed indirectly after observing Y
 - $P(X | Y = y)$ is a valid PMF/PDF; $P(X | Y = y) \propto P(Y = y | X) P(X)$
- $P(Y | X)$: Likelihood, computed based on the outcome of Y
 - $P(Y = y | X)$ is not a valid PMF/PDF
 - Note that the probability space itself changes for each value of X

Gaussian Distribution

- Gaussian $X \sim \mathcal{N}(\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{\sigma^2}}$
 - μ, σ : Only two parameters describe the entire PDF
- **Multivariate Gaussian:** $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{\sqrt{(2\pi)^n \det \boldsymbol{\Sigma}}} e^{-(\mathbf{x}-\boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x}-\boldsymbol{\mu})}$
 - $\mathbf{x} = [X_1, \dots, X_n]^T$ has n Gaussian r.v.'s that are also **jointly** Gaussian
 - $\boldsymbol{\mu} = \mathbb{E}\{\mathbf{x}\}$ is the mean vector, $\boldsymbol{\Sigma} = \mathbb{E}\{(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^T\}$ is a PSD covariance matrix
 - If $\mathbf{z} = \boldsymbol{\Sigma}^{-1/2} \mathbf{x}$, then the elements of $\mathbf{z}$ are independent Gaussian r.v.
 - Precision parameter: $\boldsymbol{\beta} = \boldsymbol{\Sigma}^{-1}$

KL Divergence

- **Maximum entropy distributions** (priors with least bias)
 - Uniform for discrete r.v.
 - Gaussian for continuous r.v.
- **Relative entropy**: $D(P \parallel Q) = \mathbb{E}_P\{\log P/Q\} \geq 0$
 - Measure of how distant is the distribution Q is different from P
- **Cross entropy**: $H(P; Q) = \mathbb{E}_P\{-\log Q\} \geq 0$ (for discrete r.v.)
 - $H(P; Q) = D(P \parallel Q) + H(P)$
 - Minimizing CE is equivalent to minimizing KLD

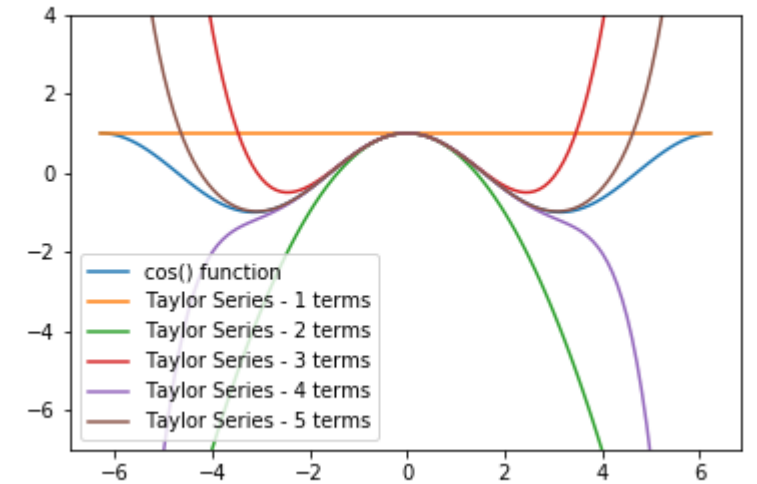
Classical vs Bayesian

Bayesian	Classical
Parameter to be estimated is a random variable	Parameter to be estimated is an unknown deterministic quantity
A prior distribution on the parameters is used	No prior (all values are equally likely - special case of Bayesian)
Posterior probability is optimized	Likelihood is optimized
Poor performance when prior is incorrect	Poor performance whenever the parameter is not uniformly distributed

Taylor Expansion

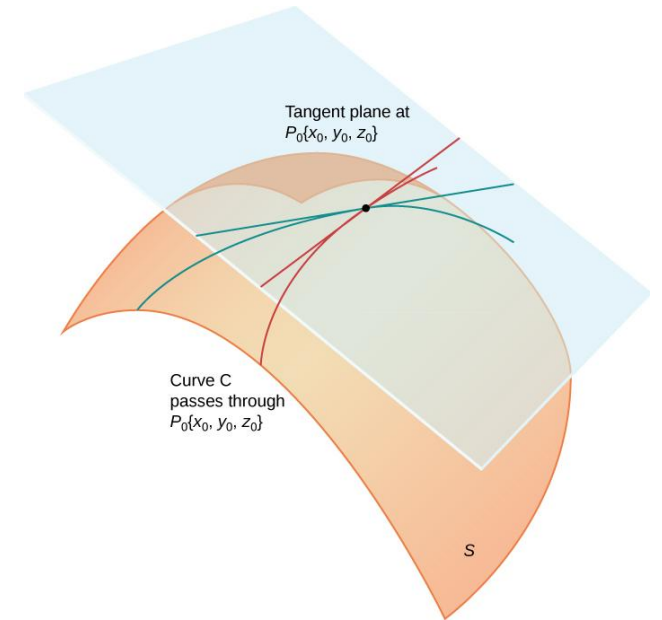
- Polynomial approximation

$$T(\mathbf{x}) = f(\mathbf{a}) + (\mathbf{x} - \mathbf{a})^\top Df(\mathbf{a}) + \frac{1}{2!}(\mathbf{x} - \mathbf{a})^\top \{D^2 f(\mathbf{a})\} (\mathbf{x} - \mathbf{a}) + \dots$$



- ($\nabla f(p) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(p) \\ \vdots \\ \frac{\partial f}{\partial x_n}(p) \end{bmatrix}$) Hessian

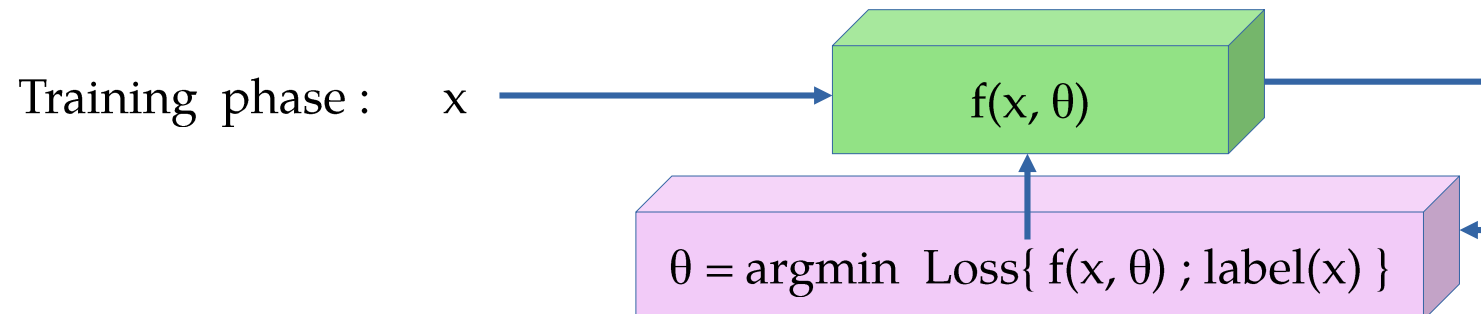
$$\mathbf{H}_f = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \dots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \dots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \dots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$



Machine Learning Fundamentals

Supervised/Unsupervised Learning

Supervised Learning	Unsupervised Learning
Training + Testing (inference) phase	Only inference phase
Requires labeled data for training	All data are unlabeled
Examples: Classification & estimation	Clustering, PCA, pattern finding



Loss Functions

- How to find the best parameters while training a supervised learning model?
 - Minimize the error between model output for input x_i and the label of x_i
- Loss function: A quantification of the error obtained during parameter optimization $\theta = \operatorname{argmin} \operatorname{Loss}\{f(x, \theta) ; \operatorname{label}(x)\}$
 - **L0** norm (sparsity-promoting), 0-1 loss: $\|y - y_{\text{true}}\|_0$
 - **L1** norm, absolute error: $\|y - y_{\text{true}}\|_1$
 - **Squared L2** norm, Euclidean distance: $\|y - y_{\text{true}}\|_2^2$
 - Mixed norms: linear combination of different p -norms ($p \geq 1$) – **convex losses**
 - Cross entropy, F-score, etc.

Regression

- Setup: $y_i = f(x_i) + \varepsilon$
- Find: $f(\cdot)$. Parameterized $f(x) = g(\mathbf{w}, x)$; linear regression: $g(\mathbf{w}, \mathbf{x}) = \mathbf{x}^T \mathbf{W}$
- Given (training data)
 - Labels: $\{y_i\}$
 - Inputs : $\{x_i\}$
- Predict (testing data)
 - Compute $\{y\}$ for unseen $\{x\}$

Classification

- Setup: $x_{i,j} \in C_i$; $i = 1, \dots, K$ and $j = 1, \dots, N$
- Find: $f(.)$ s.t. $f(x_{i,j}) = C_i$
- Given (training data)
 - Labels: $\{y_i\}$ s.t. $y_i \in \{C_i\}$
 - Inputs : $\{x_{i,j}\}$
- Predict (testing data)
 - Compute $\{y\}$ for unseen $\{x\}$ s.t. $y \in \{C_i\}$

Classification

- Binary classification: $K = 2$, $C_1 = +1$ and $C_2 = -1$
- Posterior probability for binary classification is

$$\begin{aligned}\Pr(y_i = +1|x_i) &= \frac{\Pr(x_i|y_i = +1) \Pr(y_i = +1)}{\Pr(x_i|y_i = +1) \Pr(y_i = +1) + \Pr(x_i|y_i = -1) \Pr(y_i = -1)} \\ &= \frac{1}{1 + e^{-a}} = \sigma(a); \quad a = \ln \frac{\Pr(x_i|y_i = +1) \Pr(y_i = +1)}{\Pr(x_i|y_i = -1) \Pr(y_i = -1)}\end{aligned}$$

$a = \log$ posterior ratios

- The **sigmoid** function naturally arises here; converts logits to probabilities

Classification

- Posterior probability for K-class classification is

$$\begin{aligned}\Pr(y_i = C_k | x_i) &= \frac{\Pr(x_i | y_i = C_k) \Pr(y_i = C_k)}{\sum_j \Pr(x_i | y_i = C_j) \Pr(y_i = C_j)} \\ &= \frac{e^{a_k}}{\sum_j e^{a_j}} = \sigma_k(\mathbf{a}); \quad a_k = \ln \Pr(x_i | y_i = C_k) \Pr(y_i = C_k)\end{aligned}$$

- The **softmax** function naturally arises here
- Converts logits (distance from hyperplanes) to probabilities
- **Single-layer perceptron** with sigmoid/softmax activation

Inference on Gaussian Data

- Binary classification

$$\Pr(\mathbf{x}_i | y_i = 1) = \frac{1}{(2\pi)^{M/2} \det(\Sigma)^{1/2}} e^{-\frac{(\mathbf{x}_i - \mu_k)^T \Sigma^{-1} (\mathbf{x}_i - \mu_k)}{2}}$$

$$\Pr(y_i = +1 | \mathbf{x}_i) = \sigma(\mathbf{x}_i^T \mathbf{w} + w_0); \quad \mathbf{w} = \Sigma^{-1}(\mu_1 - \mu_{-1})$$

$$w_0 = -\frac{\mu_1^T \Sigma^{-1} \mu_1}{2} + \frac{\mu_{-1}^T \Sigma^{-1} \mu_{-1}}{2} + \ln \frac{\Pr(y_i = 1)}{\Pr(y_i = -1)}$$

- Multiclass
classification

$$\Pr(y_i = +1 | \mathbf{x}_i) = \sigma_k(\mathbf{x}_i^T \mathbf{W} + \mathbf{w}_0); \quad \mathbf{W} = \Sigma^{-1}[\mu_1, \dots, \mu_K]$$

$$\mathbf{w}_0 = \left[-\frac{\mu_1^T \Sigma^{-1} \mu_1}{2} + \ln \Pr(y_i = 1), \dots, -\frac{\mu_K^T \Sigma^{-1} \mu_K}{2} + \ln \Pr(y_i = K) \right]$$

- Ridge regression

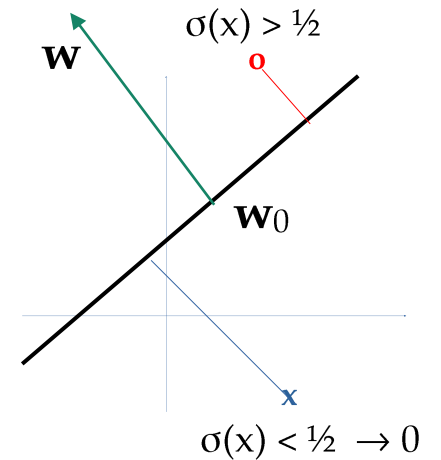
$$\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta}} \|\mathbf{y} - \mathbf{G}\boldsymbol{\theta}\|_2^2 \quad \text{s.t.} \quad \|\boldsymbol{\theta}\|_2^2 \leq B$$

$$= \arg \min_{\boldsymbol{\theta}} \|\mathbf{y} - \mathbf{G}\boldsymbol{\theta}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_2^2$$

$$= (\lambda I + \mathbf{G}^T \mathbf{G})^{-1} \mathbf{G}^T \mathbf{y}$$

Classifying Linearly Separable Data

- Hyperplane projection: $(\mathbf{x} - \mathbf{w}_0)^T \mathbf{w} = \mathbf{x}^T \mathbf{w} - \mathbf{w}_0^T \mathbf{w} = \mathbf{x}^T \mathbf{w} - w_0$
 - $\mathbf{w}_0$ is the **shift of origin** and $\mathbf{w}$ is **normal to the hyperplane**
- Binary classification
 - When $\mathbf{x}$ is on the hyperplane, $\Pr(y = C_1) = \sigma(0) = 1/2$
 - Farther from the hyperplane $\sigma(+ve) \rightarrow 1$ or $\sigma(-ve) \rightarrow 0$
 - Sigmoid output gives the PMF of the **Bernoulli** r.v. $f(x)$
- Multi-class classification
 - When $\mathbf{x}$ is on the hyperplane, $\Pr(y = C_1) = \sigma_k(\mathbf{0}) = 1/K$
 - Farther from the k^{th} hyperplane $\sigma_k(+ve) \rightarrow 1$ or $\sigma_k(-ve) \rightarrow 0$
 - Softmax output gives the PMF of the **categorical** r.v. $f(x)$



Logistic Regression – Single Layer Neural Net

- Let the classes be **one-hot encoded** (to represent a PMF)

$$\Pr(\mathbf{y}|\mathbf{x}, \mathbf{W}) = \prod_{k=1}^K \Pr(y = C_k|\mathbf{x}, \mathbf{W})^{y_k}; \quad \sum_{k=1}^K y_k = 1; \quad \sum_{k=1}^K \Pr(y = C_k|\mathbf{x}, \mathbf{W}) = 1$$

- To find $\mathbf{w}$: do **maximum likelihood regression**

$$\arg \min_{\mathbf{W}} -\ln \Pr(\mathbf{y}|\mathbf{x}, \mathbf{W}) = \arg \min_{\mathbf{W}} - \sum_{i=1}^N \sum_{k=1}^K y_{i,k} \ln \sigma_k(\mathbf{x}_i^T \mathbf{W}) = \arg \min_{\mathbf{W}} \sum_i H(y_i; \sigma_k(\mathbf{x}_i^T \mathbf{W}))$$

- The optimal hyperplanes are obtained using gradient descent

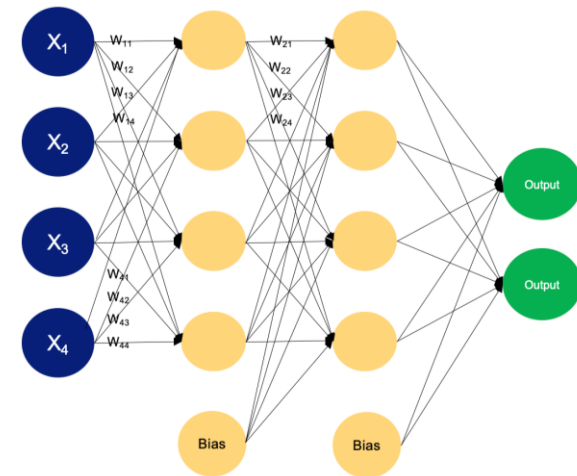
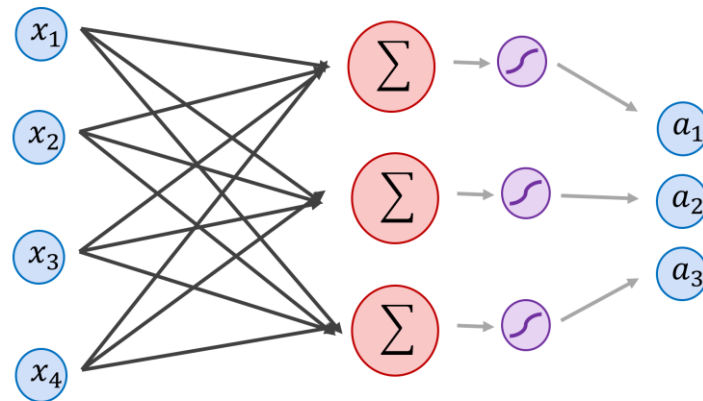
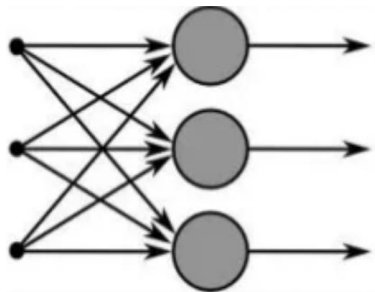
$$\mathbf{w}_{j,t+1} = \mathbf{w}_{j,t} - \eta \nabla_{\mathbf{w}_j} \mathcal{L} = \mathbf{w}_{j,t} - \eta \sum_i (\sigma_j(\mathbf{x}_i^T \mathbf{W}_t) - y_{i,j}) \mathbf{x}_i$$

Summary

- We focus on parametric supervised learning framework
- Neural models can be viewed as interpolators or curve-fitters
- Loss function: measure of deviation from expected output/label
- Optimal linear/ridge regression: a single layer linear neural net
- Optimal linear classification: a single layer neural net with sigmoid/softmax activation
- Learning weights from training data implicitly learns data prior

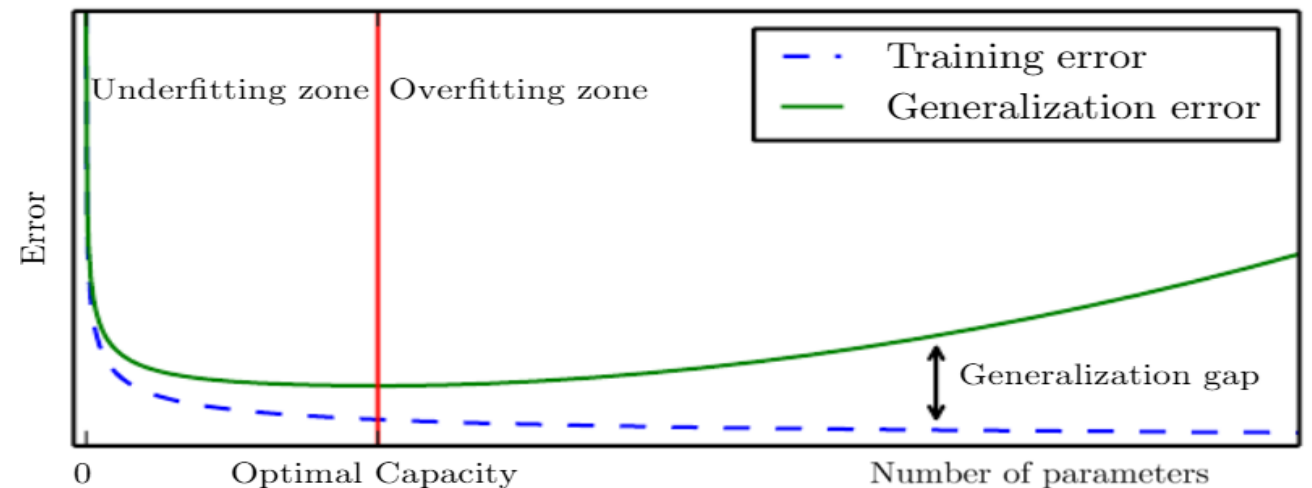
Extending to Multiple Layers

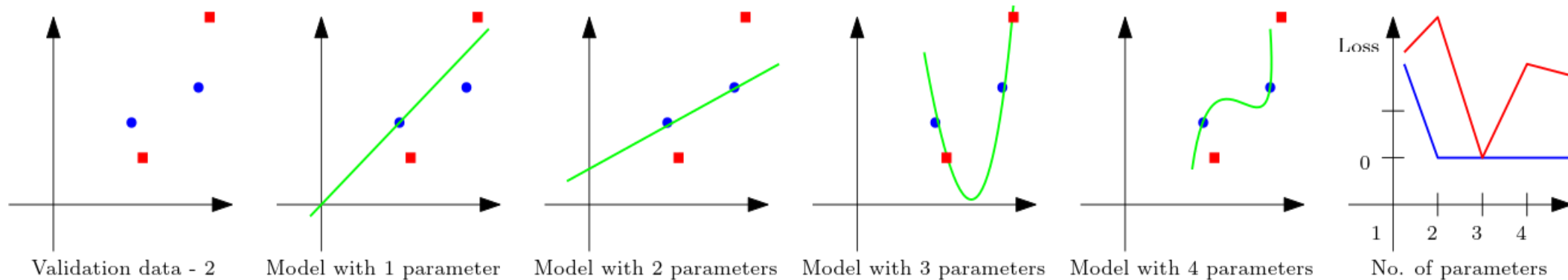
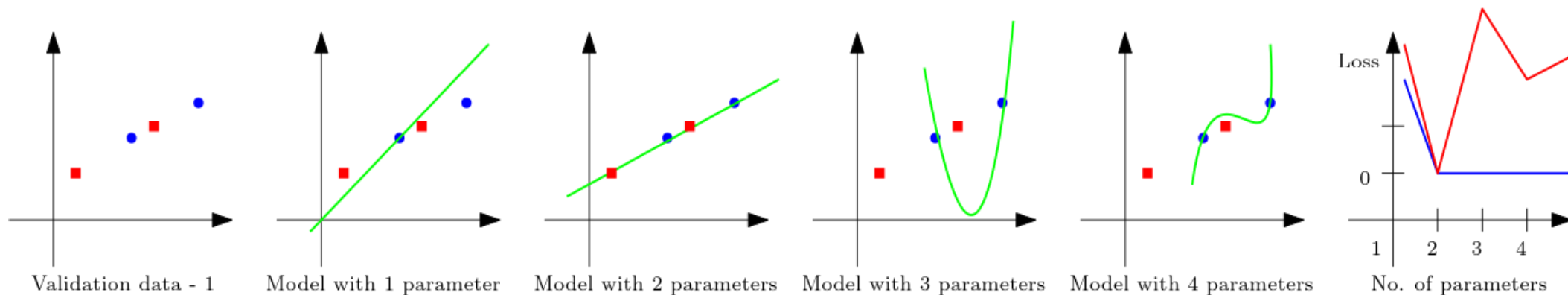
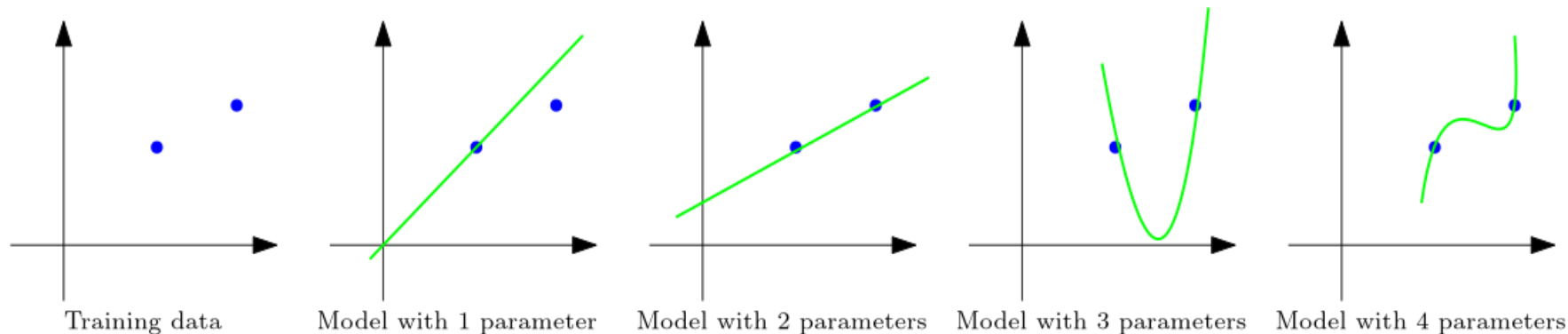
- We cannot model XOR with a linear classifier
- What if cascade multiple single layer neural net?
 - Similar to making intermediate decisions
- Can we keep increasing the number of layers/parameters?



Training/Generalization Error

- Increasing number of parameters is not always good
 - Training error may go down, but testing (generalization) error may be high
- There is an optimal number of parameters for a given kind of data
 - VC dimension ($|\text{Test error} - \text{Training error}| \leq O(d_{VC} \log n/n)^{1/2}$)
- Underfitting: Less than optimal
- Overfitting: More than optimal
 - How to identify overfitting?

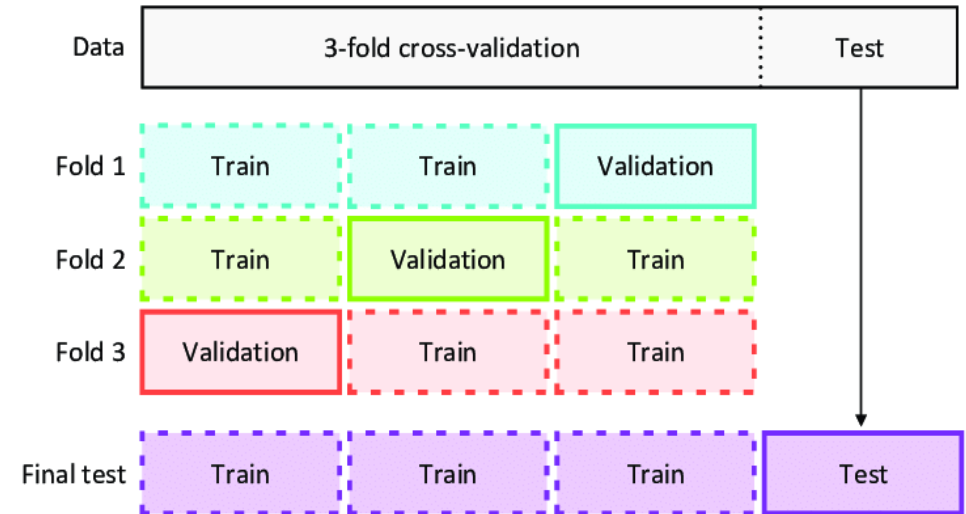




Validation Error & Overfitting

- Split available labeled data to training + validation sets
 - Sequentially minimize both training and validation errors
 - Use validation set to identify overfitting

- Validation error: The error measured with validation set – An indicator of generalization error

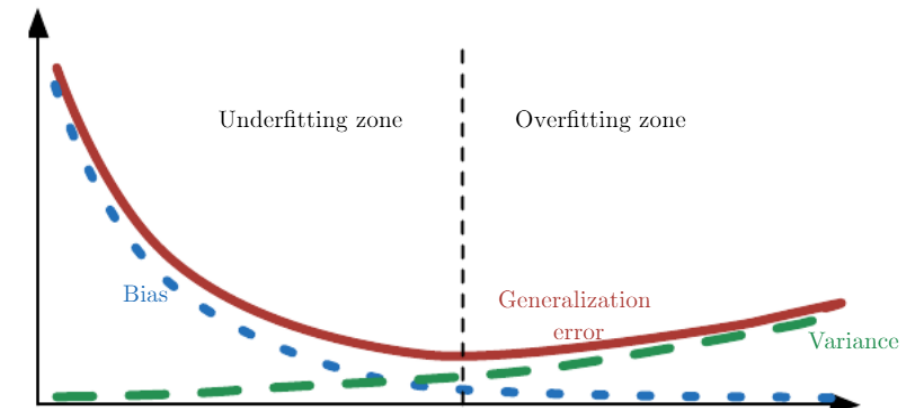


- **Cross validation:** When very few labeled data is available
 - Use different training+validation split in each iteration of training

Bias-Variance & Model-fitting

- Bias: $\mathbb{E}\{y - y_{\text{true}}\}$ – average deviation from true value
- Variance: $\text{Var}\{y\}$ – average deviation of inference
- Is lower bias or variance always good?

$$\begin{aligned}\mathbb{E}((y - \hat{y})^2) &= \mathbb{E}(y^2) + \mathbb{E}(\hat{y}^2) - 2\mathbb{E}(\hat{y})\mathbb{E}(y) \\ &= \mathbb{E}(y^2) - \mathbb{E}(y)^2 + \mathbb{E}(\hat{y}^2) - \mathbb{E}(\hat{y})^2 - 2\mathbb{E}(\hat{y})\mathbb{E}(y) + \mathbb{E}(y)^2 + \mathbb{E}(\hat{y})^2 \\ &= \text{Var}(y) + \text{Var}(\hat{y}) + \text{Bias}^2\end{aligned}$$



- Overfitting : High model variance and low MSE in training
- Underfitting : Low model variance and high MSE in training

Bayesian Inference

- Infer bias (p) of a coin from observations of its toss: $X_i \sim \text{Be}(p)$

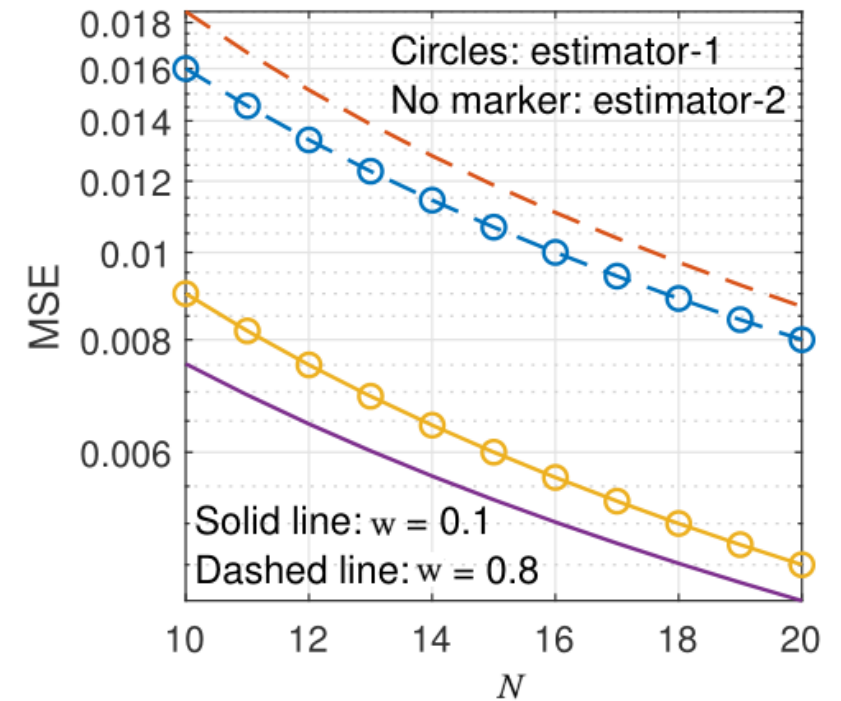
- (Classical) No bias

$$\hat{p}_1 = \arg \max_p p^{\sum y_i} (1 - p)^{N - \sum y_i} = \frac{\sum y_i}{N}$$

$$\text{Var}(\hat{p}_1) = \frac{p(1 - p)}{N} \quad \text{Bias}(\hat{p}_1) = 0$$

$$\hat{p}_3 = \arg \max_p p^{\sum y_i} (1 - p)^{N - \sum y_i} P(p) = \frac{\sum y_i}{N + 1}$$

- (Bayesian) Bias is already in-built in prior

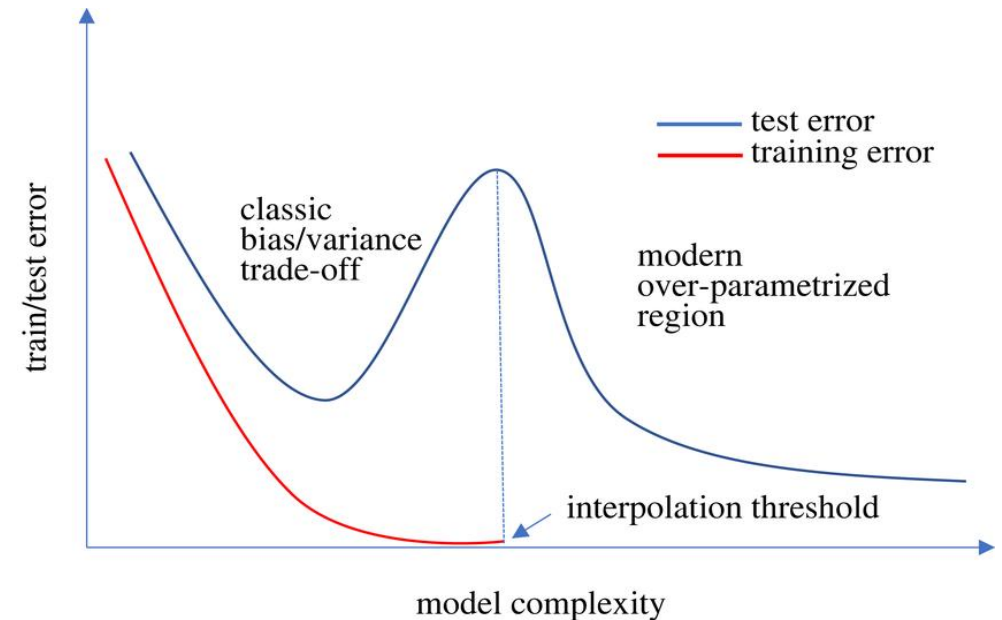


No Free-Lunch Theorem

- Training dataset may not contain all possible samples of input
 - How would the unseen data affect the learned statistics?
- **No free-lunch theorem**: As the unseen data can have different statistics, there can exist a dataset where any model (optimally trained with a training dataset) can perform poorly
 - So, will all models perform equally bad on average?
 - Tradeoff between **robustness** and **accuracy**
- **Inductive Bias**: If some statistics of the entire data is known, use that information in addition to training data to optimize the model – **different architectures**

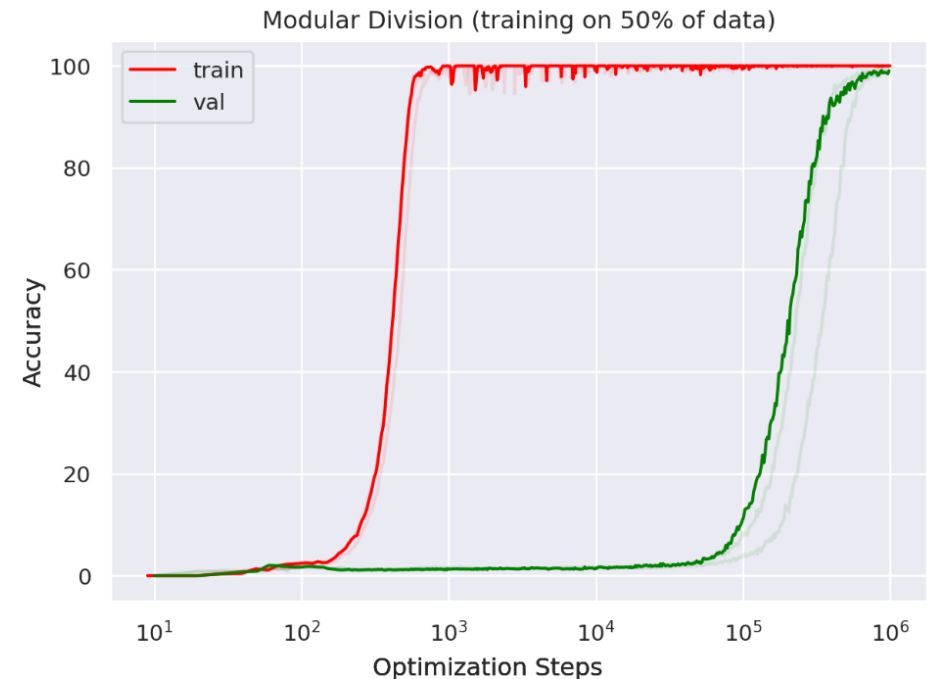
Double Descent

- Overfitting can be overcome
 - With deeper networks
- As the number of parameters (depth) increases, the overfitting is smoothened
 - Smoother interpolations and extrapolations
- Interpolation threshold
 - Where the number of parameters equals the number of data points
 - R. Shaeffer et al, *Double Descent Demystified: Identifying, Interpreting & Ablating the Sources of a Deep Learning Puzzle*
 - Some times more training data can hurt performance!



Grokking

- Generalization of over-parameterized nets by **over training**
- Amount of over training reduces with decrease in dataset size
 - *Grokking Explained: A Statistical Phenomenon, 2025*
- **Weight decay** helps
 - Additional term in the loss function to push the weights closer to zero



Assignment

- Generate training 100 samples : $y = x + 0.3x^2 + 0.7x^3 + w$
 - x is uniformly distributed in $[-3 \text{ to } 3]$
 - w is Gaussian noise with mean 0 and standard deviation 0.09
- Perform linear regression for model sizes $M = 1$ to 120 and plot
 - Bias (average of $y - f(x)$) vs M
 - Variance of y vs M
 - Training error vs M
 - Validation error vs M

Multilayer Perceptron

Feedforward Neural Network

Extending Single-Layer Neural Nets

- For linear data models
 - $y = \mathbf{w}^T \mathbf{x}$ or $\sigma(\mathbf{w}^T \mathbf{x} + b)$
 - Perceptron: $u(\mathbf{w}^T \mathbf{x} + b)$
- In general, $y = f(\mathbf{w}, \mathbf{x})$
 - What are some good parametric functions?
 - Are there a family of such functions that can fit any arbitrary N -dimensional curve?

Universal Function Approximation Theorems

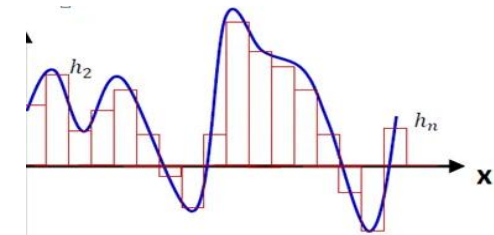
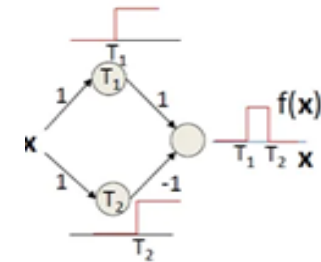
- Stone - Weierstrass (1885)
 - Every continuous function can be approximated by polynomials
- Andrew Barron (1993 - Universal Approximation Bounds for Superpositions)
 - Lipschitz functions can be approximated by infinite-width 1-layer NN
- Cybenko (1989 - Approximation by superpositions of a sigmoidal function)
 - Lipschitz functions can be approximated by infinite-width 1-layer sigmoid NN

Universal Function Approximation Theorems

- Hornik – Stinchcombe – White (1989 - Multilayer feedforward networks are universal approximators)
 - Lipschitz functions can be approximated by finite-width multi-layer NN
- Lu – Pu – Wang – Hu – Wang (2017 - The expressive power of neural network)
 - Lipschitz functions can be approximated by finite-width multi-layer ReLU NN
- Kolmogorov–Arnold (1957)
 - Lipschitz functions can be approximated by 2-layer NN

Universal Function Approximation

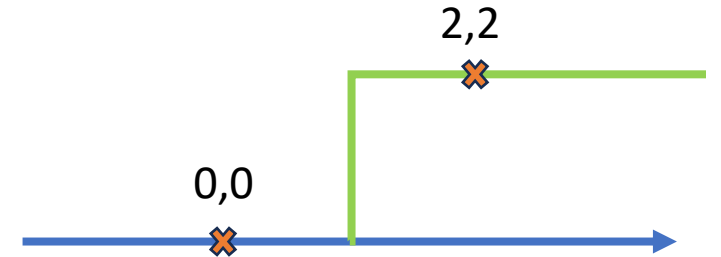
- Polynomials in high dimensions : very high complexity
- Any function $f(x)$ (with some regularity conditions) can be approximated by a superposition of scaled and shifted step functions
 - $f(x) = \sum a_i u(w_i x - b_i)$
 - w_i & b_i are the **scale** and **shift** factors
 - $u()$ is the **activation function**



Example

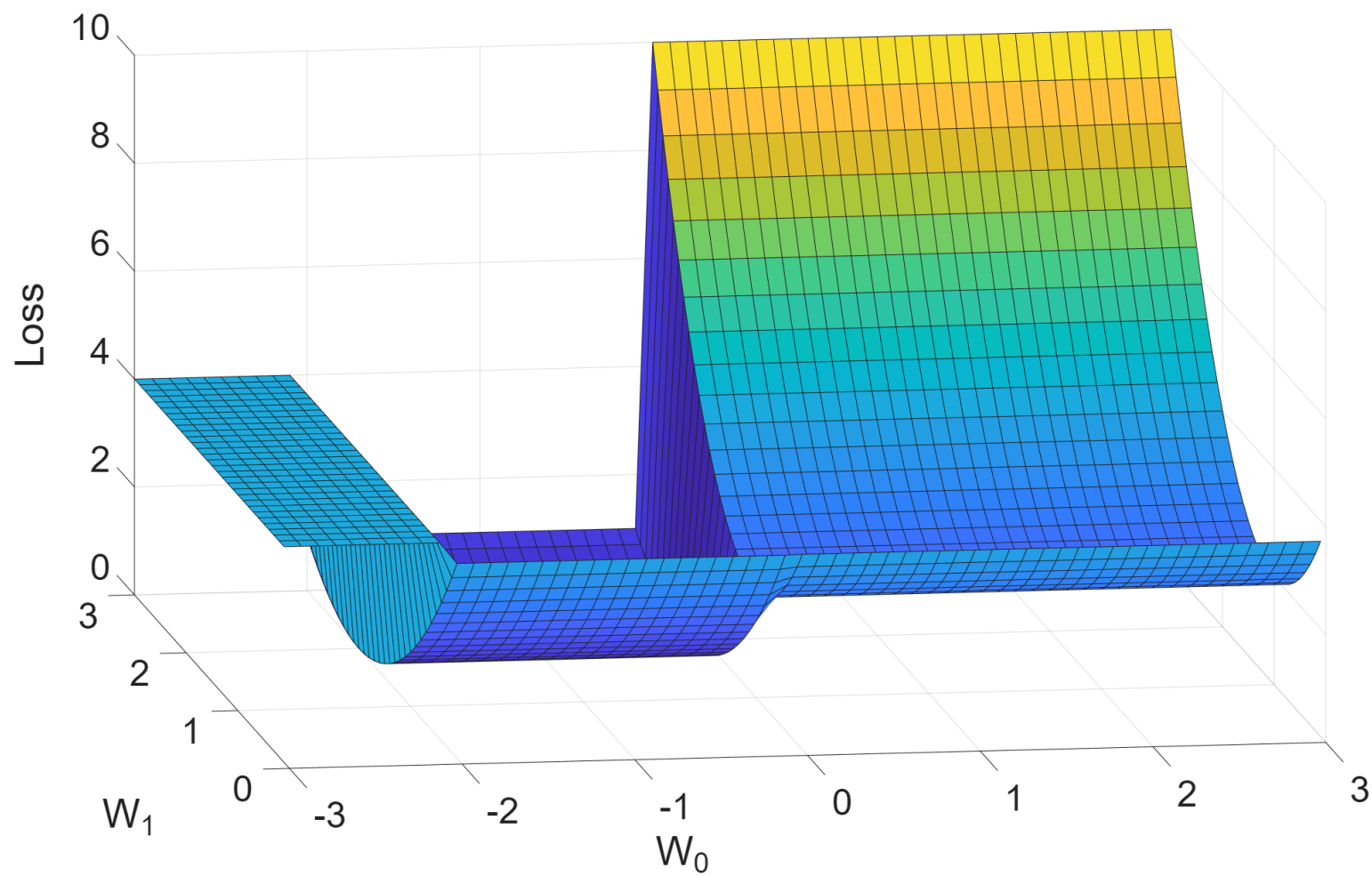
- Fit $w_1 u(x + w_0)$ with MSE loss

$$\text{Loss} = (0 - w_1 u(0 + w_0))^2 + (2 - w_1 u(2 + w_0))^2$$

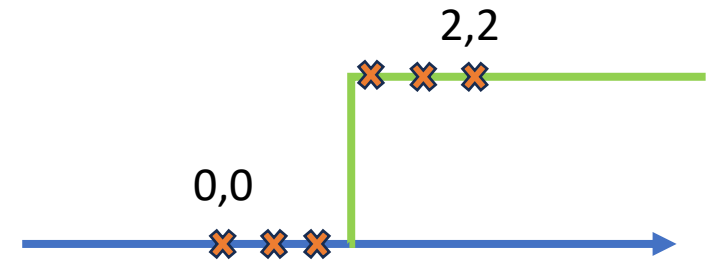
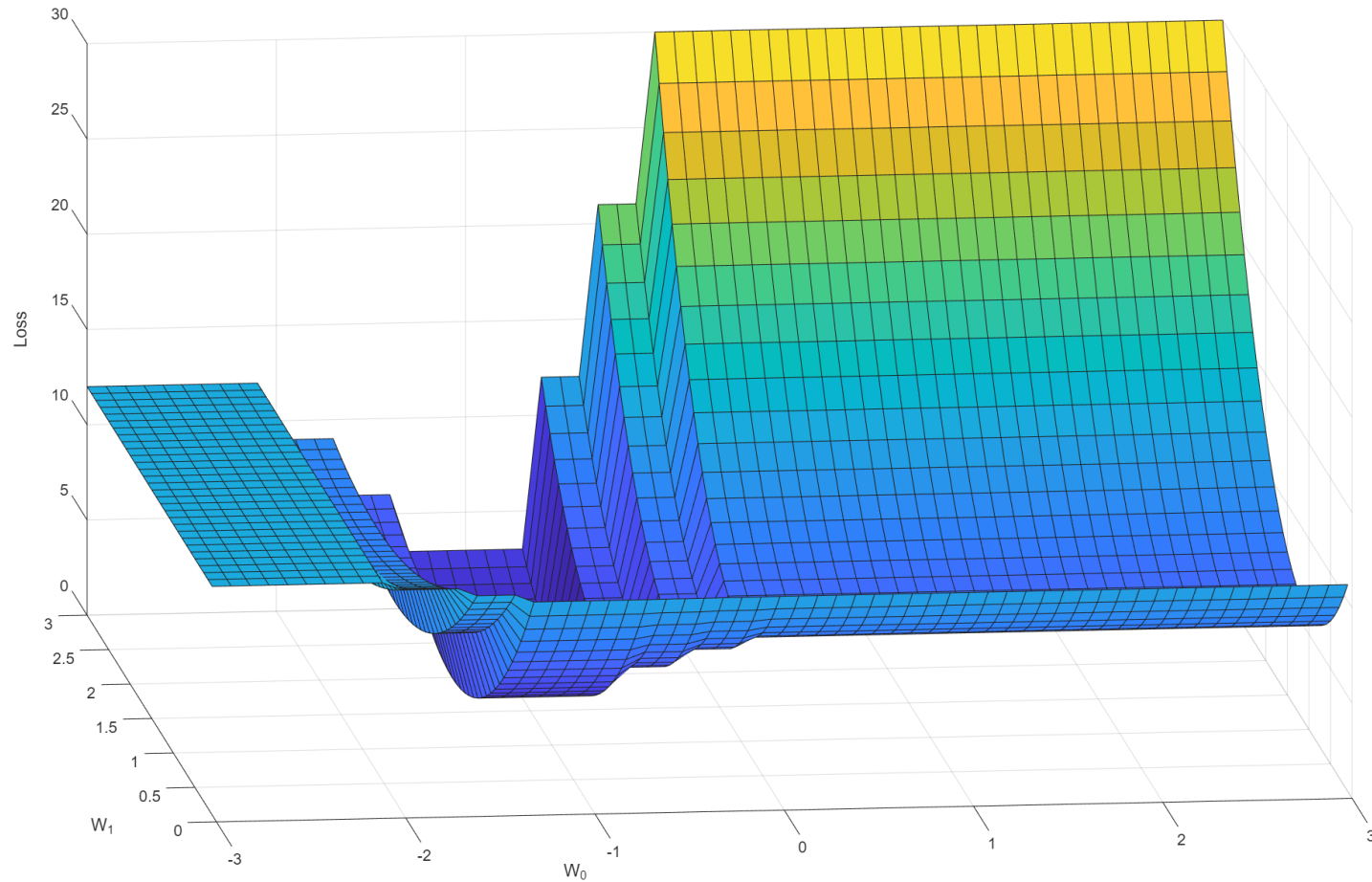


- How does this loss function behave w.r.t. the weights?
- Is it differentiable w.r.t. the weights?

Loss Surface



Loss Surface *Smoothens* with More Data



Challenges

- How to find optimal values?
 - Find roots of derivative expression
- Derivatives should exist
- Roots: Move along the curve to find a flat region
 - Convex surface? Unique solution? Local minima?
 - Initialization point?

Activation Functions

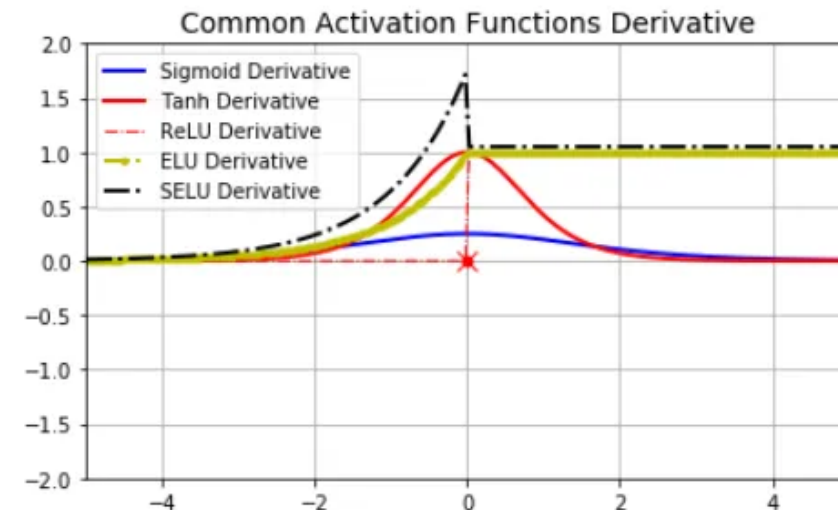
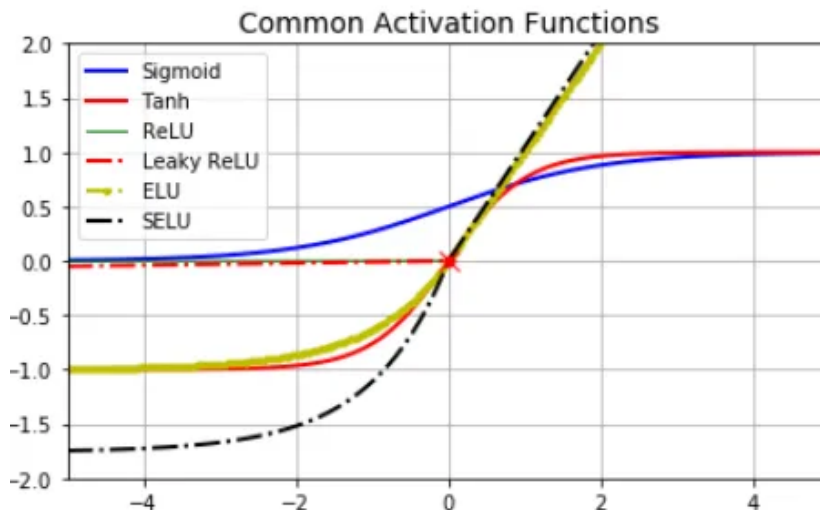
- Step function has derivative = infinity at zero-input

- Options: Sigmoid, softmax, tanh (odd symmetry)

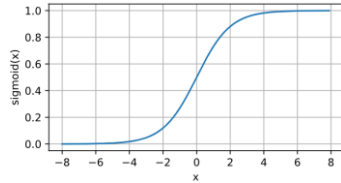
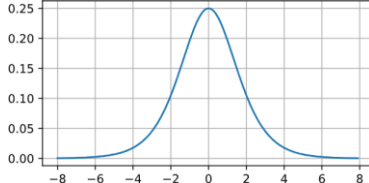
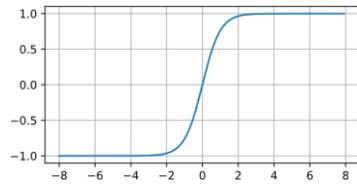
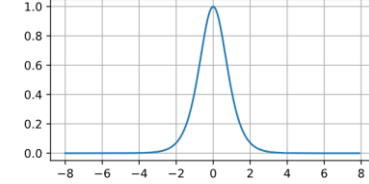
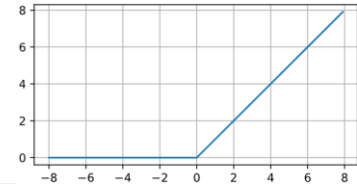
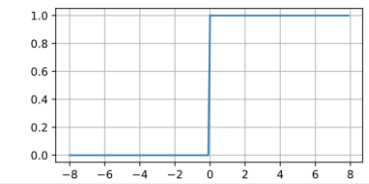
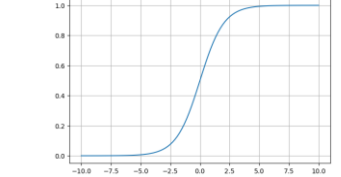
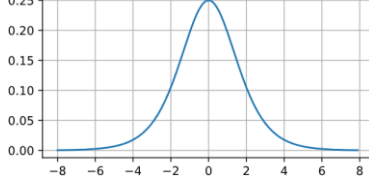
- Non-limiting activations:

- ReLU (rectified linear unit) & SeLU (scaled exponential linear unit)

$$\text{selu}(x) = \lambda \begin{cases} x & \text{if } x > 0 \\ \alpha e^x - \alpha & \text{if } x \leq 0 \end{cases}$$



Some Common Activation Functions

Function	Expression	Plot	Derivative
Sigmoid	$\frac{1}{1 + \exp(-x)}$		
Tanh	$\frac{1 - \exp(-2x)}{1 + \exp(-2x)}$		
ReLU	$\max(x, 0)$		
Softmax	$\frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$		

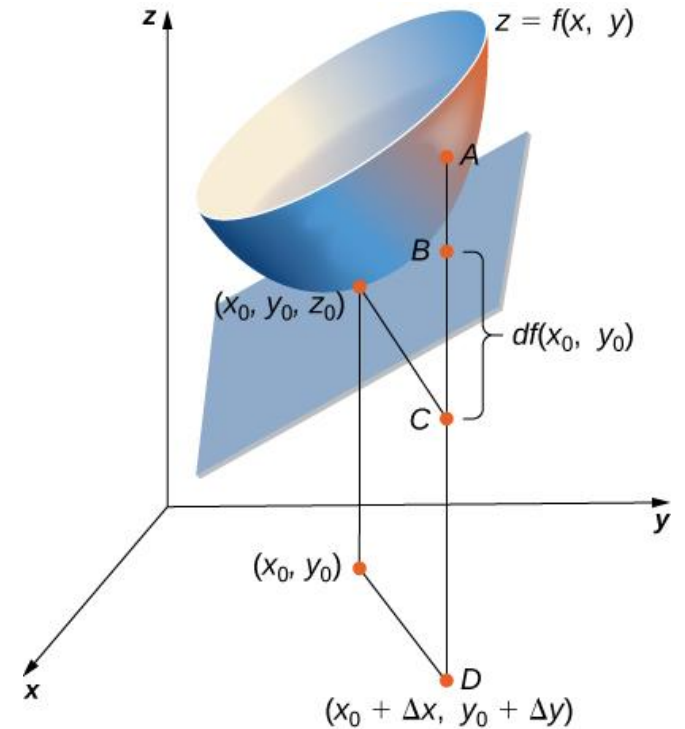
Training Neural Networks

Gradient Descent

- How to find the roots of the derivative of the loss function?
- Taylor approximation in higher dimension
 - **Gradient**: Linear approximation of a function at a point
 - **Hessian**: Curvature of a function at a point
- $L(\mathbf{w} + \boldsymbol{\varepsilon}) \approx L(\mathbf{w}) + \nabla L(\mathbf{w})^T(\mathbf{w} + \boldsymbol{\varepsilon} - \mathbf{w})$
 - For $L(\mathbf{w} + \boldsymbol{\varepsilon}) - L(\mathbf{w}) < 0$, pick $\boldsymbol{\varepsilon}$ opposite to $\nabla L(\mathbf{w})$

Gradient Descent

- Gradient descent: $\mathbf{w}_{n+1} = \mathbf{w}_n - \eta \nabla L(\mathbf{w}_n)$
 - $L(\mathbf{w}_0) \geq L(\mathbf{w}_1) \geq \dots \geq L(\mathbf{w}_n) \geq \dots$
 - η : step size or **learning rate**
- How to find gradient w.r.t to all weights?
- What is a good starting $\mathbf{w}_0$?
- What is a good learning rate?



Backpropagation

- For input-label pair (x, y) and cost/loss function $C(\cdot)$

$$C(y, f^L(W^L f^{L-1}(W^{L-1} \dots f^2(W^2 f^1(W^1 x)) \dots)))$$

- The gradient is $\frac{dC}{da^L} \cdot \frac{da^L}{dz^L} \cdot \frac{dz^L}{da^{L-1}} \cdot \frac{da^{L-1}}{dz^{L-1}} \cdot \frac{dz^{L-1}}{da^{L-2}} \dots \frac{da^1}{dz^1} \cdot \frac{\partial z^1}{\partial x}$

$$\nabla_x C = (W^1)^T \cdot (f^1)' \circ \dots \circ (W^{L-1})^T \cdot (f^{L-1})' \circ (W^L)^T \cdot (f^L)' \circ \nabla_{a^L} C$$

- At each layer, $(f^1)' \circ (W^2)^T \cdot (f^2)' \circ \dots \circ (W^{L-1})^T \cdot (f^{L-1})' \circ (W^L)^T \cdot (f^L)' \circ \nabla_{a^L} C$
 $(f^2)' \circ \dots \circ (W^{L-1})^T \cdot (f^{L-1})' \circ (W^L)^T \cdot (f^L)' \circ \nabla_{a^L} C$

$$(f^{L-1})' \circ (W^L)^T \cdot (f^L)' \circ \nabla_{a^L} C$$

$$(f^L)' \circ \nabla_{a^L} C,$$

Backpropagation with 2-layers

$$\text{Loss} = L\{f_2(\underbrace{\mathbf{W}_2}_{=\mathbf{b}_2} \underbrace{f_1(\underbrace{\mathbf{W}_1 \mathbf{x}}_{=\mathbf{a}_1})}_{=\mathbf{a}_2}); y\}$$

$$\frac{\partial L}{\partial \mathbf{W}_2} = \mathbf{b}_1 \left(f'_2 \cdot \frac{\partial L}{\partial \mathbf{b}_2} \right)^T$$
$$\frac{\partial L}{\partial \mathbf{W}_1} = \mathbf{x} \left(f'_1 \cdot \mathbf{W}_2^T f'_2 \cdot \frac{\partial L}{\partial \mathbf{b}_2} \right)^T$$

Backpropagation

- Compute $\mathbf{g} = \nabla_{\text{output}} L$
- At layer i , (starting from the last layer)
 - $\mathbf{g} = f'_i \mathbf{W}_{i+1}^T \mathbf{g}$
 - Gradient = (current layer input) $\mathbf{g}^T$
 - Repeat till first layer
- How to get $\nabla_{\text{output}} L$?

Initialization

- Can all weights be initialized to 0?
- Why we need random initialization?
- Example: 2 output nodes and 1 input node
 - $y_1 = f(w_1x + b_1)$
 - $y_2 = f(w_2x + b_2)$
 - Gradients are similar

Initialization

- Control variance of randomness
 - Xavier: For tanh, sigmoid - $\mathcal{N}(0, 2/n_{l+1}+n_{l-1})$
 - $\text{Var}(\text{output at } l\text{th layer}) = n_{\text{input}} \sigma_w \text{Var}(\text{output at } l-1\text{th layer})$
 - $\text{Var}(\text{grad at } l\text{th layer}) = n_{\text{output}} \sigma_w \text{Var}(\text{grad at } l+1\text{th layer})$
 - Kaiming He: For ReLU - $\mathcal{N}(0, 2/n_{l-1})$
 - Almost half of the output of ReLU is zero
 - LeCun: For SELU - $\mathcal{N}(0, 1/n_{l-1})$
 - Random orthogonal matrix

AutoGrad

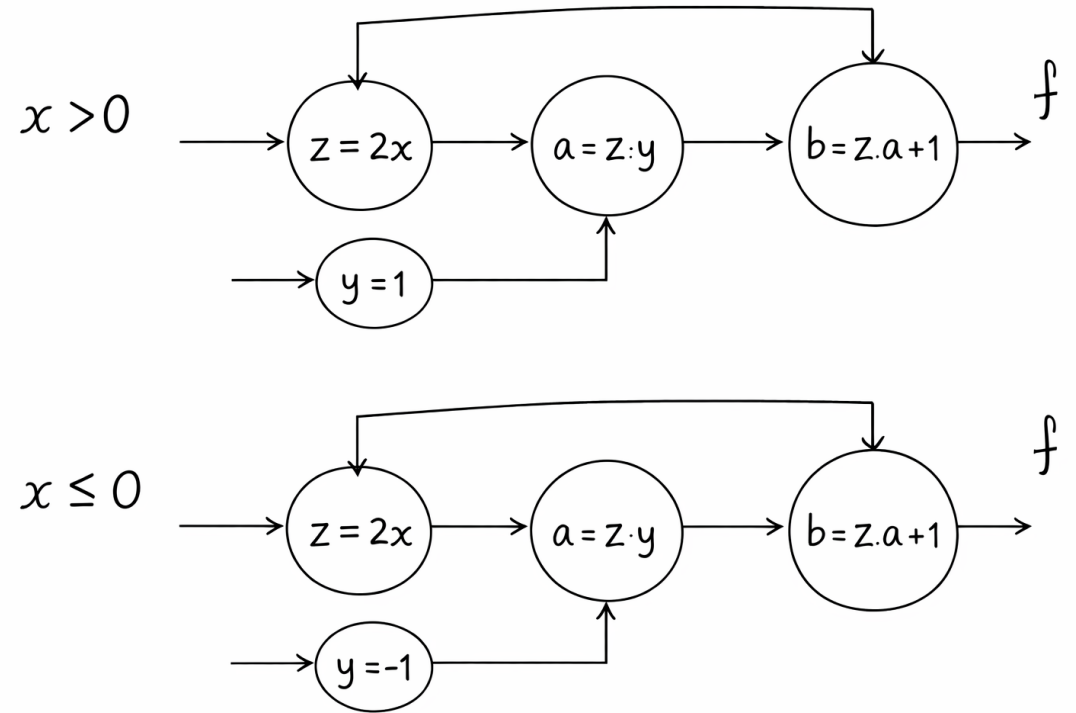
- How to compute gradients of arbitrary models & losses?
- Gradient of a function (in programming) can be computed using **autograd**
 - Exact gradient, not numerical gradient
 - No cycles
 - AG Baydin et al, *Automatic Differentiation in Machine Learning: a Survey*
- **Forward pass**: compute the output value of all nodes
- **Backward pass**: compute the gradients using the forward pass values and chain rule of derivatives
- Implemented using **computational graphs**

Computational Graph

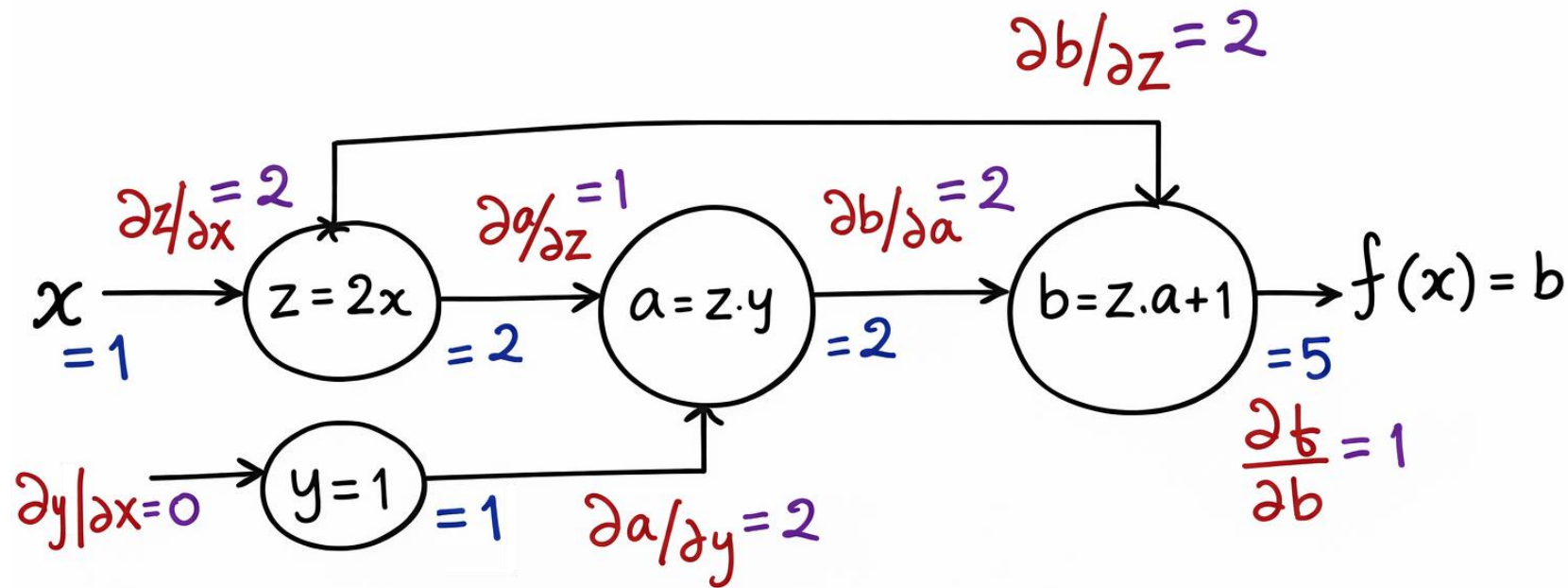
- How to compute gradients of arbitrary models & losses?
- Computational graph: a **directed acyclic graph** representing all the computations between input and output
 - **Dynamic** (Loops and conditional statements)
 - **Static**
 - Output of each (intermediate) computation is a variable or node
 - Function of input variables
 - No cycles (no variable is reused in the graph)

Example Computational Graph

```
def function(x):  
    if x>0:  
        y = 1  
    else:  
        y = -1  
    z = 2*x  
    for i in range(2):  
        y = z*y + i  
    return y
```



Example Computational Graph



$$\frac{\partial f}{\partial b} = 1$$

$$\frac{\partial b}{\partial z} = a$$

$$\frac{\partial a}{\partial z} = y$$

$$\frac{\partial z}{\partial x} = 2$$

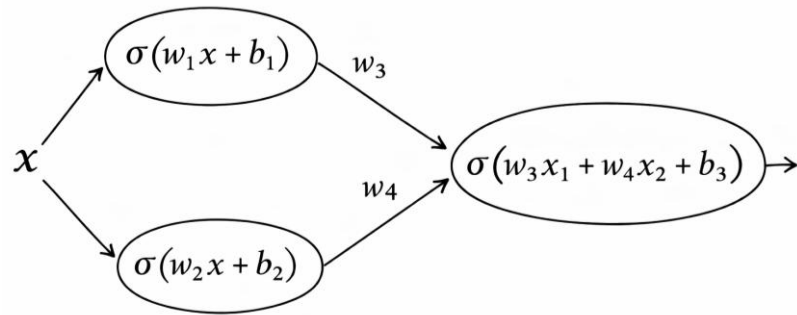
$$\frac{\partial b}{\partial a} = z$$

$$\frac{\partial a}{\partial y} = z$$

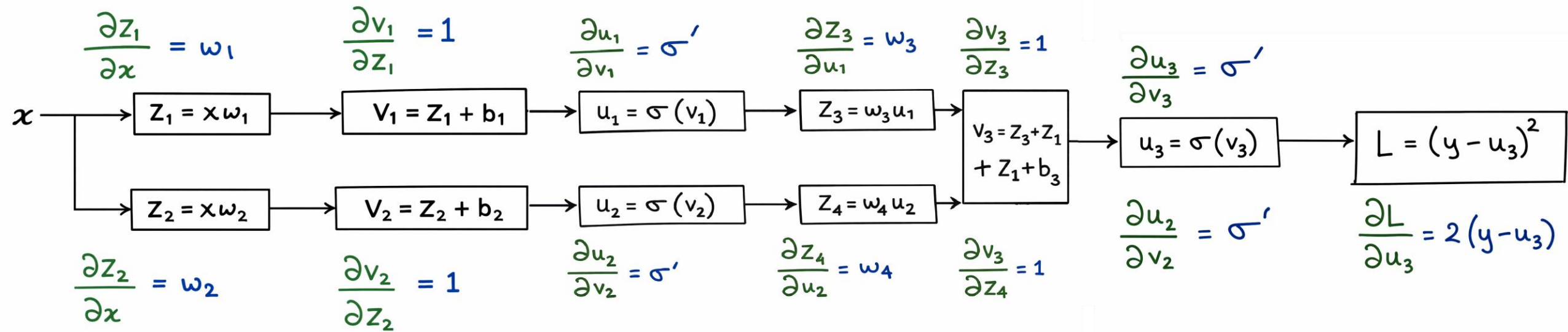
$$\frac{\partial y}{\partial x} = 0$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial b} \left[\frac{\partial b}{\partial z} \cdot \frac{\partial z}{\partial x} + \frac{\partial b}{\partial a} \cdot \left(\frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial x} + \frac{\partial a}{\partial y} \cdot \frac{\partial y}{\partial x} \right) \right] = 2(a + yz)$$

Backpropagation Example



$$\frac{\partial L}{\partial w_3} = \frac{\partial L}{\partial u_3} \frac{\partial u_3}{\partial w_3} = \frac{\partial L}{\partial u_3} \sigma' \frac{\partial v_3}{\partial w_3} = \frac{\partial L}{\partial u_3} \sigma' \frac{\partial v_3}{\partial z_3} u_1$$



Backpropagation

- Autograd computes gradient through backpropagation
 - Gradients *flow* from one node to another
 - Recursion and matrix operations can have derivatives
 - M. Seeger et al, Auto-Differentiating Linear Algebra, 2019
- Gradients do not flow: **vanishing** or **exploding** cases – finite precision
- Autograd fails for
 - Non-differentiable functions
 - Improper computational graph
 - e.g., `if x = 2 then 4 else x * x`, autograd output at 2 = 0 instead of 4
 - Python: array assignment (<https://github.com/HIPS/autograd/blob/master/docs/tutorial.md>)

Learning NN parameters $\mathbf{w}$ – Training Phase

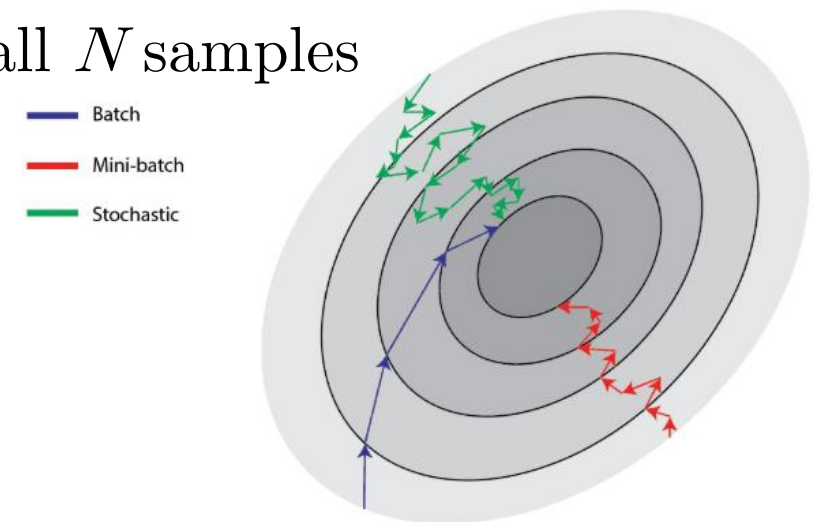
- Initialize $\mathbf{w}_0$: Xavier, K. He, LeCun, etc., and compute loss -- *forward pass*
- For $n = 1$ to till convergence; Dataset with N samples,
 - Compute/estimate gradient: $\mathbf{g}_n = \sum_i \nabla_{\mathbf{w}_{n-1}} L_i / N$
 - For each i , compute $\nabla_{\mathbf{w}_{n-1}} L_i$ using backpropagation (autograd) -- *backward pass*
 - SGD, SAG, SAGA
 - Compute learning rate: η
 - AdaGrad, RMSprop, Adam
 - Update parameters: $\mathbf{w}_n = \mathbf{w}_{n-1} - \eta \mathbf{g}_n$
 - First order, second order
 - Compute loss: $L(\text{label}, \text{Model}\{\mathbf{w}_{n+1}, \text{input}\})$ -- *forward pass*
- End

Stochastic Gradient Descent

- What if number of samples is large? Compute N gradients in each step?
- SGD – at each descent step, find gradient for only one random sample:
 - $\mathbf{w}_n = \mathbf{w}_{n-1} - \eta \nabla_{\mathbf{w}} L_j$
 - j is randomly chosen from $\{1, \dots, N\}$ at every descent step:
 - Estimated gradient is a random (stochastic) sample: $\nabla_{\mathbf{w}} L_j = \nabla_{\mathbf{w}} L + z = \text{average (true) gradient} + \text{noise}$
 - Note that, for large N , $\sum_i \nabla_{\mathbf{w}} L_i / N \rightarrow \mathbb{E}\{\nabla_{\mathbf{w}} L\}$
 - For correlated data samples, $\nabla_{\mathbf{w}} L_j$ could be close to average
 - Uncorrelated samples (noisy gradient) may lead to divergence (variance of z is high)
 - Very slow convergence $O(1/\sqrt{n})$

Stochastic Gradient Descent - Batched

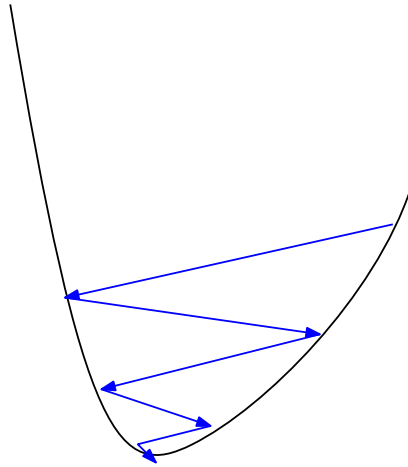
- Improve average gradient estimate through mini-batches
 - SGD with minibatch: $\mathbf{w}_n = \mathbf{w}_{n-1} - \eta \sum_{k=1 \text{ to } M} \nabla_{\mathbf{w}} L_{j_k} / M$
 - j_k is randomly chosen from $\{1, \dots, N\}$, for $k = 1$ to M , at every descent step
 - Batch size: $M \ll N$, reduces variance; converges at $\mathcal{O}(1/n)$ – theoretical best
- **Epoch**: Set of descent steps required to go through all N samples
 - GD: 1 epoch = 1 descent step
 - SGD: 1 epoch = N descent steps
 - SGD-minibatch: 1 epoch = N/M descent steps



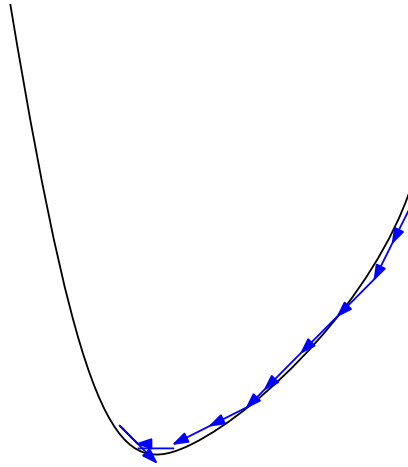
Stochastic Average Gradient

- Can we improve SGD without a mini-batch?
 - SAG: $\mathbf{w}_{n+1} = \mathbf{w}_n - \eta(\nabla_{\mathbf{w}}L_j^{(n)} - \nabla_{\mathbf{w}}L_j^{(n-1)} + \sum_i \nabla_{\mathbf{w}}L_i^{(n-1)})/N$
 - At step $n = 0$, set $\nabla_{\mathbf{w}}L_i^{(0)} = 0$ and compute average gradient as $\sum_i \nabla_{\mathbf{w}}L_i^{(0)}/N$
 - At step n , j is randomly chosen from $\{1, \dots, N\}$ and only $\nabla_{\mathbf{w}}L_j^{(n)}$ is evaluated
 - Update average gradient as $(\nabla_{\mathbf{w}}L_j^{(n)} + \sum_{i \neq j} \nabla_{\mathbf{w}}L_i^{(n-1)})/N$
 - For all i , at each step, store gradients $\nabla_{\mathbf{w}}L_i^{(n)}$
 - However, this average gradient is a biased estimate
- SAGA (SAG Accelerated): $\mathbf{w}_{n+1} = \mathbf{w}_n - \eta(\nabla_{\mathbf{w}}L_j^{(n)} - \nabla_{\mathbf{w}}L_j^{(n-1)} + \sum_i \nabla_{\mathbf{w}}L_i^{(n-1)})/N$
 - Unbiased estimate of the average gradient; converges at $O(1/n)$
 - Defazio et al “SAGA: A Fast Incremental Gradient Method With Support for Non-Strongly Convex Composite Objectives”

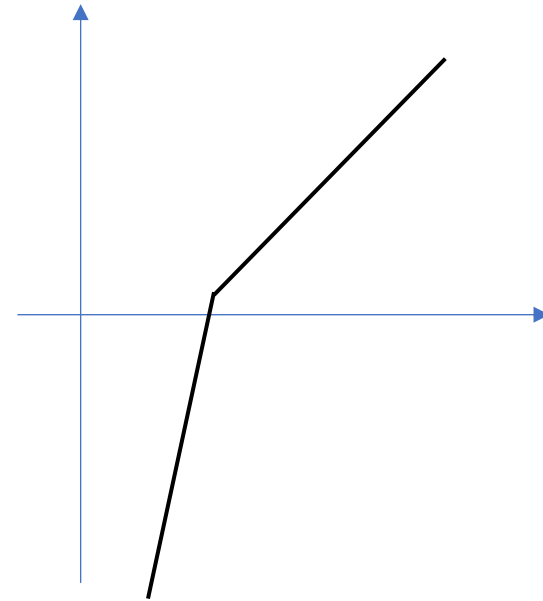
Effect of Learning Rate



Large step size



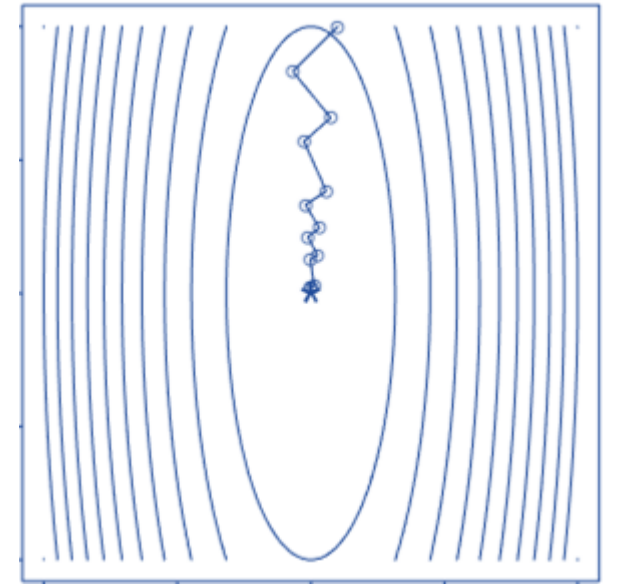
Small step size



Gradient

Optimal Learning rate – Steepest Descent

- Optimal $\eta^* = \operatorname{argmin}_{\eta} L(\mathbf{w}_{n-1} - \eta \nabla \mathbf{w}_{n-1} L) = \operatorname{argmin}_{\eta} L(\mathbf{w}_{n-1} - \eta \mathbf{g}_n)$
 - Not always computable
- At optimal η^* , $\nabla_{\eta} L(\mathbf{w}_{n-1} - \eta \mathbf{g}_n) = 0$
 - That is, $\nabla L(\mathbf{w}_n)^T \mathbf{g}_n = \mathbf{g}_{n+1}^T \mathbf{g}_n = 0$
- Consecutive gradients are orthogonal
 - Slow convergence, undoing previous minimizations
 - Fix: differently adapting learning rate or gradient



Adapting Learning Rate

- Linear decay: $\eta_n = (1 - \alpha)\eta_0 + \alpha\eta_t$
 - t : terminal rate instance; $\alpha = \min(1, n/t)$
- AdaGrad: $\eta_n = \alpha / (\epsilon + \sqrt{\mathbf{h}_n})$; $\mathbf{h}_n = \mathbf{h}_{n-1} + (\sum_i \nabla_w L_i^{(n)} / N)^2 = \mathbf{h}_{n-1} + \mathbf{g}_n^2$
 - Change η depending on the **magnitude of slope** of each coordinate in the gradient
 - Element-wise learning rate, ϵ is for numerical stability
 - Reduce learning rate when **accumulated gradient** is large and vice versa
 - (+) Good for sparse gradients
 - (−) decreases learning rate early – vanishing; problem when gradient is noisy
- RMSprop: $\eta_n = \alpha / (\epsilon + \sqrt{\mathbf{h}_n})$; $\mathbf{h}_n = \beta \mathbf{h}_{n-1} + (1 - \beta)(\sum_i \nabla_w L_i^{(n)} / N)^2 = \beta \mathbf{h}_{n-1} + (1 - \beta)\mathbf{g}_n^2$
 - Accumulation is averaged to prevent early decrease in learning rate

Adaptive Moment Estimation

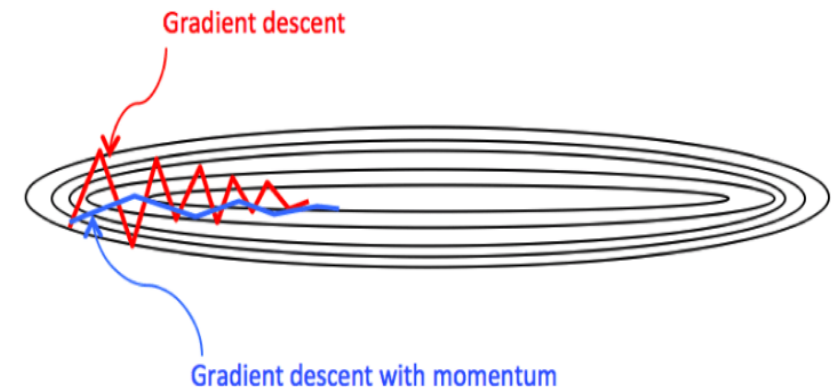
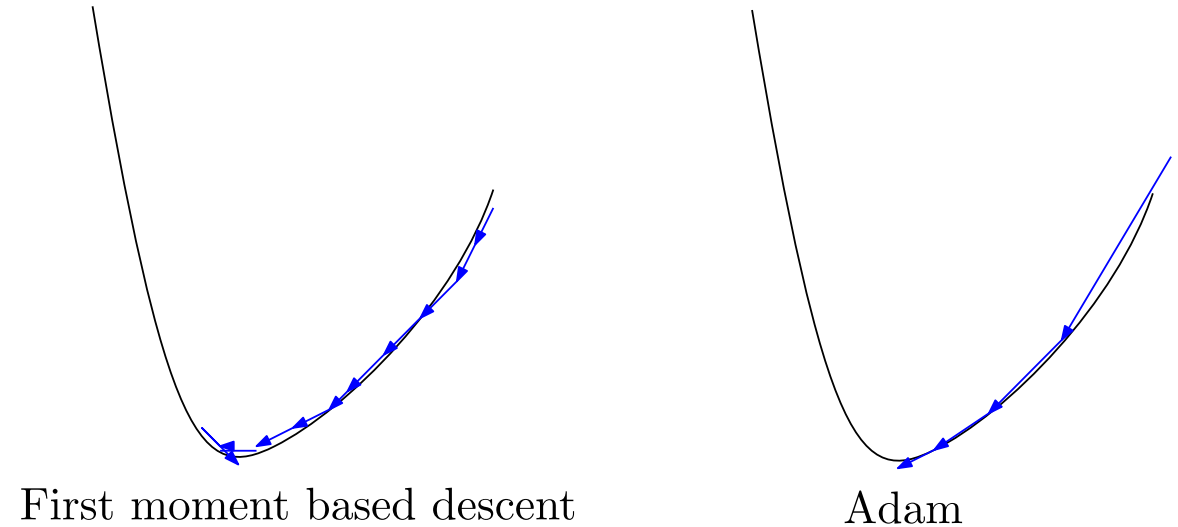
- AdaGrad & RMS prop compute root sum square & RMS of gradients, respectively
 - Results in learning rate decay as n increases and gradient magnitude accumulates
 - However, the actual direction of gradient is not used to increase or decrease learning rate
- Compute **first moment**: $\mathbf{m}_n = \beta_1 \mathbf{m}_{n-1} + (1 - \beta_1) \mathbf{g}_n$
 - β_1 is less than 1 (usually very close to 1)
 - $\mathbf{m}_n$ is **weighted moving average** of gradients – **smoother** trajectory (thus, skips small local minima)
 - For $\mathbf{m}_0 = 0$ and a constant gradient $\mathbf{g}_n = \mathbf{g}$, $\mathbf{m}_n \rightarrow \beta_1 \mathbf{g}_n$
 - Biased towards zero (as initial point is 0); **unbiased** moment = $\mathbf{m}_n / (1 - \beta_1^n)$
 - Builds **momentum** that helps to move out of small wells

Adam Optimizer

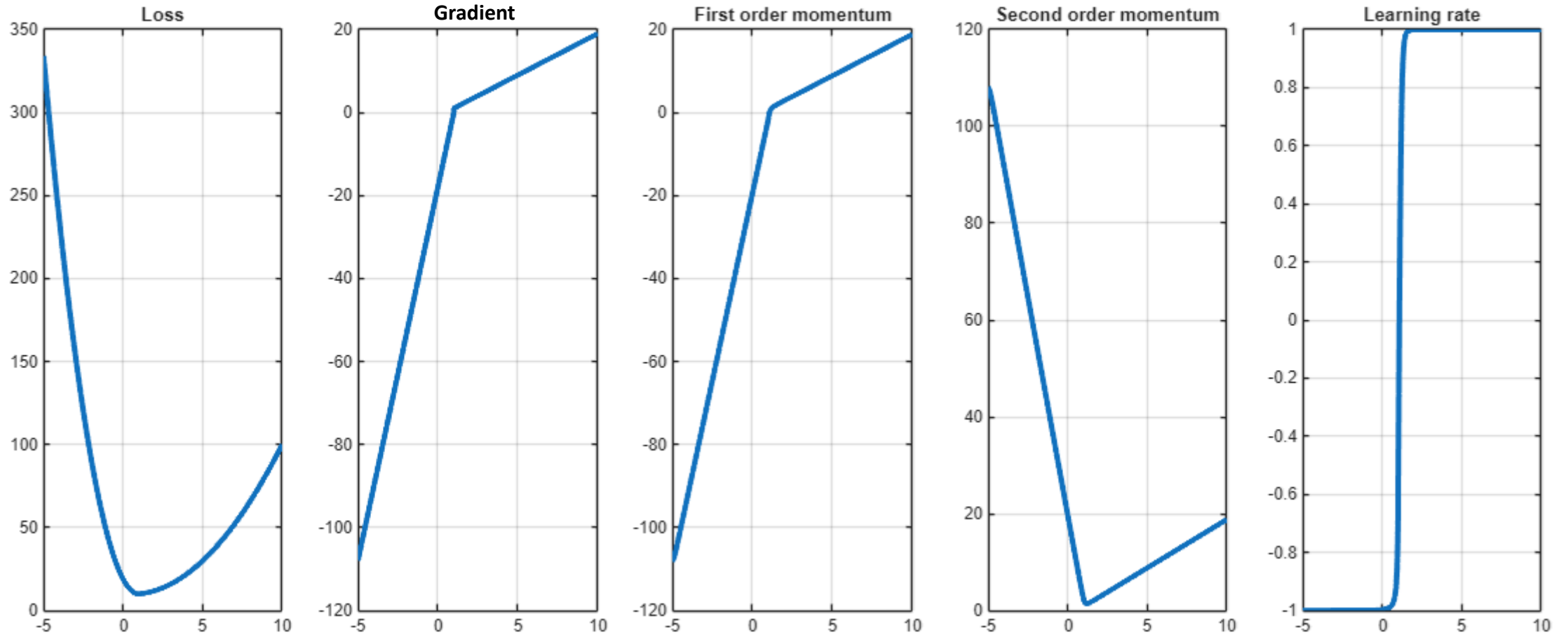
- Large momentum can lead to moving past a significant local minima
- A solution: When the variance of the gradient is high, take smaller steps to not miss local minima and vice versa
- Compute **non-central second moment**: $\mathbf{v}_n = \beta_2 \mathbf{v}_{n-1} + (1 - \beta_2) \mathbf{g}_n^2$
 - β_2 is less than 1 (usually very close to 1)
 - $\mathbf{v}_n$ is **weighted moving average** of the (uncentered) variance of the gradients
 - Biased towards zero; **unbiased** moment $\mathbf{s}_n = \sqrt{(\mathbf{v}_n / (1 - \beta_2^n))}$
 - Divide learning rate by $(\mathbf{s}_n + \epsilon)$
 - Helps to **reduce oscillations**

Adam Optimizer

- Adam
 - Compute $\mathbf{m}_n$ and $\mathbf{k}_n = \mathbf{m}_n / (1 - \beta_1^n)$
 - Compute $\mathbf{v}_n$ and $\mathbf{s}_n = \sqrt{\mathbf{v}_n / (1 - \beta_2^n)}$
 - Update $\mathbf{w}_{n+1} = \mathbf{w}_n - \eta \mathbf{k}_n / (\mathbf{s}_n + \epsilon)$
 - ϵ – to avoid division by zero
 - D. P. Kingma et al, *Adam: A Method for Stochastic Optimization*
- Learning rate is adapted coordinate-wise
- Multiple variants of Adam exist in literature

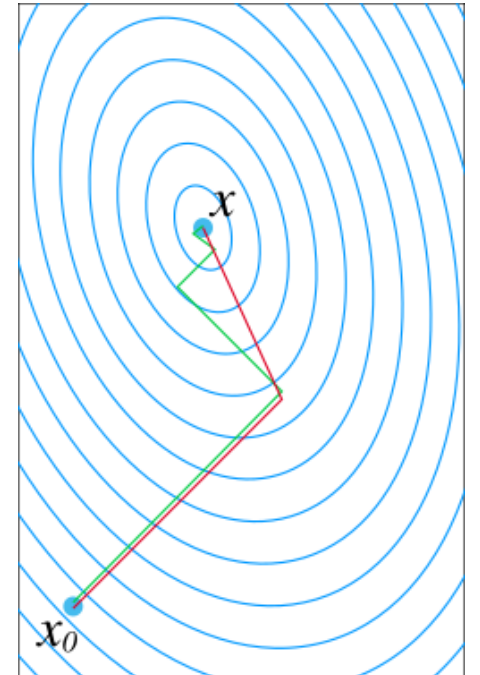


Adam Method



Conjugate Gradient Descent

- Another option is to change the gradient computation instead of LR
- At step n , if $\mathbf{d}_n$ is the direction along which we are descending
 - Avoid $\mathbf{d}_n^T \mathbf{d}_{n-1} = 0$, by setting $\mathbf{d}_n = \mathbf{g}_n + \beta \mathbf{d}_{n-1}$
 - Such that they are **conjugate directions**: $\mathbf{d}_n^T \mathbf{H} \mathbf{d}_{n-1} = 0$
 - $\mathbf{H}$ is the Hessian matrix; optimal directions: eigenvectors of $\mathbf{H}$
 - Substituting, we get $\beta = -\mathbf{g}_n^T \mathbf{H} \mathbf{d}_{n-1} / \mathbf{d}_{n-1}^T \mathbf{H} \mathbf{d}_{n-1}$
 - **Fletcher-Reeves approximation**: $\beta = -\mathbf{g}_n^T \mathbf{g}_{n-1} / \mathbf{g}_{n-1}^T \mathbf{g}_{n-1}$
 - Update step: $\mathbf{w}_n = \mathbf{w}_{n-1} - \eta \mathbf{d}_n$



Second-Order: Newton's Method

- $L(\mathbf{w} + \boldsymbol{\epsilon}) \approx L(\mathbf{w}) + \nabla L(\mathbf{w})^T(\mathbf{w} + \boldsymbol{\epsilon} - \mathbf{w}) + \boldsymbol{\epsilon}^T \mathbf{H} \boldsymbol{\epsilon} / 2$
 - Quadratic approximations of loss instead of linear
- Similarly, $\nabla L(\mathbf{w} + \boldsymbol{\epsilon}) \approx \nabla L(\mathbf{w}) + \mathbf{H} \boldsymbol{\epsilon}$
 - $\mathbf{H} \boldsymbol{\epsilon}$ is the **change of gradient** along the direction $\boldsymbol{\epsilon}$
 - $\mathbf{v}^T \mathbf{H} \boldsymbol{\epsilon} = 0$: $\mathbf{v}$ is direction orthogonal to the change of gradient
 - If the gradient at the next step has to be zero, then $-\boldsymbol{\epsilon} \approx \mathbf{H}^{-1} \nabla L(\mathbf{w})$
 - To prevent instability: $-\boldsymbol{\epsilon} = (\mathbf{H} + c\mathbf{I})^{-1} \nabla L(\mathbf{w})$
 - Update step: $\mathbf{w}_n = \mathbf{w}_{n-1} - \eta (\mathbf{H}_{\mathbf{w}_{n-1}} + c\mathbf{I})^{-1} \nabla L(\mathbf{w}_{n-1})$
 - Convergence: $\mathcal{O}(1/n^n)$ – superlinear (Non-asymptotic Global Convergence Analysis of BFGS with the Armijo-Wolfe Line Search, 2024)

Broyden-Fletcher-Goldfarb-Shanno Optimizer

- Change in weights: $\mathbf{d}_n = \mathbf{w}_n - \mathbf{w}_{n-1}$
- Change in gradient: $\mathbf{h}_n = \mathbf{g}_n - \mathbf{g}_{n-1}$
- Hessian at that location: $\mathbf{H}_n \mathbf{d}_n \approx \mathbf{h}_n$
 - Can we find a matrix $\mathbf{H}_n$ that satisfies this equation?
 - Constraints: symmetric, positive definite, close to $\mathbf{H}_{n-1}$
 - Solve $\operatorname{argmin} \|\mathbf{A} - \mathbf{H}_{n-1}\|$ such that $\mathbf{A} = \mathbf{A}^T$ and $\mathbf{A} \mathbf{d}_n = \mathbf{h}_n$
 - Solution: $\mathbf{H}_n^{-1} \approx \mathbf{B}_n = \mathbf{B}_{n-1} + \mathbf{d}_{n-1} \mathbf{d}_{n-1}^T / \mathbf{h}_{n-1}^T \mathbf{d}_{n-1} - \mathbf{B}_{n-1} \mathbf{h}_{n-1} \mathbf{h}_{n-1}^T \mathbf{B}_{n-1} / \mathbf{h}_{n-1}^T \mathbf{B}_{n-1} \mathbf{h}_{n-1}$
 - Initialization: $\mathbf{B}_n = \mathbf{0}$; Update step: $\mathbf{w}_n = \mathbf{w}_{n-1} - \eta \mathbf{B}_{n-1} \nabla L(\mathbf{w}_{n-1})$

Nesterov Optimization

- Evaluate gradient/Hessian near the update point
 - Predict the next point and find the gradient there: **gradient look-ahead**
- Compute $\nabla_{\mathbf{w}_{n-1} + \mu \mathbf{r}_{n-1}} L_i$ instead of $\nabla_{\mathbf{w}_{n-1}} L_i$
 - $\mathbf{r}_n$ is the parameter update term such that $\mathbf{w}_n = \mathbf{w}_{n-1} + \mathbf{r}_n$
 - Convergence rate: $O(1/n^2)$ – first order
- Applicable to all the methods discussed so far and later
 - **NAdam** is popular

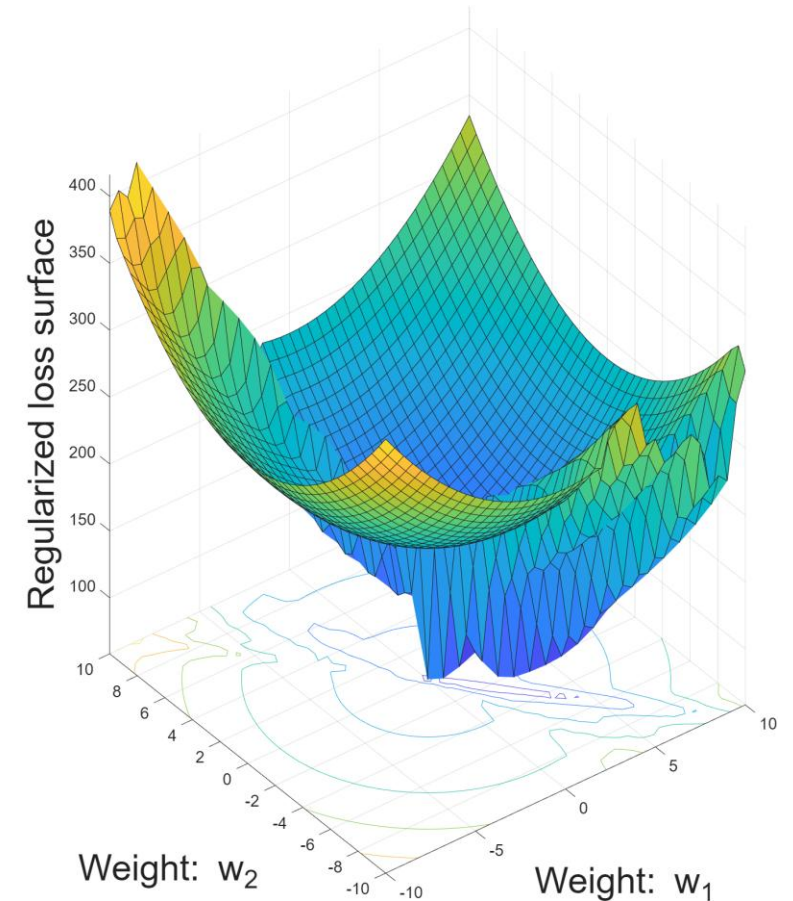
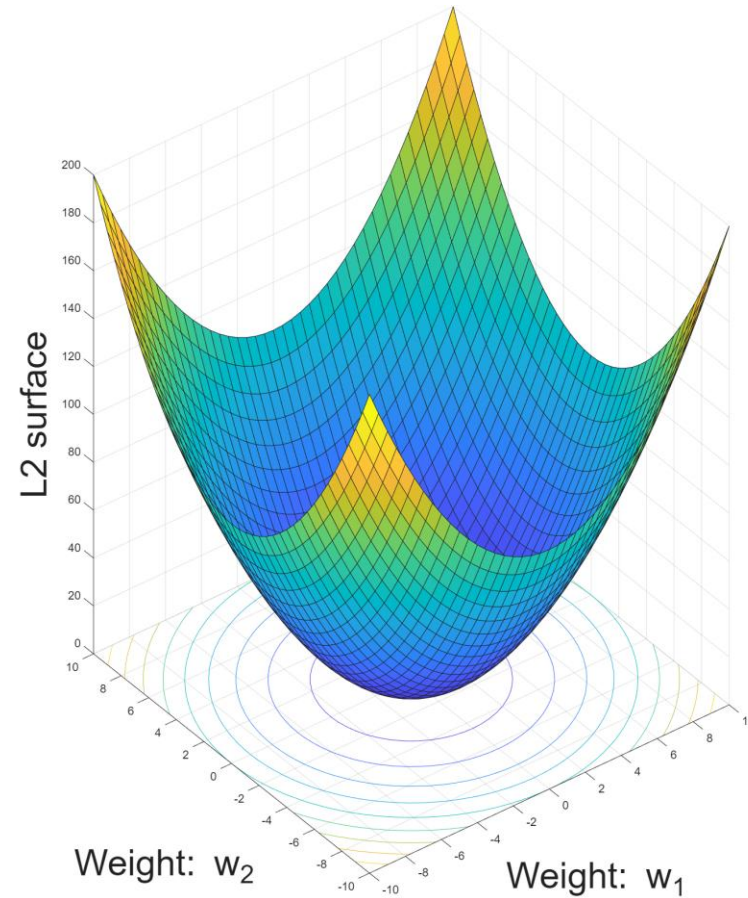
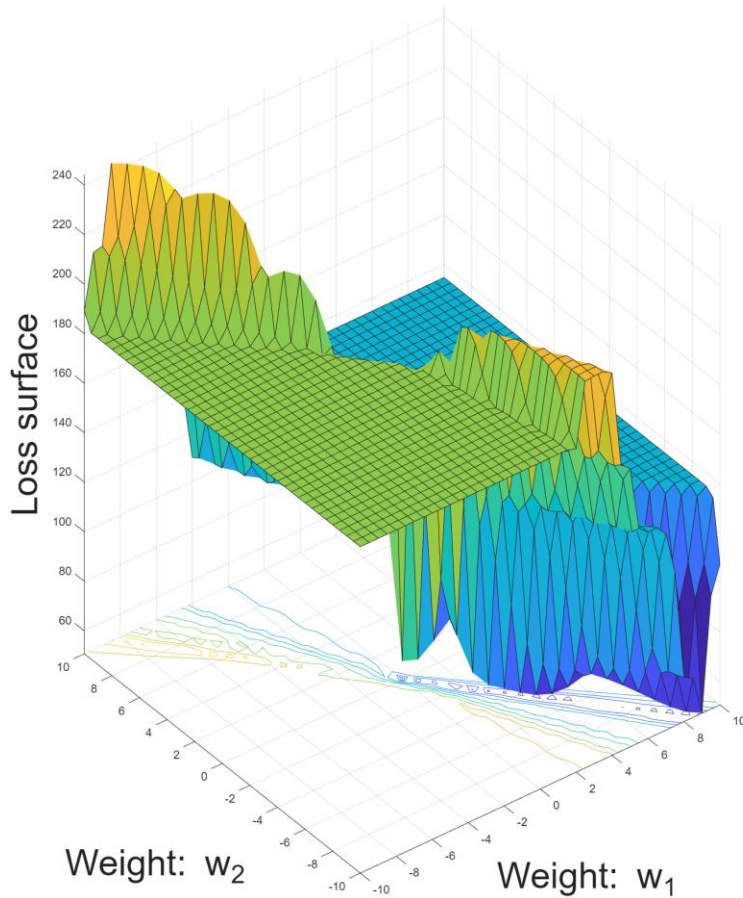
Generalization Performance

- Does having more layers/neurons always help?
 - Increasing depth/width = increasing number of parameters
 - Recall model selection & bias-variance tradeoff
- Underfitting: training loss is plateauing/increasing
- Overfitting
 - Decreasing training loss, but plateauing/increasing validation loss
- How to get good generalization? Avoid overfitting (memorization)?

Regularization

- Improve weight/parameter selection through prior information
 - Adding constraints on the parameters
- **Weight decay** (only for weights, not biases)
 - Loss function: $L(\mathbf{w}) + \lambda \|\mathbf{w}\|^2/2$ -- Smoothens the loss surface
 - **L2 or Tikhonov regularization, ridge regression**, push weights towards zero
 - Update step: $\mathbf{w}_n = (1 - \eta\lambda) \mathbf{w}_{n-1} - \eta\mathbf{g}_{n-1}$
 - L1 regularization as used for sparse weights/parameters
- **Parameter sharing**: Constrain the weights to be similar to another fixed pretrained neural network: $L(\mathbf{w}) + \lambda \|\mathbf{w} - \mathbf{w}_{\text{fixed}}\|^2$ -- fine tuning

Visualizing Loss Function Smoothing



Data Augmentation

- As we saw earlier, increasing training dataset size improves loss surface & generalization
- **Noise injection**/perturbation: Add noise to training data
 - Adding Gaussian noise ($\epsilon \sim \mathcal{N}(0, \sigma^2)$) to input has the same behavior as L2 regularization!
 - Illustration: $\text{Loss} = \mathbb{E}(\mathbf{y} - \mathbf{w}^T(\mathbf{x} + \epsilon))^2 = \mathbb{E}(\mathbf{y} - \mathbf{w}^T\mathbf{x})^2 + \sigma^2 \|\mathbf{w}\|^2$
 - $\mathbb{E}(\mathbf{y} - \mathbf{w}^T(\mathbf{x} + \epsilon))^2 = \mathbb{E}(\mathbf{y} - \mathbf{w}^T\mathbf{x})^2 + \mathbb{E}(\mathbf{w}^T\epsilon)^2 - \mathbb{E}(2(\mathbf{y} - \mathbf{w}^T\mathbf{x}) \mathbf{w}^T\epsilon)$
 - $\mathbb{E}(2(\mathbf{y} - \mathbf{w}^T\mathbf{x}) \mathbf{w}^T\epsilon) = \mathbb{E}(\mathbf{a}^T\epsilon) = 0; \quad \mathbb{E}(\mathbf{w}^T\epsilon)^2 = \mathbf{w}^T \mathbb{E}(\epsilon\epsilon^T)\mathbf{w} = \sigma^2 \|\mathbf{w}\|^2$

Label smoothing

- Change 0/1 labels as ε and $1 - \varepsilon$ (label noise)
 - One hot encoded label: $\mathbf{y} = [0, \dots, 1, \dots, 0]^T$
 - If softmax, $\sigma_k(\mathbf{w}^T \mathbf{x}) = 0$, then $\mathbf{w}^T \mathbf{x} \rightarrow -\infty$: **exploding gradient** and weights
 - Smoothen label by a convex combination of label and uniform distribution
 - Let labels be $\mathbf{z} = (1 - \varepsilon)\mathbf{y} + \varepsilon\mathbf{u} = [\varepsilon/K, \dots, 1 - \varepsilon + \varepsilon/K, \dots, \varepsilon/K]^T$
 - Cross entropy loss: $H(\mathbf{z}; \sigma_K(\mathbf{w}^T \mathbf{x})) = (1 - \varepsilon) H(\mathbf{y}; \sigma_K(\mathbf{w}^T \mathbf{x})) + \varepsilon H(\mathbf{u}; \sigma_K(\mathbf{w}^T \mathbf{x}))$
 - Make outputs to lie inside the simplex instead of on the vertices
 - Uniform distribution has finite distance from any PMF, so loss is never infinity – smoothens loss surface, thereby, decaying weights instead of exploding

Teacher Training and Knowledge Distillation

- Train a smaller (student) network using a larger (teacher) network
 - Knowledge from deep teacher net is distilled to a shallow student net
 - Teacher net is a conventional deep net with CE loss
 - Student's loss: $(1 - \epsilon) H(\text{input}; \text{label}) + \epsilon H(\text{input}; \text{teacher's output})$
 - Teacher's outputs are soft labels – smoothen; ϵ depends on [temperature parameter](#)
 - Another possible loss: $\text{MSE}(\text{input}; \text{teacher's logits})$
 - Better generalization; M. Phuong et al, *Towards Understanding Knowledge Distillation*

Semi-Supervised Learning

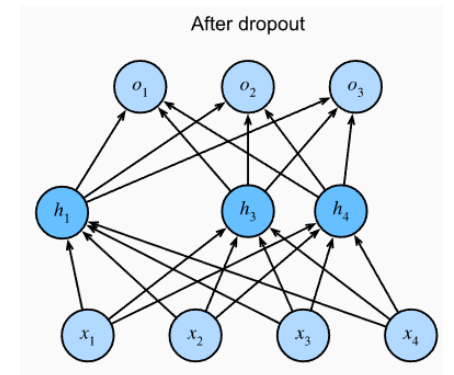
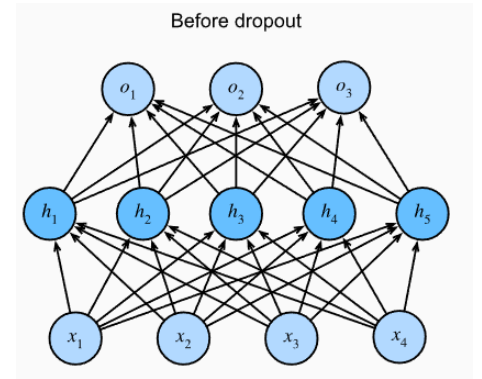
- Similar to data augmentation – generate new data samples instead of adding noise to existing samples
 - Unsupervised learning: Learn data distribution – joint PDF $P(\text{input}, \text{label})$ and sample from this distribution to get new samples
 - Examples: Histogram, clustering, kernel density estimation, etc.
 - **Autoencoders:** A non-linear PCA-like unsupervised learning
 - Encoder: $\mathbf{y} = f(\mathbf{x})$: high dimension $\rightarrow$ low dimension; decoder is vice versa
 - Train an autoencoder with given dataset; feed noise to decoder to generate new data

Bagging / Ensemble Learning

- **Bootstrapping**: Create multiple training datasets by drawing a subset of samples from the original dataset with replacement
- **Ensemble**: Train multiple neural networks with each of these datasets
- **Aggregation**
 - Classifier: Maximum voting based inference
 - Regression: Average all the outputs
- **Bagging**: bootstrap aggregation
 - If error of each network is $e_m(\mathbf{x}) = \mathbb{E}\{f_m(\mathbf{x}) - y(\mathbf{x})\}$,
then bagging error is $\mathbb{E}\{(f(\mathbf{x}) - y(\mathbf{x}))^2\} = \mathbb{E}\{\Sigma(e_m(\mathbf{x})/M)^2\}$

Dropout

- Another way to implement bagging without high computational complexity
 - Randomly drop a fraction (p) of neurons in hidden layers during training
 - Different layers can have different values of p
- **Dropout** (regularization) – practically effective
 - Multiplicative noise on weights
 - For training with each data point in backpropagation, a mask is used
 - Each iteration trains a different ‘subset’ of the neural network
 - Training can take longer to converge as gradients are noisy
 - In testing phase, the outgoing weight of a neuron is multiplied with p
 - Output is produced by **averaging the outputs of each of these ‘subsets’**
 - N. Srivastava et al, *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*



Dropout

- Dropout approximates training an exponential number of sub-models simultaneously – reduces computations due to fewer non-zero weights
- Illustration: $\text{loss} = (y - \sum w_i m_i x_i)^2 = y^2 + (\sum w_i m_i x_i)^2 - 2y \sum w_i m_i x_i$
 - m_i is the mask = 1 w.p. p and 0 w.p. $1 - p$
 - $\mathbb{E}\{m_i\} = \mathbb{E}\{m_i^2\} = p$; $\mathbb{E}\{m_i m_j\} = p^2$
 - Thus, $\text{loss} = (y - p \mathbf{w}^T \mathbf{x})^2 + p(1 - p) \sum w_i^2 x_i^2$
 - If $\mathbb{E}\{x_i^2\} = 1$, then $\text{loss} = (y - p \mathbf{w}^T \mathbf{x})^2 + p(1 - p) \|\mathbf{w}\|^2$

Early Stopping

- Typically, during training: simple patterns are learned first, followed by complicated patterns (possibly noisy - overfitting)
- Stop training if validation loss did not improve beyond p epochs
 - Patience parameter: p
- Produces similar effect as L2 regularization
 - L. Prechelt, *Early Stopping – but when?*

Exploding/Vanishing Gradients

- Deeper networks have more expressivity due to more parameters
 - Overfitting due to more parameters is avoided through regularization
- Many layers $\rightarrow$ product of many gradients
 - Exploding or vanishing
 - Activation function with smaller gradients (sigmoid) can vanish
 - Poor choice of initial point can lead to explosion
- All of the regularization methods, tries to keep the gradients stable
 - Non-smooth loss surface $\rightarrow$ Exploding or vanishing gradients

Input Normalization

- Scale input features to prevent gradient vanishing/exploding
 - Preserves input distribution shape but not scale or shift
- **Min-Max scaling:** $x = \frac{x_{\min} - x_{\min}}{x_{\max} - x_{\min}}$
 - Scale data to $[0, 1]$ or $[-1, 1]$
 - Sensitive to outliers and corrupt data
- **Z-score standardization:** $x = \frac{x - x_{\text{mean}}}{\sqrt{x_{\text{var}}}}$
 - Centers input to zero and less sensitive to outliers

Distribution Shift Across Layers

- $\mathbf{x}_2 = f_1(\mathbf{W}_1\mathbf{x}_1 + \mathbf{b}_1)$: when the weights are learned, the distribution of $\mathbf{x}_2$ changes as $\mathbf{W}_1$ changes – **internal covariate shift**
- When logits are large, activations saturate, gradients vanish
- Deeper layers may have exploding/vanishing gradients
 - **Gradient clipping** – cap the values of gradient at each layer
- Keep normalizing at each layer

Batch Normalization

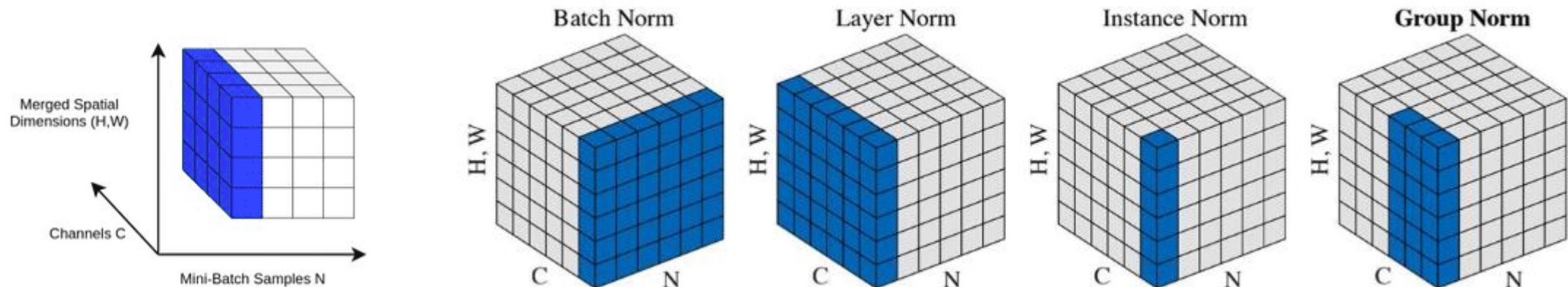
- Stabilize the distribution of activations for (mini) batch size m and reduce gradient variance
 - $\mu = \sum x_i / m$; $\sigma^2 = \sum (x_i - \mu)^2 / m$
 - $y_i = \beta + \gamma (x_i - \mu) / (\sigma + \epsilon)$
 - β and γ are learned parameters
 - Behaves like noise injection, and hence, [regularization](#)
 - Larger batch sizes provide better estimate of statistics
 - Is implemented as a layer by itself along with its own backpropagation
 - Observed to be good for CNN type architectures
- *S. Ioffe and C. Szegedy, Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*

Layer Normalization

- Stabilize the distribution of activations within a layer's output for a single sample for a layer of size d
 - $\mu = \sum x_i / d$; $\sigma^2 = \sum (x_i - \mu)^2 / d$
 - $y_i = \beta + \gamma (x_i - \mu) / (\sigma + \epsilon)$
 - β and γ are learned parameters
 - Can be implemented as a linear layer after a hidden layer (thus, allowing backprop)
 - Behaves like noise injection, and hence, [regularization](#)
 - When there are multiple channels, all of them are included in LN
 - Observed to be good for RNN type architectures
 - *J. Lei et al, Layer Normalization*

Instance and Group Normalizations

- **Instance normalization:** multiple channels and layer norm is applied to only one channel
- **Group normalization:** multiple channels and layer norm is applied to a subset of channels
- Three dimensions to normalize over: batch, channel, #neurons
 - Not all normalizations are suitable for any architecture/task
 - Pick a normalization based on the dimension(s) that is/are most correlated in a problem

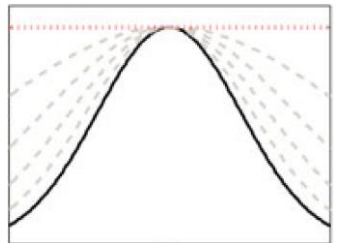


Curriculum Training

- Can the shift of variance be handled by varying the dataset used for training? – change the distribution of data over epochs
 - Train the NN from *easier to harder* to learn data
 - *Y. Bengio et al., Curriculum learning*
- **Curriculum**: a sequence of training dataset – $\mathcal{D}_1, \dots, \mathcal{D}_T$
 - Each dataset is used of L epochs or until convergence of loss
 - The entropy of the training datasets increases over epochs
 - Causing distribution shift to learn – loss surface changes with distribution

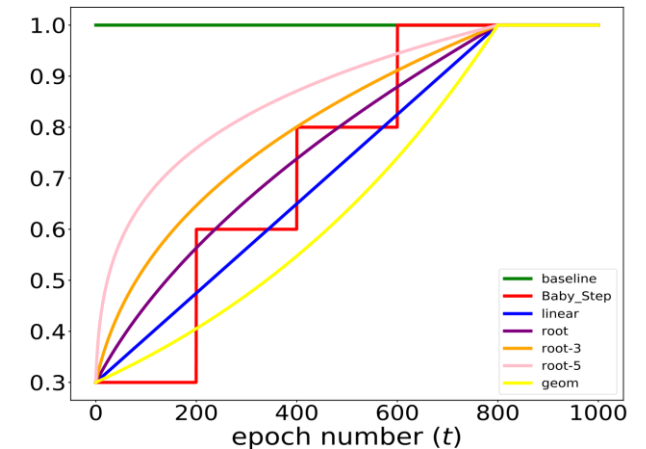
Scoring and Pacing Functions

- How to measure difficulty: scoring function $s(d)$
 - Number of classes, samples per class, entropy, SNR/distortion, distance/norm of feature vectors/embedding, sentence/sequence rarity, etc.
- When to train with a difficult sample: pacing function $p(t)$
 - **Discrete**: change dataset at fixed intervals – $\mathcal{D}_1 \leq \mathcal{D}_2 \leq \dots \leq \mathcal{D}_T$
 - **Continuous**: change dataset at each epoch – sample from a distribution
 - The parameters of the distribution can be updated over epochs
 - Start with Gaussian and move to uniform: $f(t, d) = p(t)\mathcal{N}(d) + (1 - p(t))U(d)$



Self-Paced Learning

- $\text{Loss} = \mathbb{E}_{\mathbf{f}}\{\nabla_{\mathbf{w}_t} L_i\} = \sum_i \nabla_{\mathbf{w}_t} L_i / N$ for uniform distribution
- Can we learn scoring & pacing functions?
 - $\text{Loss} = \sum_i f_i \nabla_{\mathbf{w}_t} L_i - \lambda \|\mathbf{f}\|_1$
 - Samples with smaller loss values are selected first
 - ‘Harder’ samples have higher loss values – higher variance of gradients
 - Reduces gradient variance early in training
 - Reinforcement learning can be used to dynamically change dataset
- Often used in NLP & CV, wherever dataset has very high variance



Transfer Learning

- What if we initialize a neural network, or parts of it, with weights/parameter from another pre-trained model?
 - Can be seen as a special case of curriculum or teacher training
- Two goals: parameter transfer or feature representation transfer
- **Inductive** (different task) vs **transductive** (same task) transfer
- Neural net output $y(\mathbf{w}, \mathbf{x}) = g(\mathbf{w}_1, h(\mathbf{w}_2, \mathbf{x}))$
 - Feature extractor - $h()$, task-specific inference function - $g()$

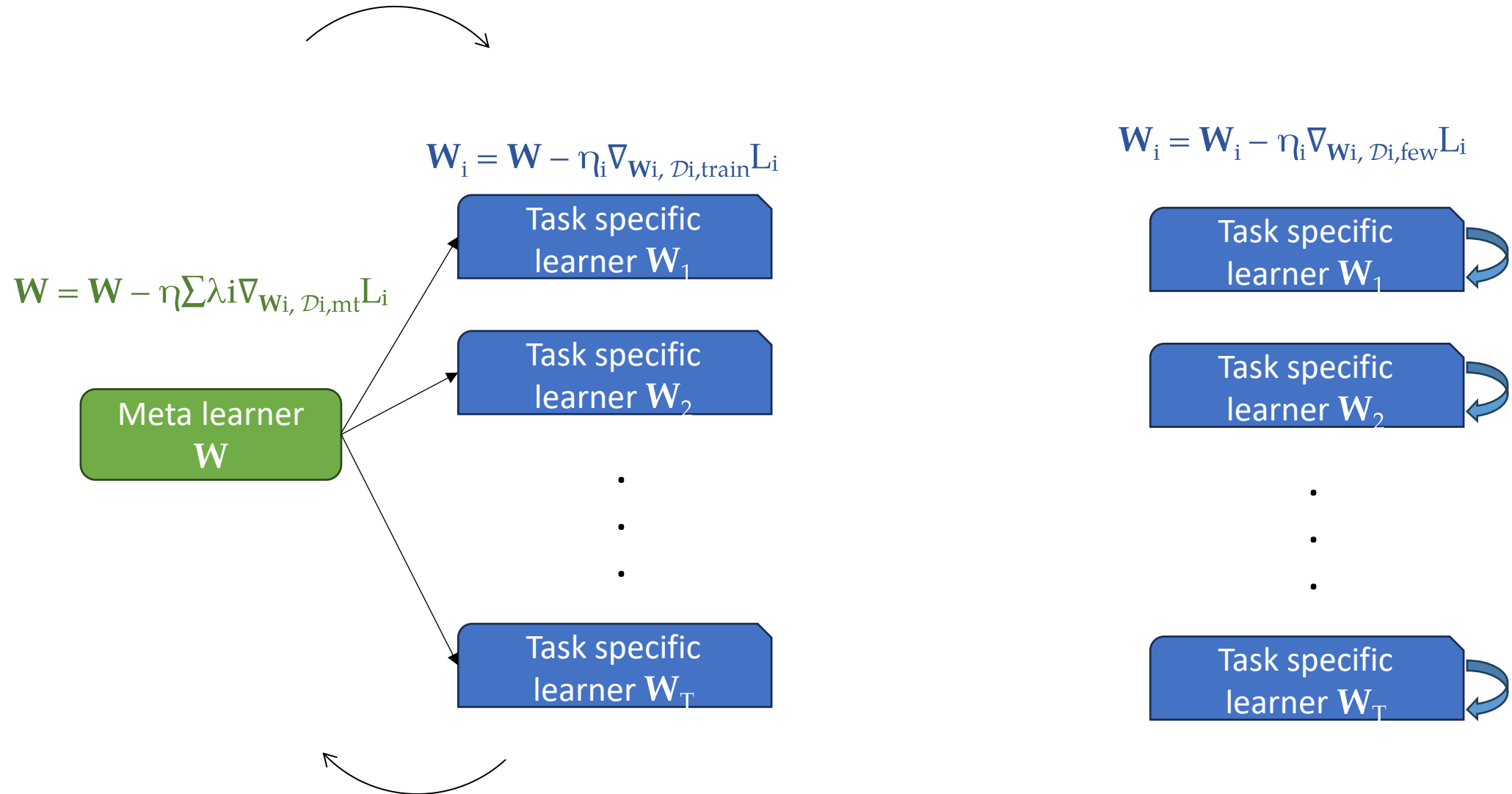
Transfer Learning

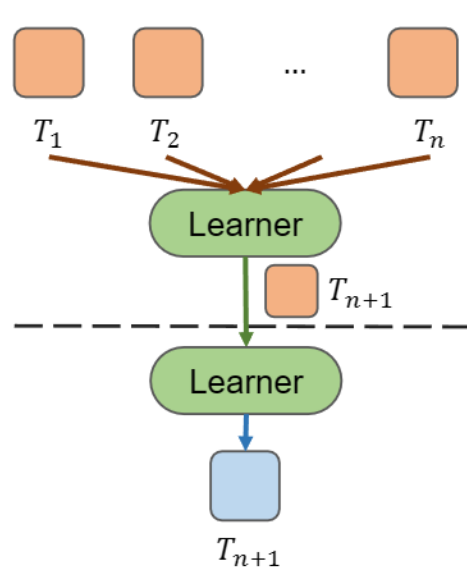
- Inductive: same features can be used across different tasks
 - Different tasks may share the same input dataset with different labels and loss functions
 - Transfer feature representation function - $h()$ – share initial layers' weights
- Transductive: Same task, different data - transfer inference func $g()$
- Prevent too much **model drift**: $L(\mathbf{w}) + \lambda \|\mathbf{w} - \mathbf{w}_{\text{pretrained}}\|^2$
- **Multi-task learning** loss: $\min_{\mathbf{w}} L_1(\mathbf{w}_1, h(\mathbf{w}_0, \mathbf{x})) + \lambda L_2(\mathbf{w}_2, h(\mathbf{w}_0, \mathbf{x}))$

Meta Learning & Few-Shot Learning

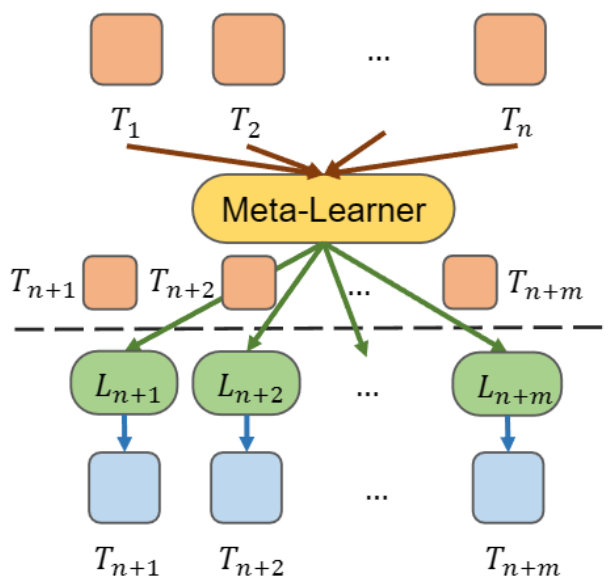
- How to learn different model parameters for different tasks (loss) with different train & test (correlated) datasets?
 - Meta learning: $\min \sum_i \lambda_i L_i(\mathbf{W}_i, \mathcal{D}_{i, \text{meta-train}})$
 - Task specific learning: $\mathbf{W}_i = U(\mathbf{W}, \mathcal{D}_{i, \text{train}})$
 - Model-agnostic meta learning
 - Bayesian model: Common prior ($\mathbf{W}$) to all tasks
 - Entropy in $\mathbf{W}_i$ reduces when conditioned on $\mathbf{W}$ – thus, requires lesser training effort
- Few-shot learning: $\mathcal{D}_{i, \text{Test}}$ is a small dataset

Meta Learning & Few-Shot Learning

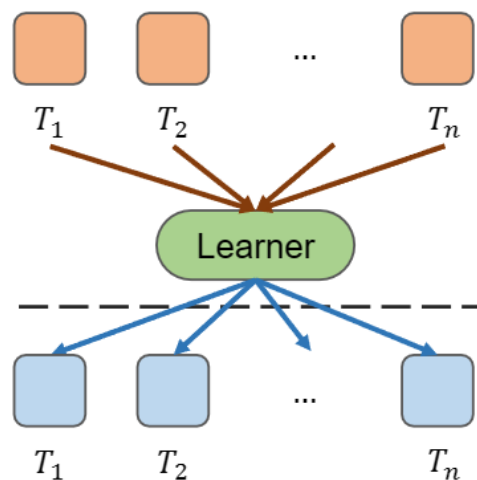




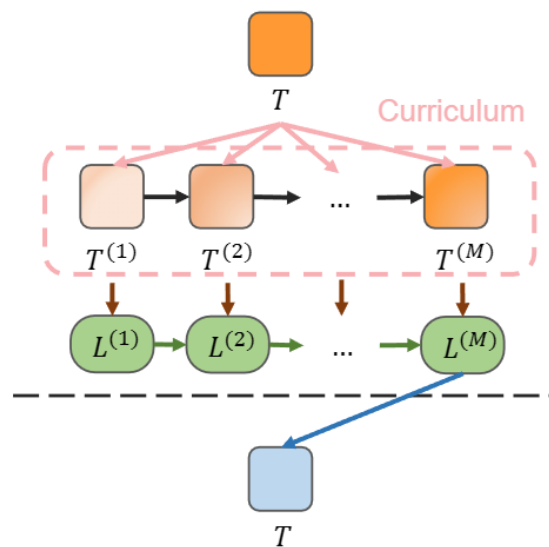
Transfer Learning



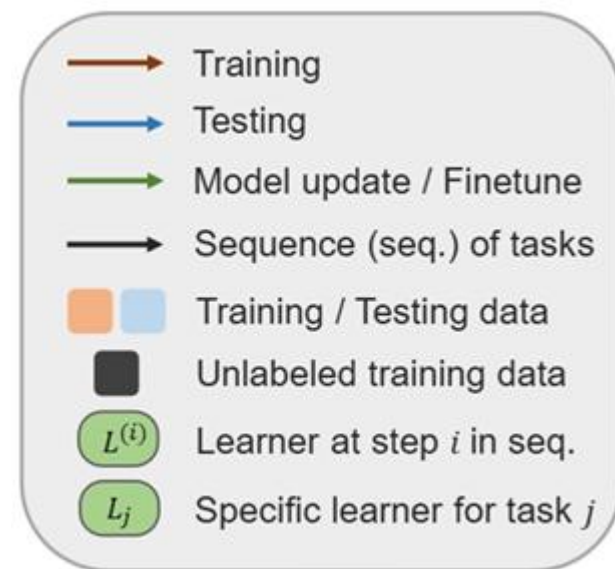
Meta-Learning



Multi-task Learning

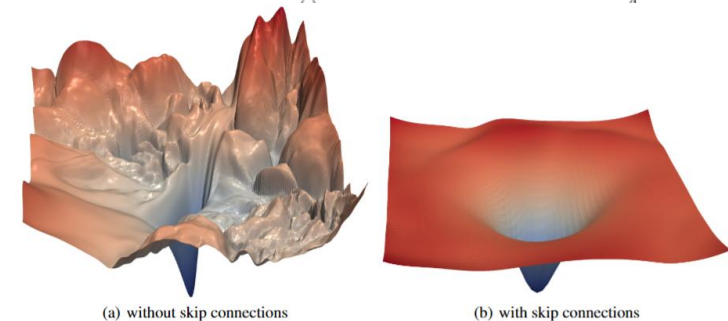
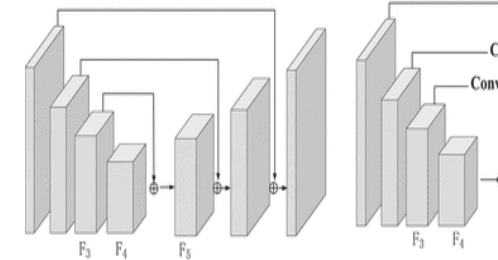


Curriculum Learning



Skip Connection

- Avoid vanishing/exploding gradient by **skipping between layers**
 - E.g., $f_3(\mathbf{W}_3\{f_2(\mathbf{W}_2f_1(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) + f_1(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)\} + \mathbf{b}_3)$
- Randomly choose to **add the output of a layer to that of another layer ahead**
 - Short skip (between layers of same size) - *ResNet*
 - Long skip (brings in features that may have been 'forgotten')
 - Has a behavior **similar to ensemble learning**
- Skip connection via concatenation
 - Increase output size - *DenseNet*
- H. Li et al, *Visualizing the Loss Landscape of Neural Nets*



Where to Add Skip Connections

- Use in deep networks (>10 layers)
 - When loss is non-decreasing and gradients are unstable
- Initial layers get non-vanishing gradient updates in backprop
- When features learned by earlier layers need to be preserved
 - Adding residue requires same feature dimensions
 - Concatenate when features need to be retained as such
 - Add residue (to logits) before activation
 - CNN (short skip), encoder-decoder (long skip) and attention models

Hyperparameter Optimization

- How to find the optimal hyperparameters?
 - $\operatorname{argmin}_{\lambda} \min_{\mathbf{w}} L(\mathbf{w}, \lambda)$
 - Gradient descent over λ ?
- Typical hyperparameters
 - Learning rate, momentum factors, weight decay regularization coefficient, dropout rate, label smoothing level, training batch size, number of layers, hidden units, etc.
- Tools: Hyperopt, skopt, raytune, wandb, optuna

Hyperparameter Search

- Train model with different hyperparameter values and pick the one with the least validation error
- **Grid search**: search from a pre-fixed set of values
- **Random search**: sample from a distribution
 - Learning rates – loguniform; dropout – uniform
 - Bayesian search: fit a distribution (e.g., Gaussian) to the hyperparameters, estimate its parameters and sample from it
- **Hyperband**: Pick multiple candidates, train shortly (early stop), drop high-loss ones and keep only promising ones – dynamically allot more/less training epochs

Hyperparameter Tuning

- **Tuning step 1:** Choose a search set for: depth, width, activation functions, GD & regularization parameters
- **Tuning step 2:** **Train the network**
 - **Step 0:** Pick a depth, width and activation function from the search set
 - **Step 1:** Initialize weights and biases (with random values)
 - **Step 2:** Split data to training + validation
 - **Step 3:** Do gradient descent (e.g., SGD with Adam) over the loss function by backpropagating (include early stop, dropouts and skip connections if needed)
 - **Step 4:** Stop when gradient descent converges and compute the training loss and validation loss
 - **Step 5:** Go to **step 1** & repeat until validation loss converges
- **Tuning step 3:** Go to **step 2** till the search set is exhausted
- **Tuning step 4:** Pick the model with the least training and validation losses

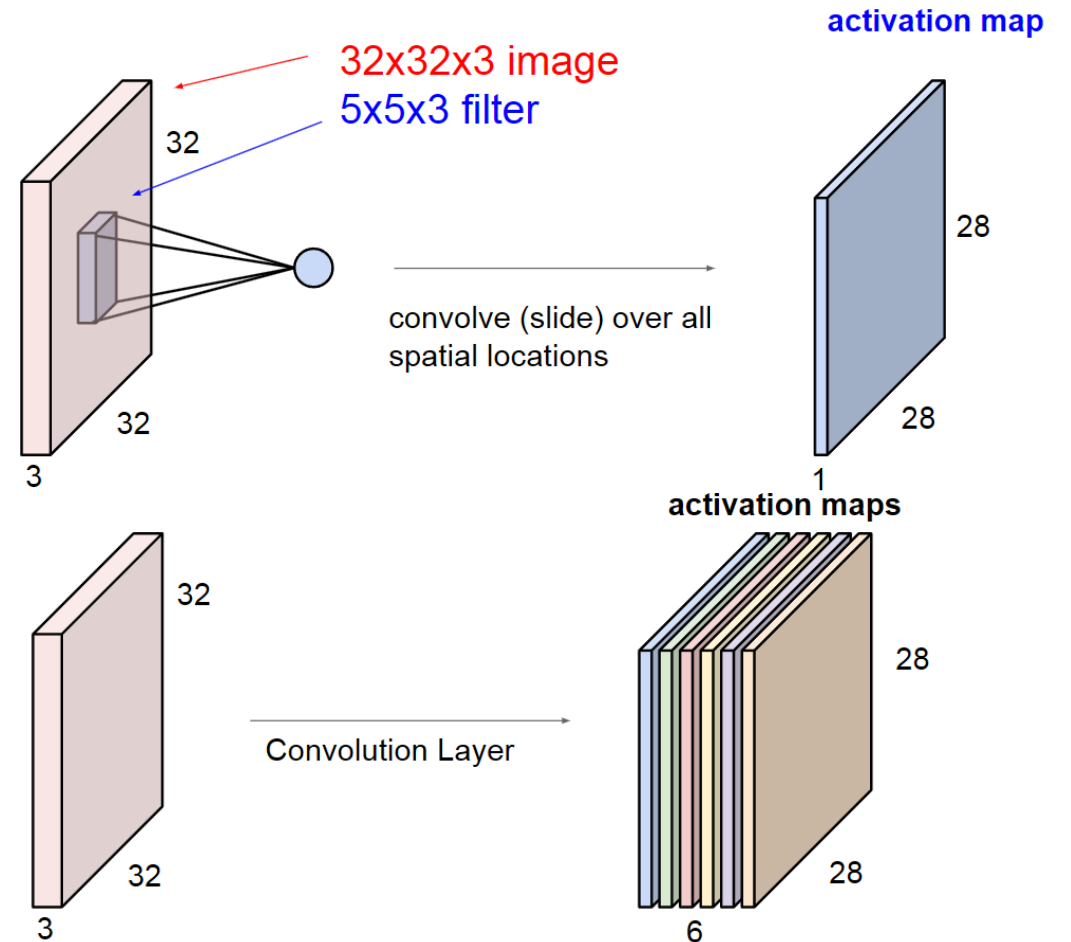
Convolutional Neural Network

Identifying a Pattern in any Position

- What if we want to identify if a pattern $[3 \ 1 \ 4 \ 2 \ 5]$ is present in a 128×1 vector?
 - How to train an MLP or FFNN or FCNN for this? What is the dataset?
- Can we simplify the network architecture? Reduce dataset size?
 - Scan over the vector and match local regions to the pattern
- Making the neural network **shift invariant**

Creating Shift Invariant Features

- Segment input data
 - Each segment is mapped to a scalar value by a ‘filter’
 - Multiple such filters can be used
 - Resulting in multiple channels
- Filtering captures a pattern
 - By projecting input on to a filter
 - ‘Matched filter’



Convolution Operation

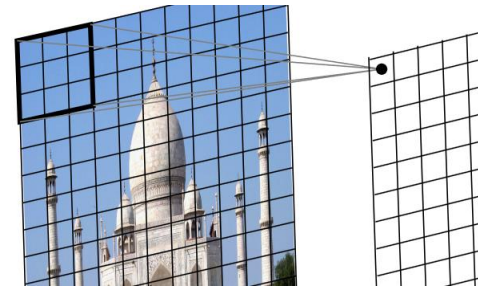
- Technically, correlation: $(I * K)_{ij} = \sum_{a=0}^{m-1} \sum_{b=0}^{n-1} I_{i-a, j-b} K_{a,b}$
- Filter or kernel



$$\begin{matrix} & 1 & 1 & 1 \\ * & 1 & 1 & 1 \\ & 1 & 1 & 1 \end{matrix} =$$



$$\begin{matrix} & 1 & 1 & 1 \\ * & 1 & -8 & 1 \\ & 1 & 1 & 1 \end{matrix} =$$



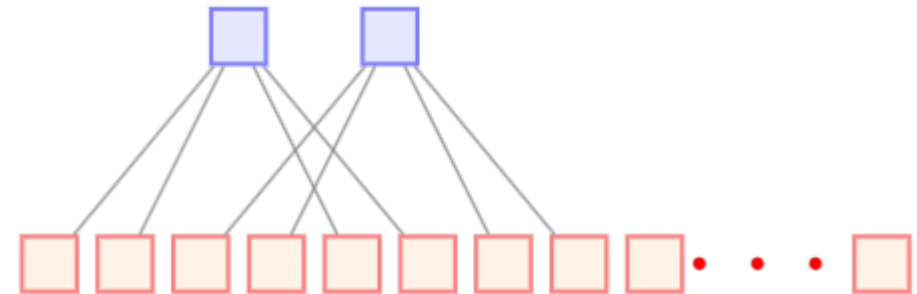
Input				Kernel	
a	b	c	d	w	x
e	f	g	h		
i	j	k	ℓ	y	z

Output		
aw+bx+ey+fz	bw+cx+fy+gz	cw+dx+gy+hz
ew+fx+iy+jz	fw+gx+jy+kz	gw+hx+ky+ℓz

Convolution Layer

- The segmentation and filtering – convolution can be implemented with an MLP; however,

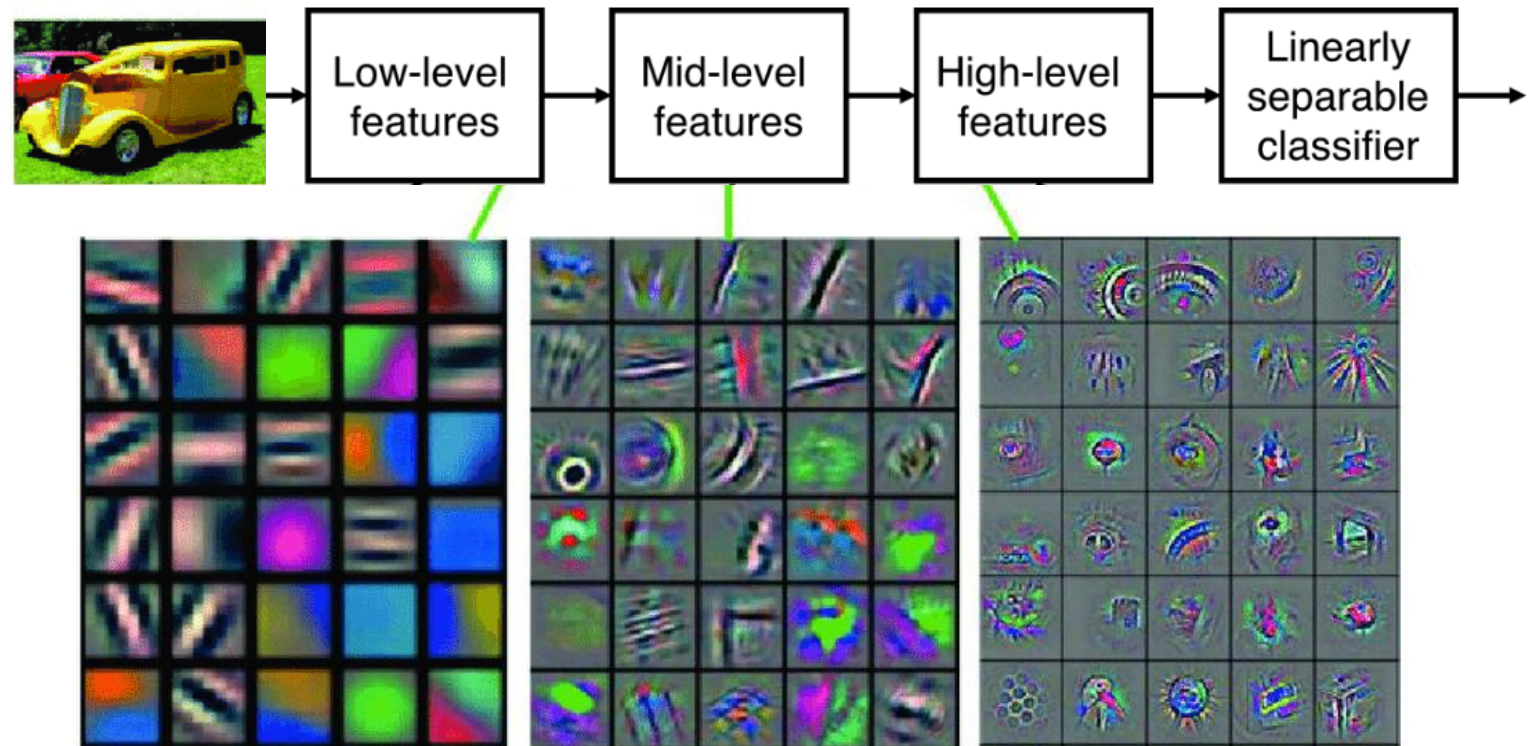
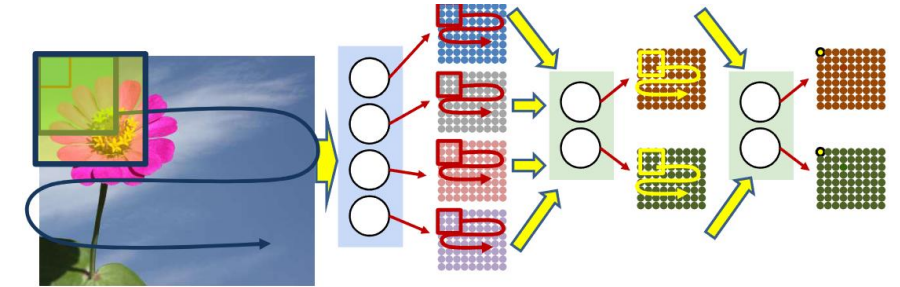
- Weights are sparse
- In a layer, weights are shared



- For an input of size $W \times H \times D$ and filter/kernel $F \times F \times D$
 - Output per kernel is $W - F + 1 \times H - F + 1$
 - Total output size for K kernels: $W - F + 1 \times H - F + 1 \times K$

Nested Convolution Layers

- First layer of filters can capture a pattern
- Subsequent layers
 - Patterns over patterns
 - **Receptive field** increases with depth
- What if output feature size is still large?



Stride and Padding

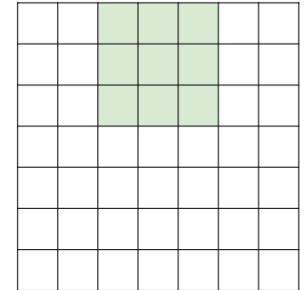
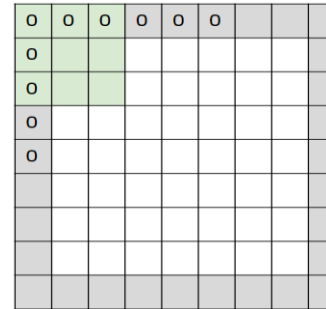
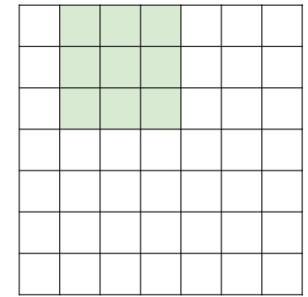
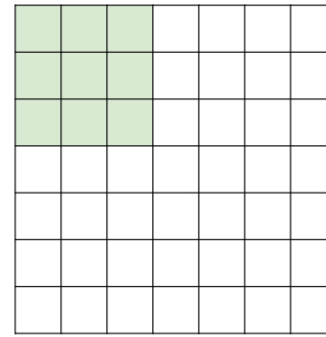
- **Padding** borders of input to obtain feature information from corners

- Padding length P – depends on F

- **Stride**: Filtering is not continuous

- Downsampled filter output
- Stride length S : downsampling factor

- Output feature size: $(W - F + 2P)/S + 1 \times (H - F + 2P)/S + 1 \times K$



Pooling

- Reducing feature size even further

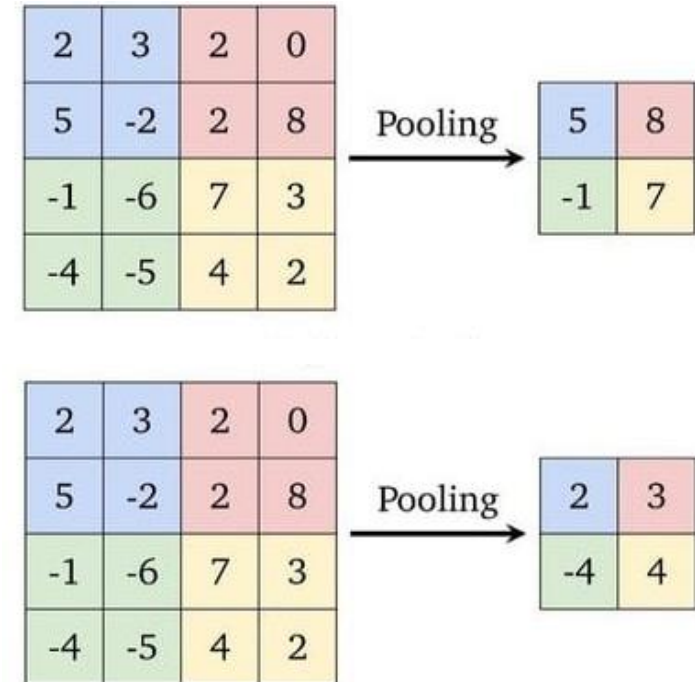
- Max pooling

- $y = \sum w_i x_i$ such that $w_i = 1$ if $x_i = x_{\max}$, 0 else
 - $\partial y / \partial x_i = 1$ if $x_i = x_{\max}$, 0 else

- Average pooling

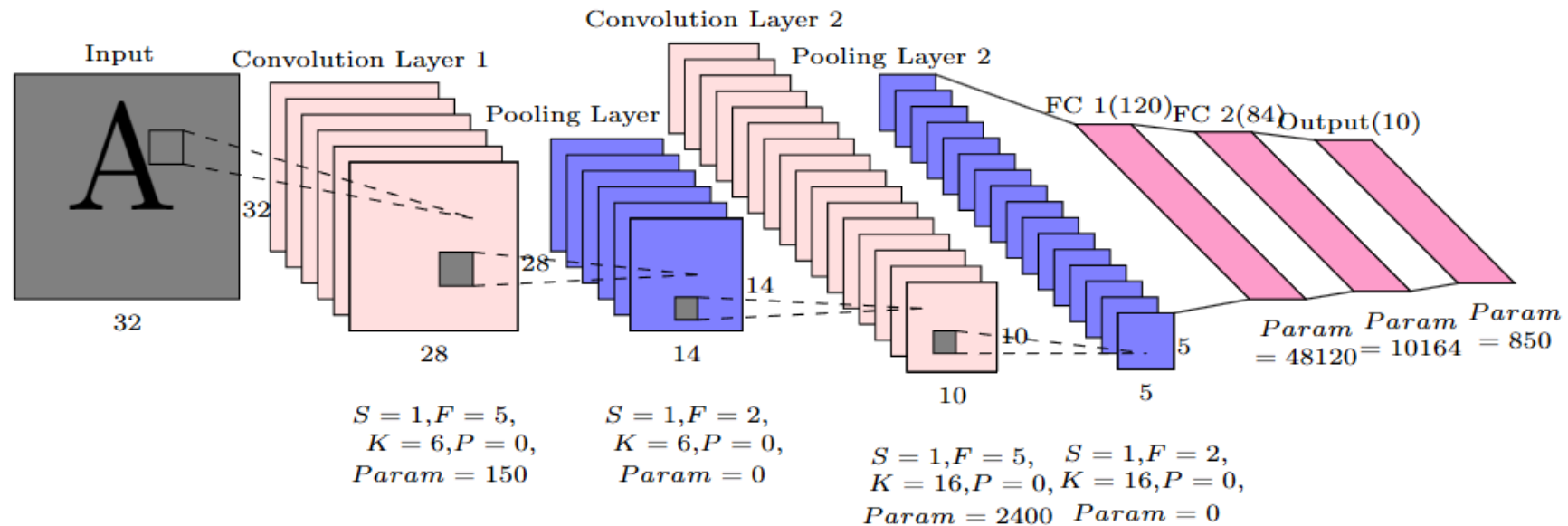
- Pooling after activation

- Obtaining similar features even with greater stride
- Handling ‘pixel jitter’ – local invariance – globally equivariant
 - Approximate global invariance achieved by deeper layers and pooling



Example CNN

- After convolutions & pooling, the order does not matter; now, flatten and pass it to an MLP (most parameters are here)
- Demo: <https://cs.stanford.edu/people/karpathy/convnetjs/demo/cifar10.html>



AlexNet (62M Params)

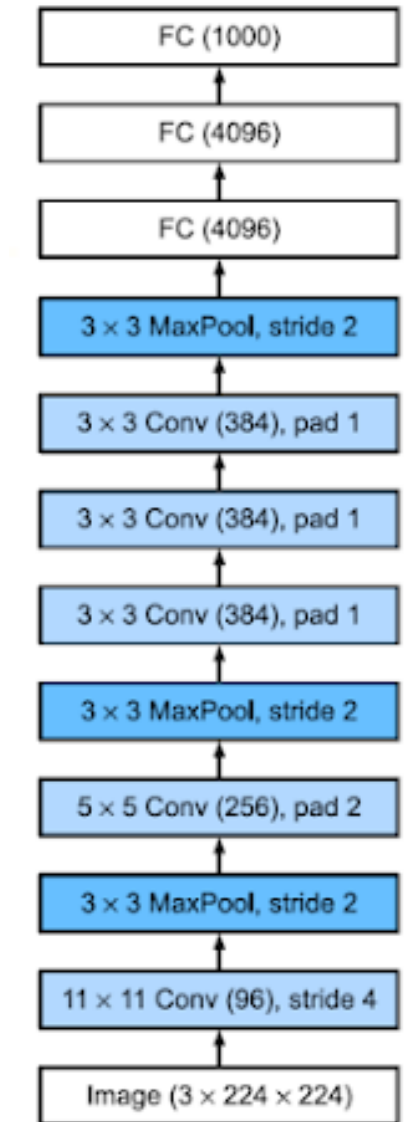
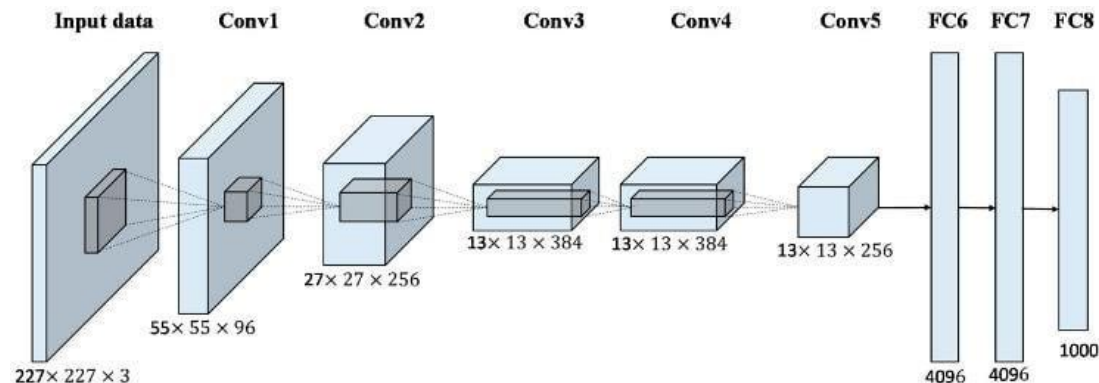
- Alex Krizhevsky et al, ImageNet Classification with Deep Convolutional Neural Networks, 2012

- Key properties:

- ReLU, SGD with momentum, batch size of 128, max pooling, dropout
- Data augmentation: flipping, jittering, cropping, color normalization
- Early layers: large kernels and strides
- Later layers: smaller kernels and strides
 - Receptive field: almost whole image

- In hindsight

- No deep enough
- Too big kernels
- No proper normalization



VGGNet (138M Params)

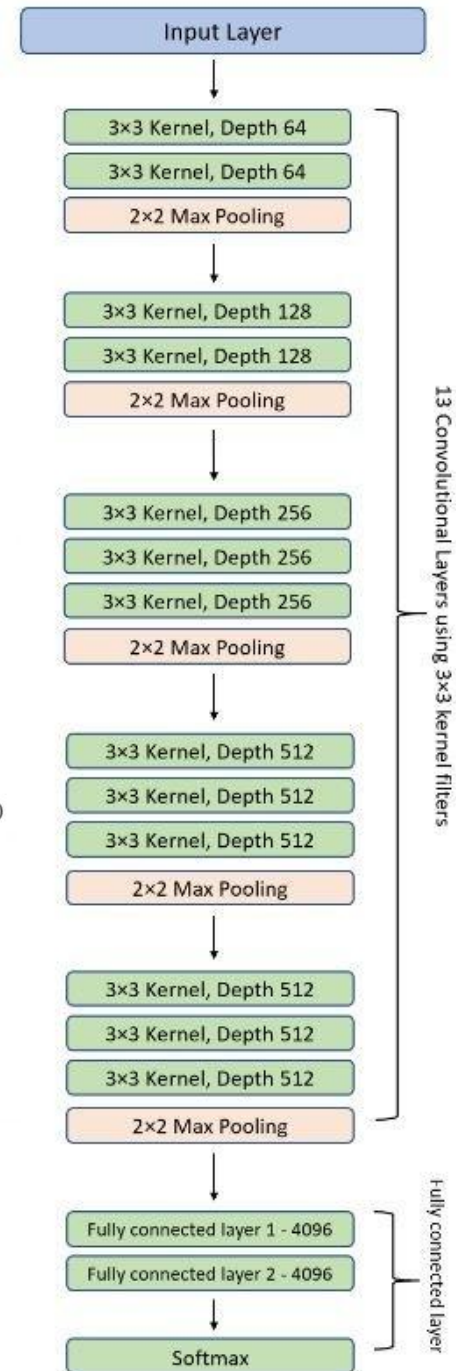
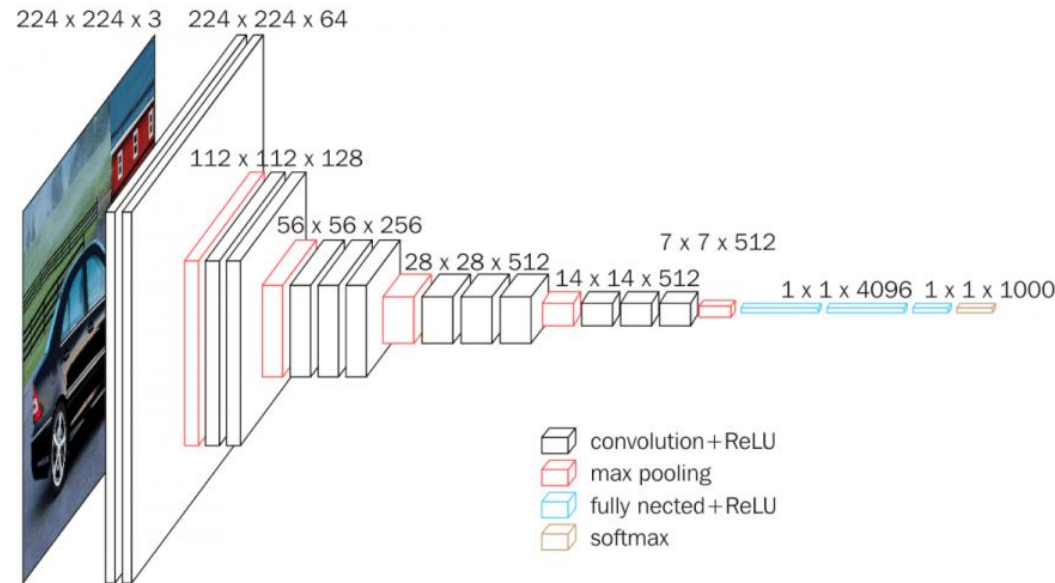
- K. Simonyan and A. Zisserman, Very Deep Convolutional Networks for Large-Scale Image Recognition, 2014

- Key properties:

- Smaller filter, stride 2
- More depth
- Receptive field grows gradually, captures more features

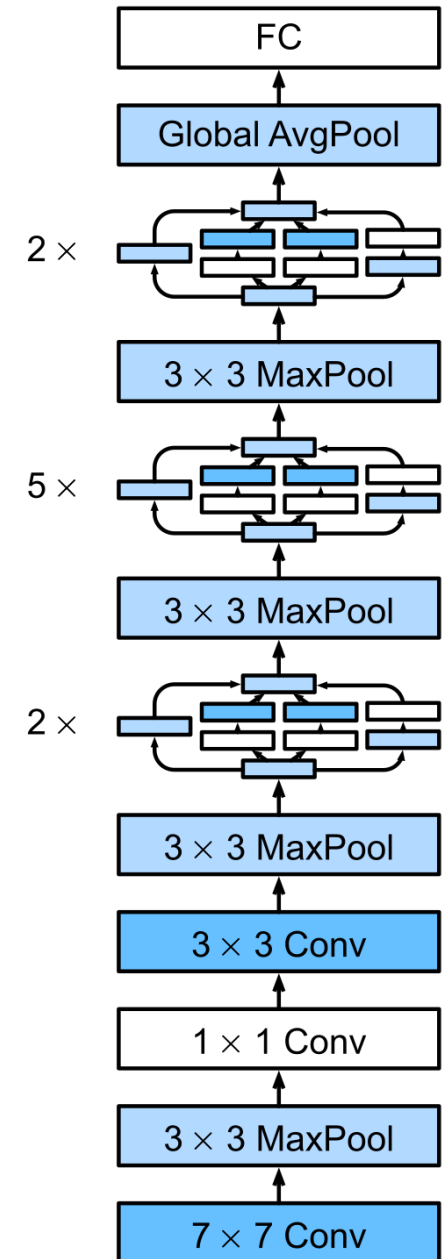
- In hindsight

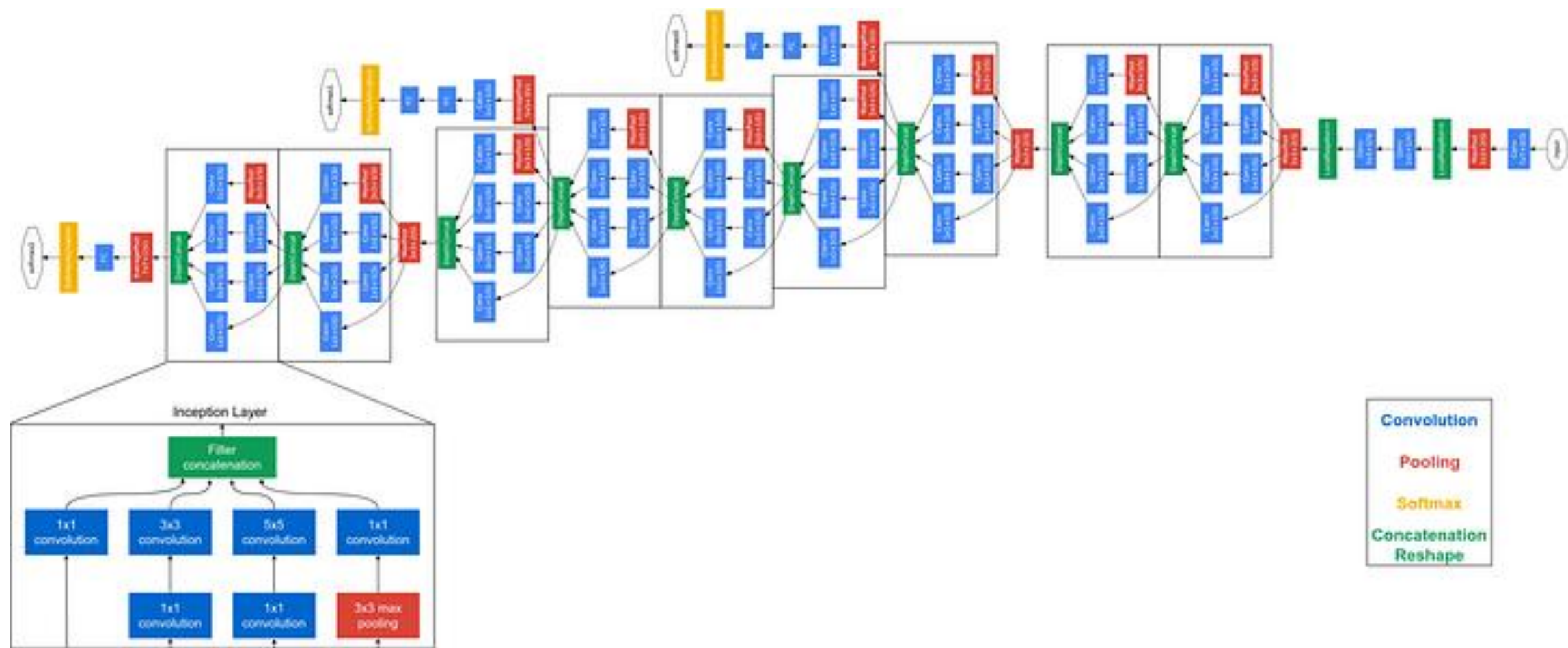
- More depth means more vanishing gradients



GoogleNet (6.8M Params)

- Can we use multiple kernels simultaneously?
 - Each kernel has a different receptive field
 - Concatenate features of all kernels – inception
 - 1 x 1 convolution = weighted summation across channels
 - Dimensionality reduction – removing channels
 - Global pooling – reducing parameters
 - Vanishing grad: Multiple outputs at different depths
 - Only final output is retained in testing
 - Loss function: $0.3L_{D1} + 0.3L_{D2} + L_{\text{End}}$
- In hindsight
 - Cannot go deeper! – needed for better generalization





ResNet-18 (12M Params)

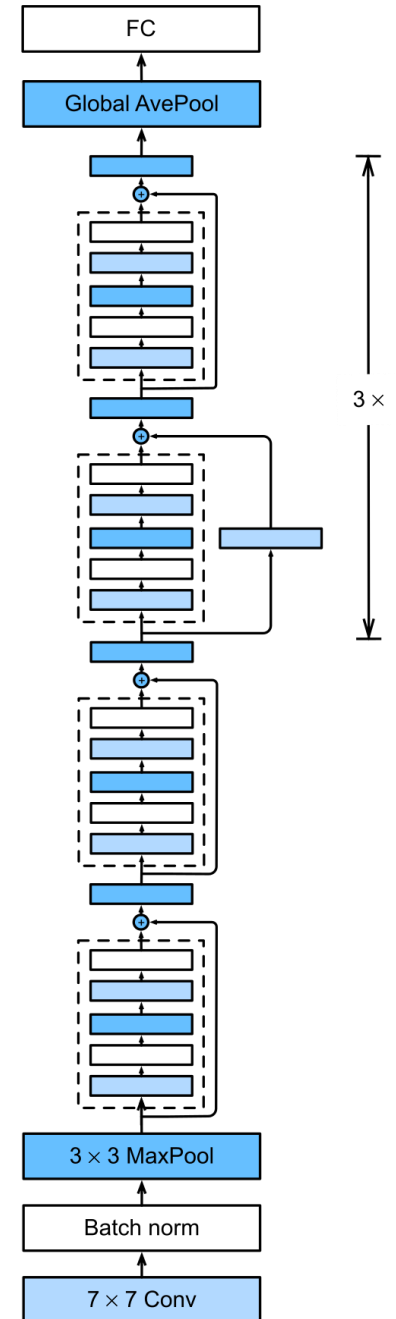
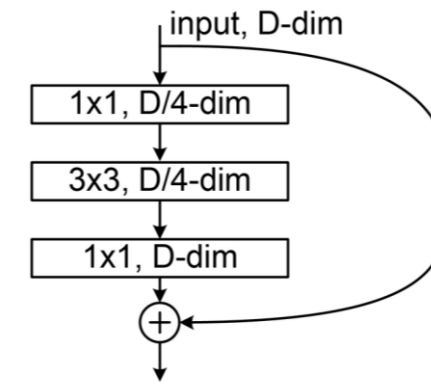
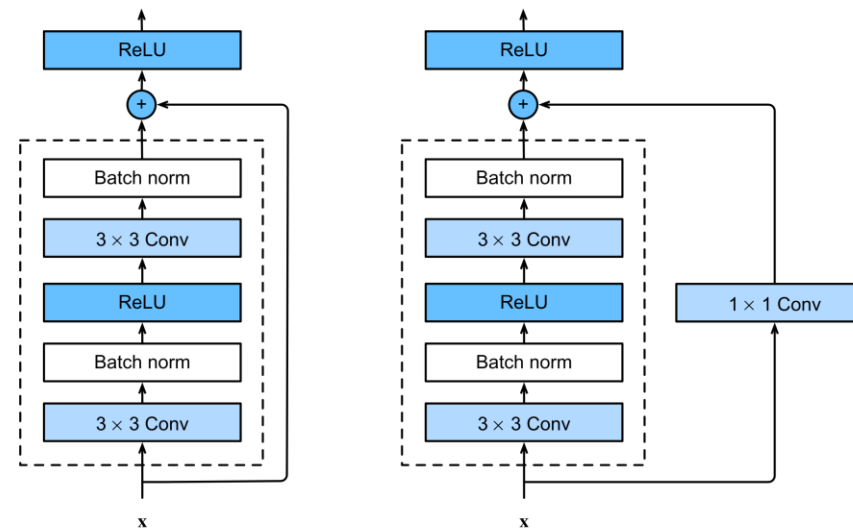
- K. He et al, Deep Residual Learning for Image Recognition 2016

- Key properties:

- Skip connection (more depth)
- Batch normalization
- Bottleneck layer (only important features are passed through)

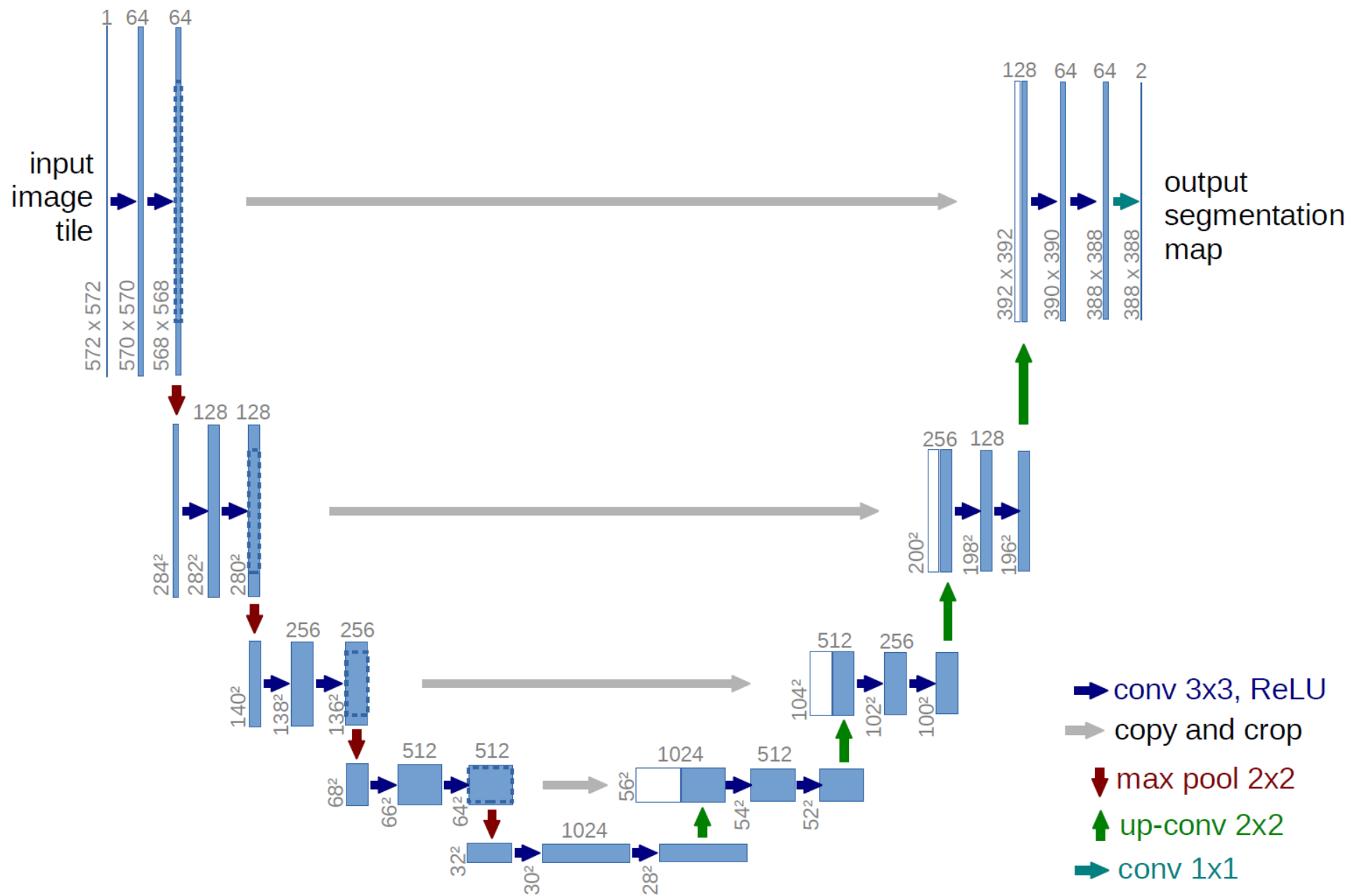
- In hindsight

- DenseNet (feature reuse)
- ResNeXt (Inception + ResNet)
- Neural ODE



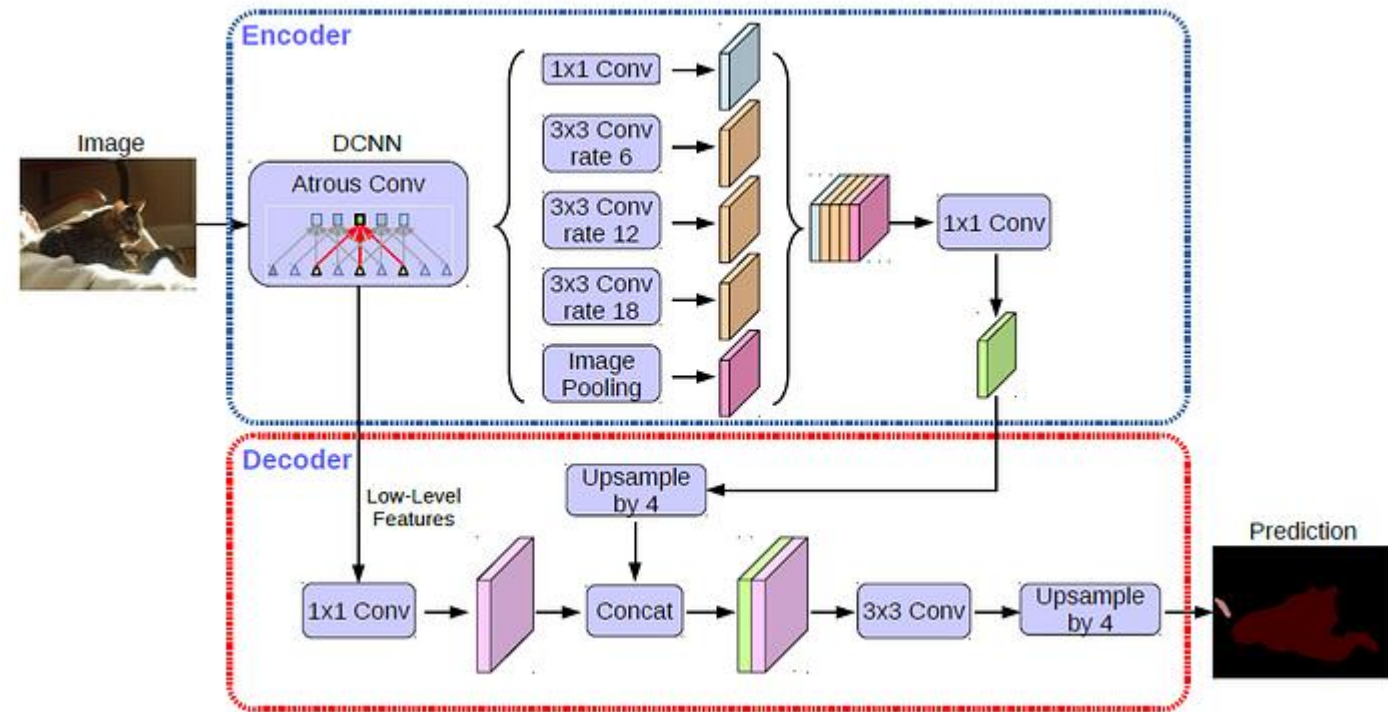
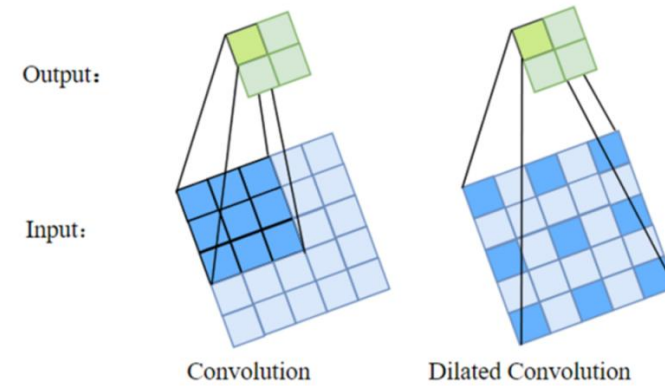
U-Net (31M Params)

- O. Ronneberger et al, U-Net: Convolutional Networks for Biomedical Image Segmentation, 2015
- Autoencoder: downsample and then upsample
 - Results in denoising (regression), segmentation (classification)
- Encoder (Compressor): Extract ‘principal’ features and reduces size
- Decoder (Expander): Reconstructs ‘principal’ features
- Skip connections: across layers of same size
 - Provides feature-location information, compensates information loss

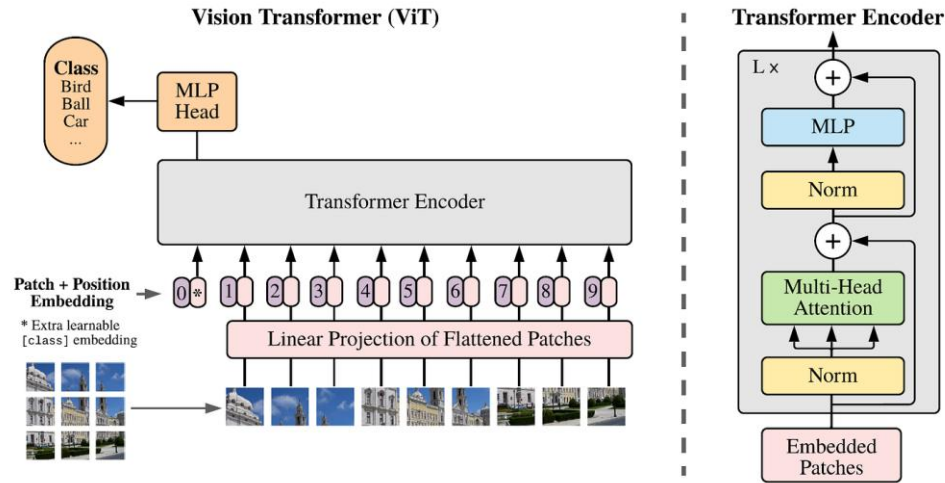


DeepLabV3

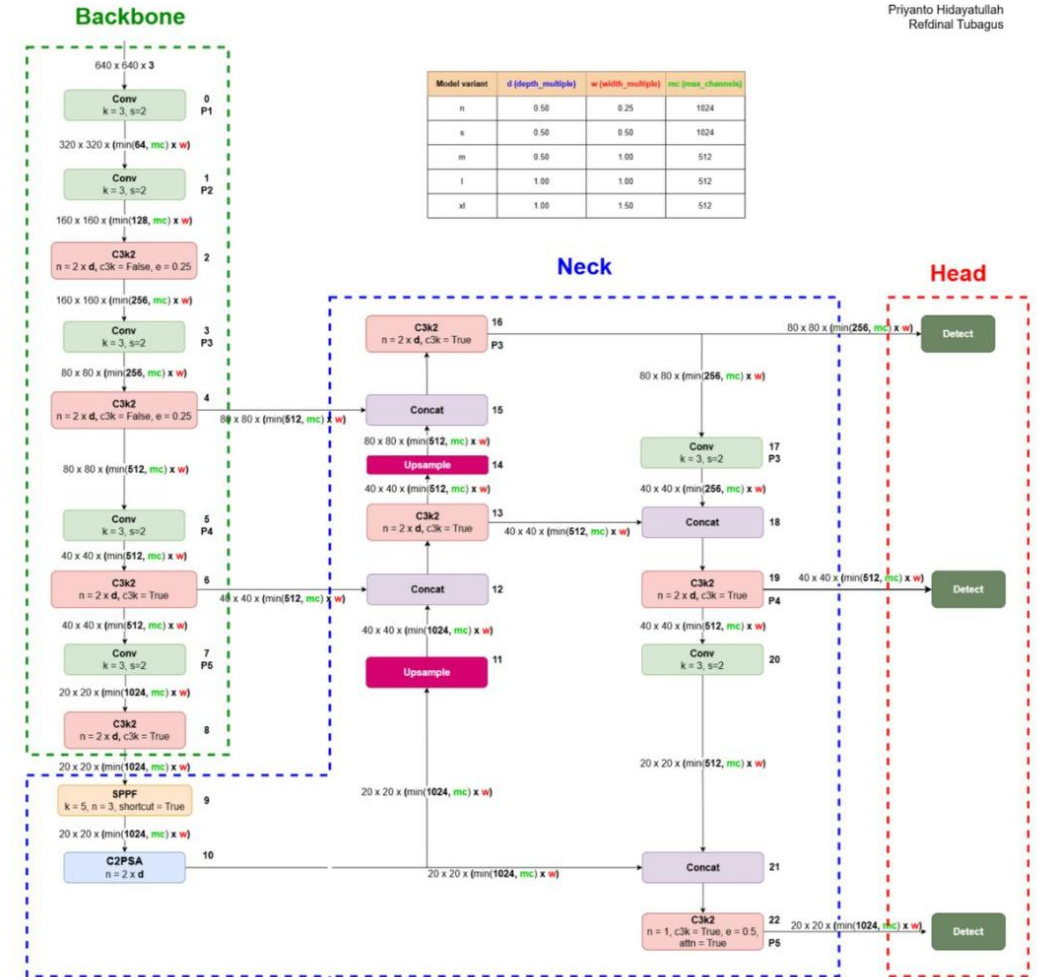
- Pooling results in coarse segmentation
- Encoder (backbone)
 - Atrous spatial pyramid pooling
 - Multi-scale features
 - Atrous convolution
 - Larger receptive field
 - Inception + pooling
- Decoder
 - Similar to U-Net
 - Low resolution features – semantic info
 - Upsampled features – spatial info



Latest Models



- Vision Transformer (ViT)
 - Global attention and context understanding
- YOLO26



Priyanto Hidayatullah
Refidinal Tubagus

- A. Chaman and I. Dokmanic, Truly shift-invariant convolutional neural networks, 2021

GradCAM (Class Activation Mapping)

- **Interpretability:** Which spatial region caused the decision
 - Gradient of the output logit (y^c – for class c) w.r.t. features
 - A_{kij} – Feature at k^{th} channel and (i, j) spatial position
 - Computed from backprop Gradient $\partial y^c / \partial A_{kij}$
 - Global average pooling – neuron importance weight: $a_k^c = 1/L \sum_{ij} \partial y^c / \partial A_{kij}$
 - Grad-CAM = $\text{ReLU}(\sum_k a_k^c \mathbf{A}_k)$
 - Positive gradient implies affirmative contribution by (i, j) pixel to class c
- R R Selvaraju et al, Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization, 2019
- Aditya C et al, Grad-CAM++: Improved Visual Explanations for Deep Convolutional Networks



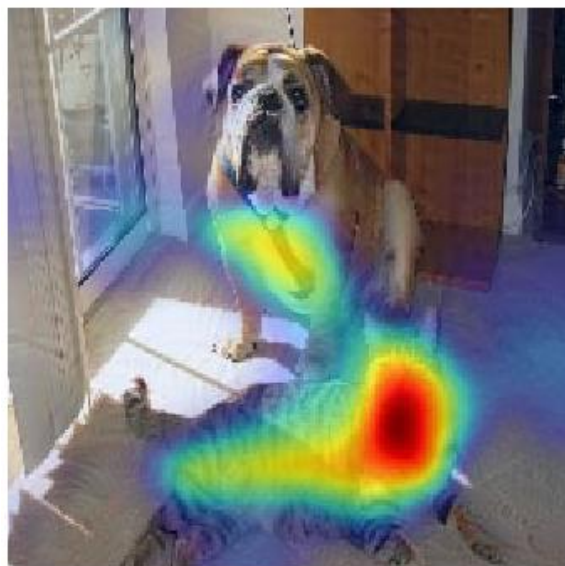
Boxer: 0.4 Cat: 0.2
(a) Original image



Airliner: 0.9999
(b) Adversarial image



Boxer: $1.1e-20$
(c) Grad-CAM "Dog"



Tiger Cat: $6.5e-17$
(d) Grad-CAM "Cat"



Airliner: 0.9999
(e) Grad-CAM "Airliner"



Space shuttle: $1e-5$
(f) Grad-CAM "Space Shuttle"

Recurrent Neural Network

Finite and Infinite Response Systems

- CNN and FCNN require data of fixed length
- How to handle data of unknown (even infinitely long) length?
 - Need some assumptions on input-output correlation: $\mathbf{y}_t = f(\mathbf{x}_{-\infty \dots t \dots \infty})$
 - **Finite length response:** $\mathbf{y}_t = f(\mathbf{x}_{t-N \dots t}) = f(\mathbf{x}_t, \mathbf{x}_{t-1}, \mathbf{x}_{t-2} \dots \mathbf{x}_{t-N})$
 - **Infinite length response:** $\mathbf{y}_t = f(\mathbf{x}_{-\infty \dots t}) = f(\mathbf{x}_{t-N \dots t}, \mathbf{y}_{t-N \dots t-1})$
 - One non-zero input could possibly produce non-zero output forever
- Assumed to be **causal**

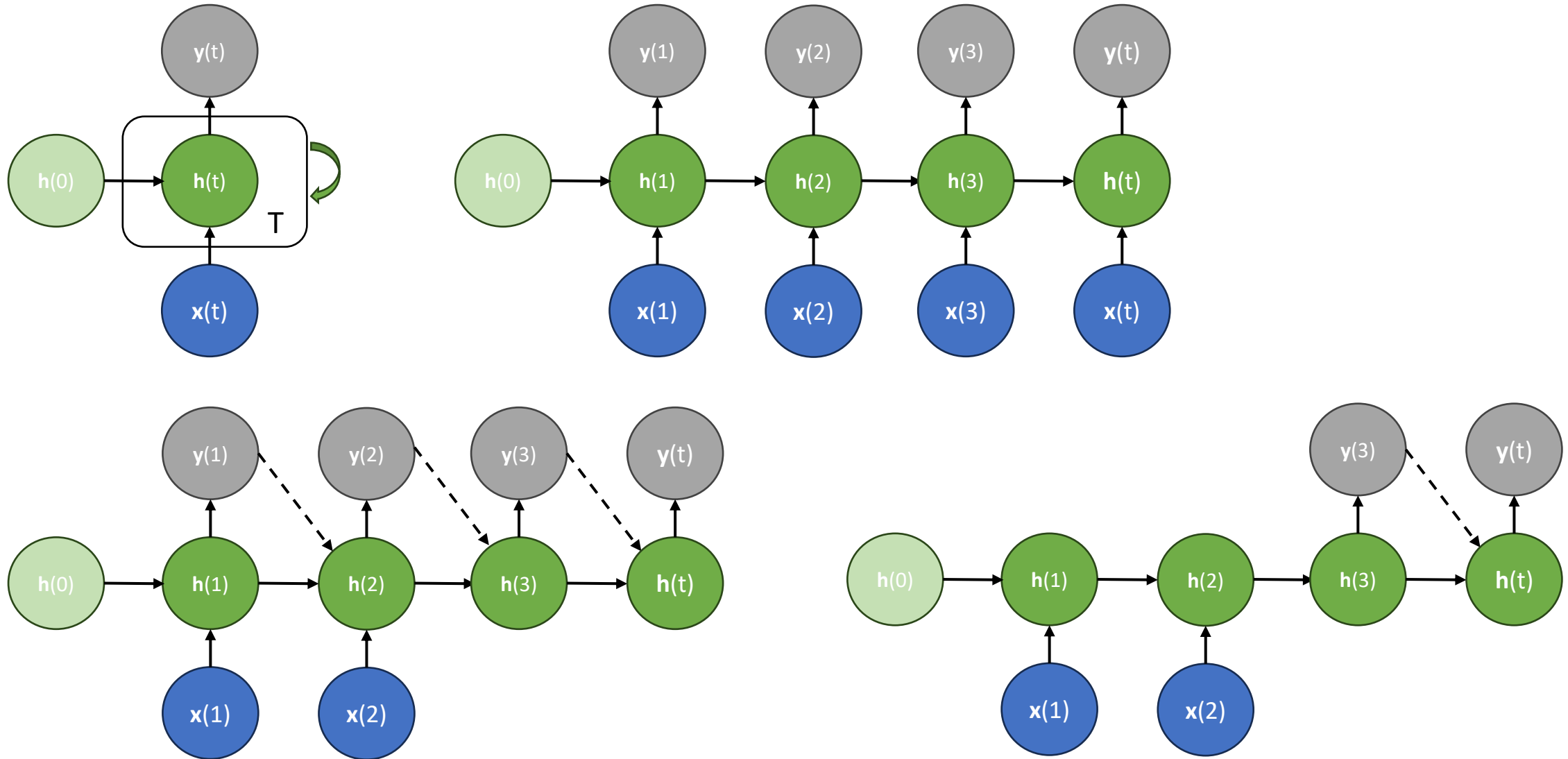
AR and NARX Processes

- Autoregressive – AR(N) process: $\mathbf{y}_t = \sum_{i=1 \text{ to } N} \mathbf{W}_i \mathbf{y}_{t-i} + \mathbf{x}_t$
- Moving average – MA(N) process: $\mathbf{y}_t = \sum_{i=0 \text{ to } N} \mathbf{W}_i \mathbf{x}_{t-i}$
 - ARMA: $\mathbf{y}_t = \sum_{i=0 \text{ to } N} \mathbf{W}_i \mathbf{x}_{t-i} + \sum_{i=1 \text{ to } M} \mathbf{V}_i \mathbf{y}_{t-i}$
 - Single layer linear perceptron
- Wold's representation theorem – ARMA process is universal
 - Any WSS random process = deterministic process + stochastic process
- Non-linear autoregressive with exogenous input (NARX)
 - $\mathbf{y}_t = f(\mathbf{y}_{0 \dots t-I}, \mathbf{x}_{0 \dots t})$

State Space Model

- Hidden state: $\mathbf{h}_t = g(\mathbf{x}_t, \mathbf{h}_{t-1}); \mathbf{y}_t = f(\mathbf{h}_t)$
 - State summarizes information about the entire past
 - Retains the sufficient statistics/information from past
 - Initial state affects state evolution/output: $\mathbf{h}_0$
- To learn
 - Neural networks: state predictor - $g()$, and output predictor - $f()$
 - Parameters of $g()$ (and $f()$) are shared for all t

Recurrent Neural Architectures



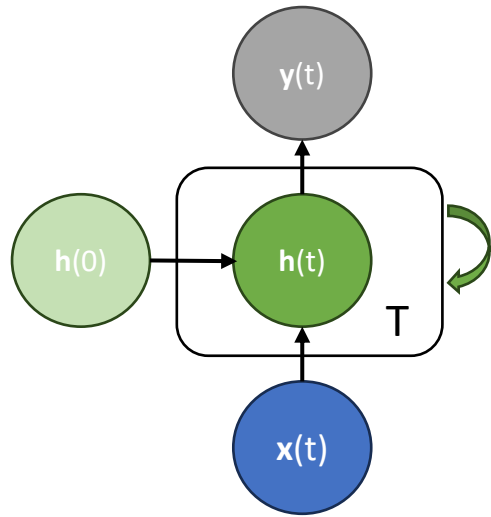
Examples

- Time series (stock market) prediction – one to one
- Sequence (next word) prediction, speech recognition, text classification, sentiment analysis, weather forecast, anomaly detection – many to one
- Sequence to sequence conversion, language translation, data summarization, text to speech and vice versa – many to many

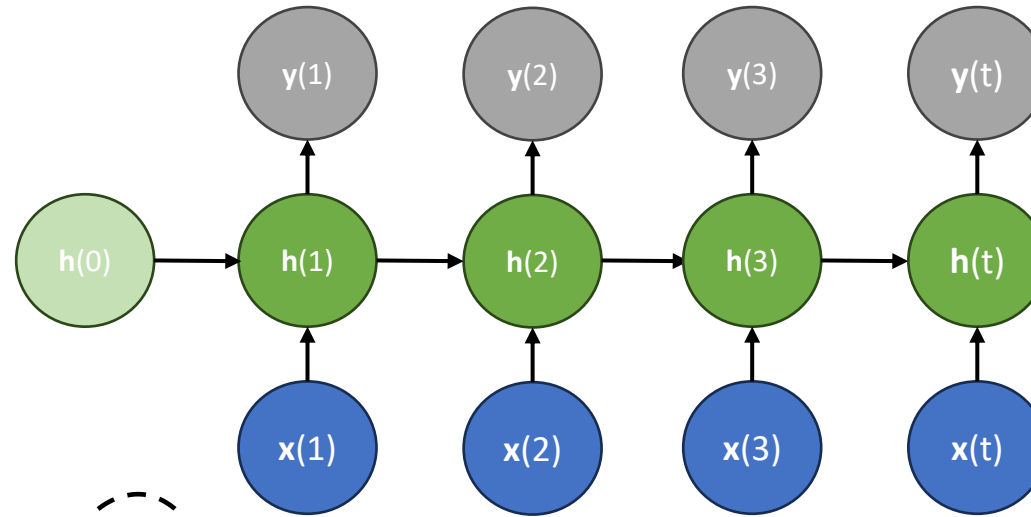
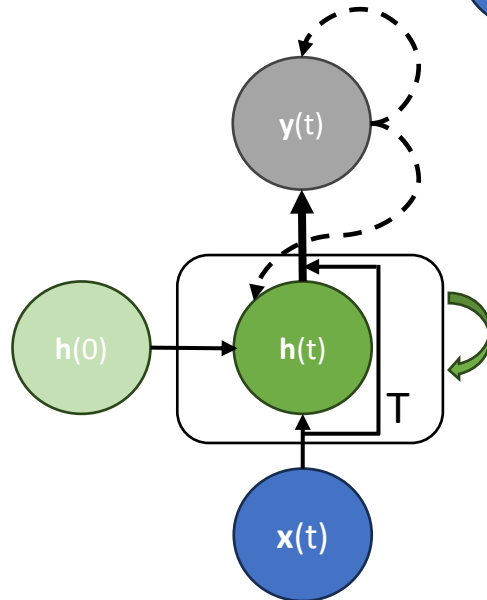
Training RNN

- Unfolding an RNN over time gives an MLP
 - Backpropagation over time – backprop over the unrolled NN
 - Issues with gradient: (stability by eigen value analysis of weight matrix)
 - Example: $h_t = wh_{t-1} + cx_t = cw^t x_0$
 - Output will shrink/expand according to the t^{th} power of the eigen values
 - If the inputs are along smaller eigen values they vanish and vice versa
 - Gradients coming from ‘deeper layers’ – ‘longer away in time’ vanish
 - Long term ‘memory’ is lost

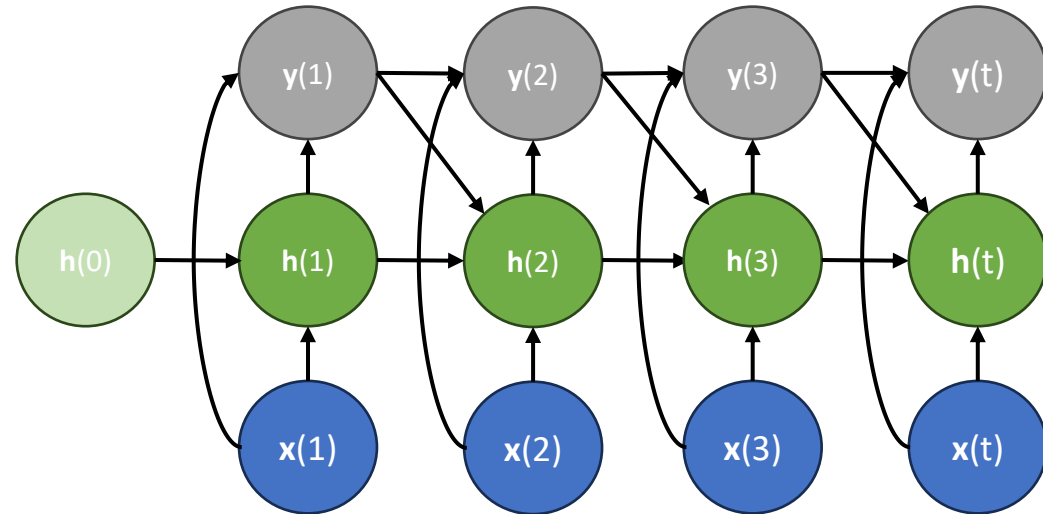
Unrolled RNN



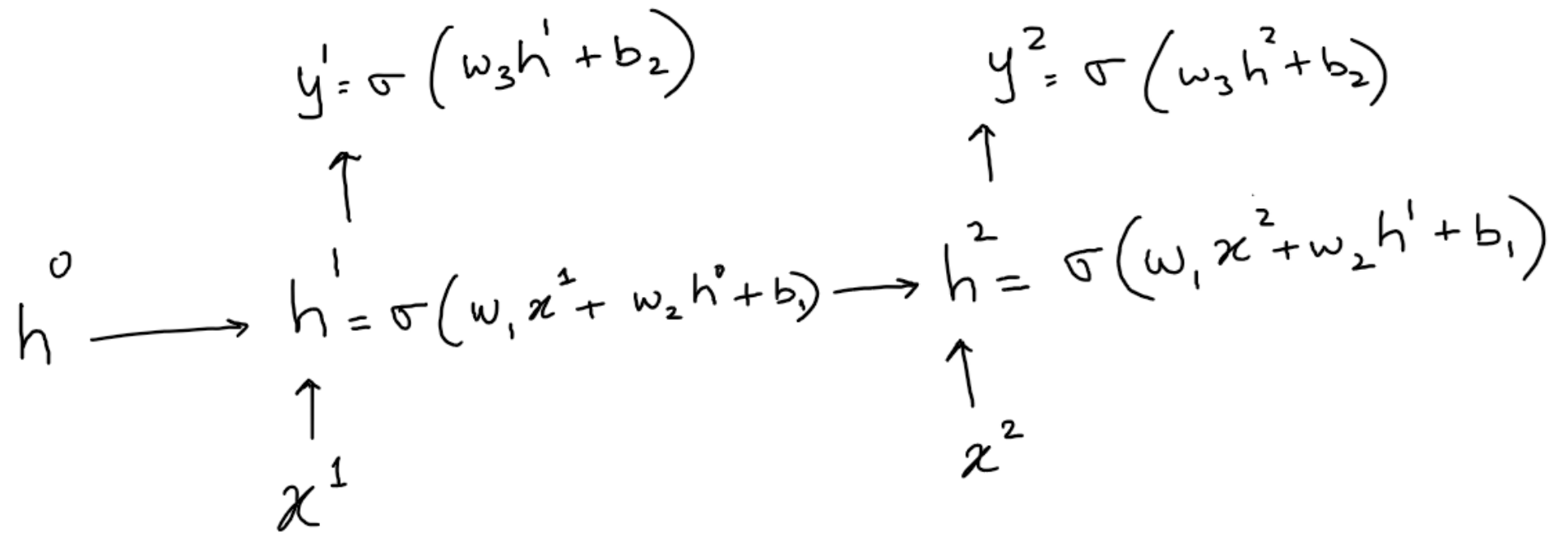
General
 $h_t = g(x_t, h_{t-1}, y_{t-1})$
 $y_t = f(x_t, h_t, y_{t-1})$

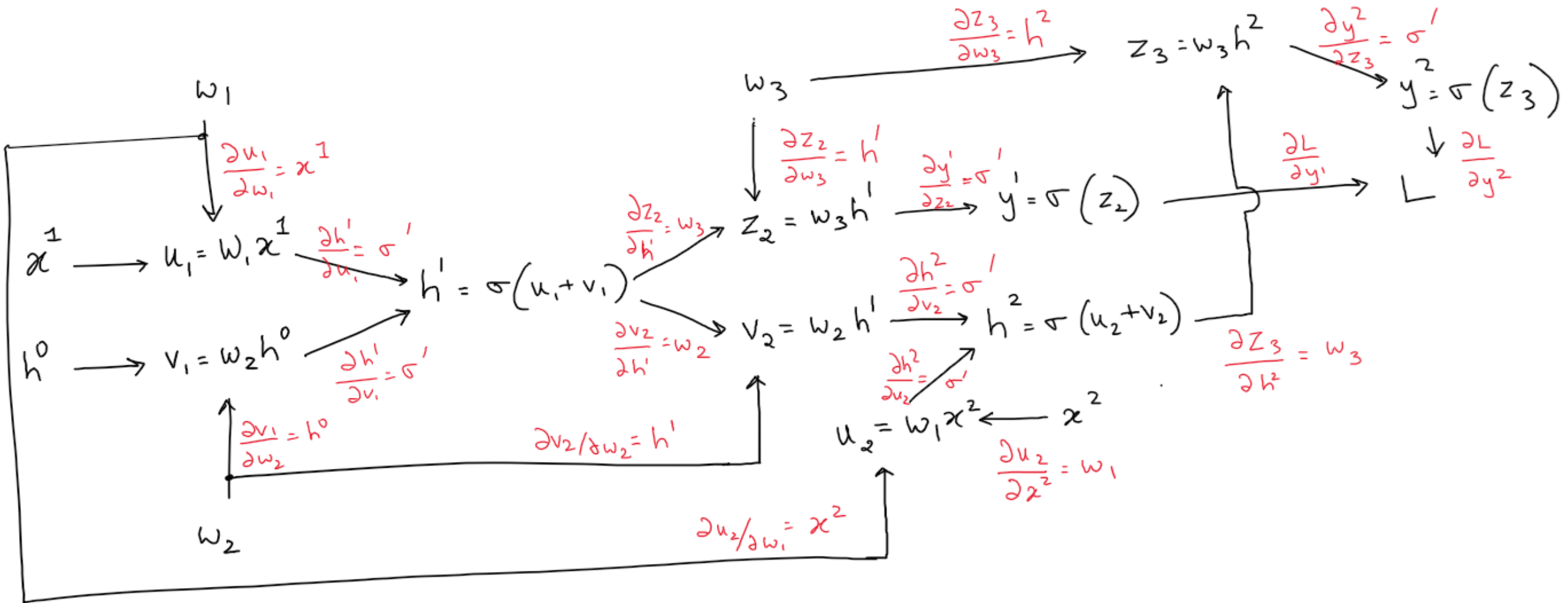


Simplified
 $h_t = g(x_t, h_{t-1})$
 $y_t = f(h_t)$



BPTT





$$\begin{aligned} \frac{\partial L}{\partial w_3} &= \frac{\partial L}{\partial y^2} \cdot \frac{\partial y^2}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_3} + \frac{\partial L}{\partial y^1} \cdot \frac{\partial y^1}{\partial z_2} \cdot \frac{\partial z_2}{\partial w_3} \\ &= \sigma' \cdot \left(\frac{\partial L}{\partial y^1} h^1 + \frac{\partial L}{\partial y^2} h^2 \right) \end{aligned}$$

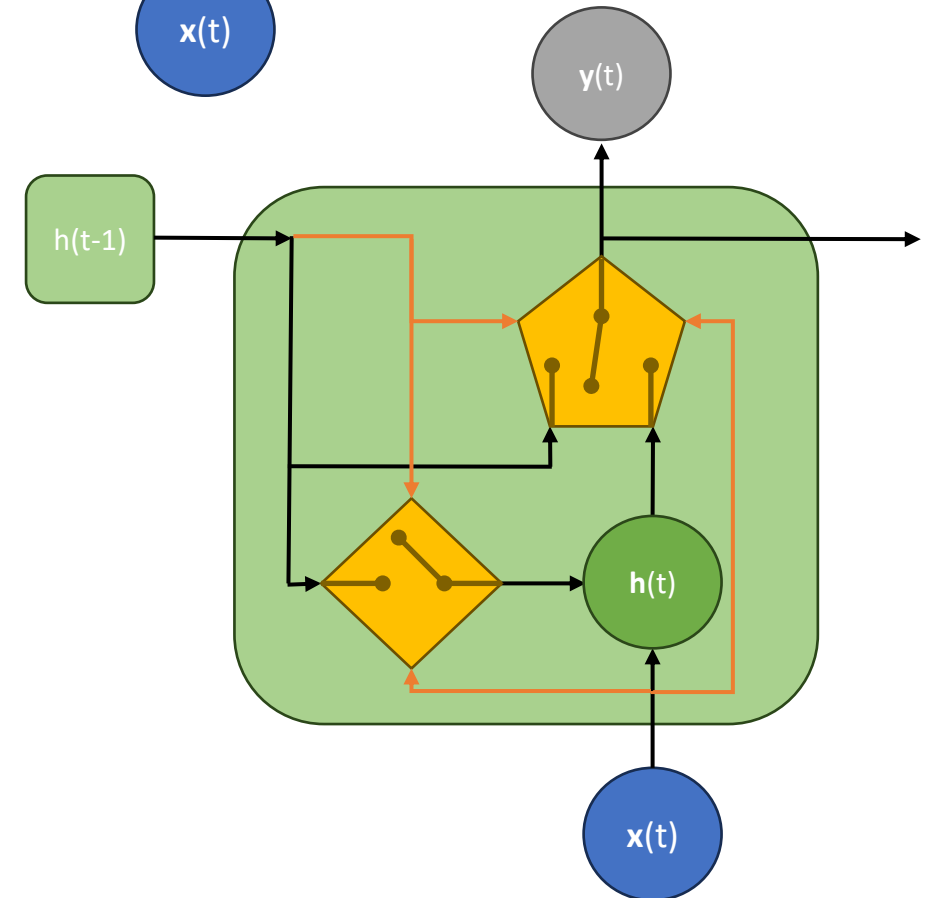
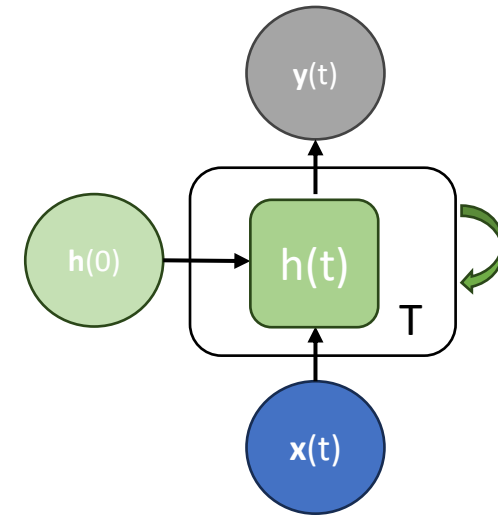
$$\begin{aligned} \frac{\partial L}{\partial w_2} &= \frac{\partial L}{\partial y^1} \cdot \frac{\partial y^1}{\partial z_2} \cdot \frac{\partial z_2}{\partial h^1} \cdot \frac{\partial h^1}{\partial v_1} \cdot \frac{\partial v_1}{\partial w_2} + \frac{\partial L}{\partial y^2} \cdot \frac{\partial y^2}{\partial z_3} \cdot \frac{\partial z_3}{\partial h^2} \cdot \frac{\partial h^2}{\partial v_2} \cdot \frac{\partial v_2}{\partial w_2} \\ &= (\sigma')^2 \cdot w_3 \cdot \left(\frac{\partial L}{\partial y^1} h^0 + \frac{\partial L}{\partial y^2} h^1 \right) \end{aligned}$$

Gating to Retain Memory

- Should the previous state always be used?
 - Skipping through time is equivalent to adding a previous state gated with a counter
- Gating the previous state: $\mathbf{h}'_t = g(\mathbf{x}_t, \mathbf{r}_t \bullet \mathbf{h}_{t-1})$
 - In computational graph, gradients are blocked when **reset gate** $\mathbf{r}_t = 0$
 - Promotes short-term memory; resets long-term to nil – **forget gate**
- Gating at the output: $\mathbf{h}_t = \mathbf{u}_t \bullet \mathbf{h}_{t-1} + (1 - \mathbf{u}_t) \bullet \mathbf{h}'_t$
 - When **update gate** $\mathbf{u}_t = 1$, in computational graph, gradient flows directly from past (without an added product of derivative of activation function – ‘short circuits’ the connection between $\mathbf{h}_t$ and $\mathbf{h}_{t-1}$, thereby, avoiding terms that could vanish the gradient)
 - Promotes long-term memory; retains previous state as current

Gated Recurrent Unit

- $\mathbf{r}_t = \sigma(\mathbf{W}_{rx}\mathbf{x}_t + \mathbf{W}_{rh}\mathbf{h}_{t-1} + \mathbf{b}_r)$
- $\mathbf{u}_t = \sigma(\mathbf{W}_{ux}\mathbf{x}_t + \mathbf{W}_{uh}\mathbf{h}_{t-1} + \mathbf{b}_u)$
 - Decides what will be the next hidden state
 - Either short term: newly computed
 - How short is short term? $\mathbf{r}_t$ decides that
 - Either use only current input
 - Or include past state & current input
 - Or long term: earlier state as such
- Same as RNN when $r_t = 1, u_t = 0$



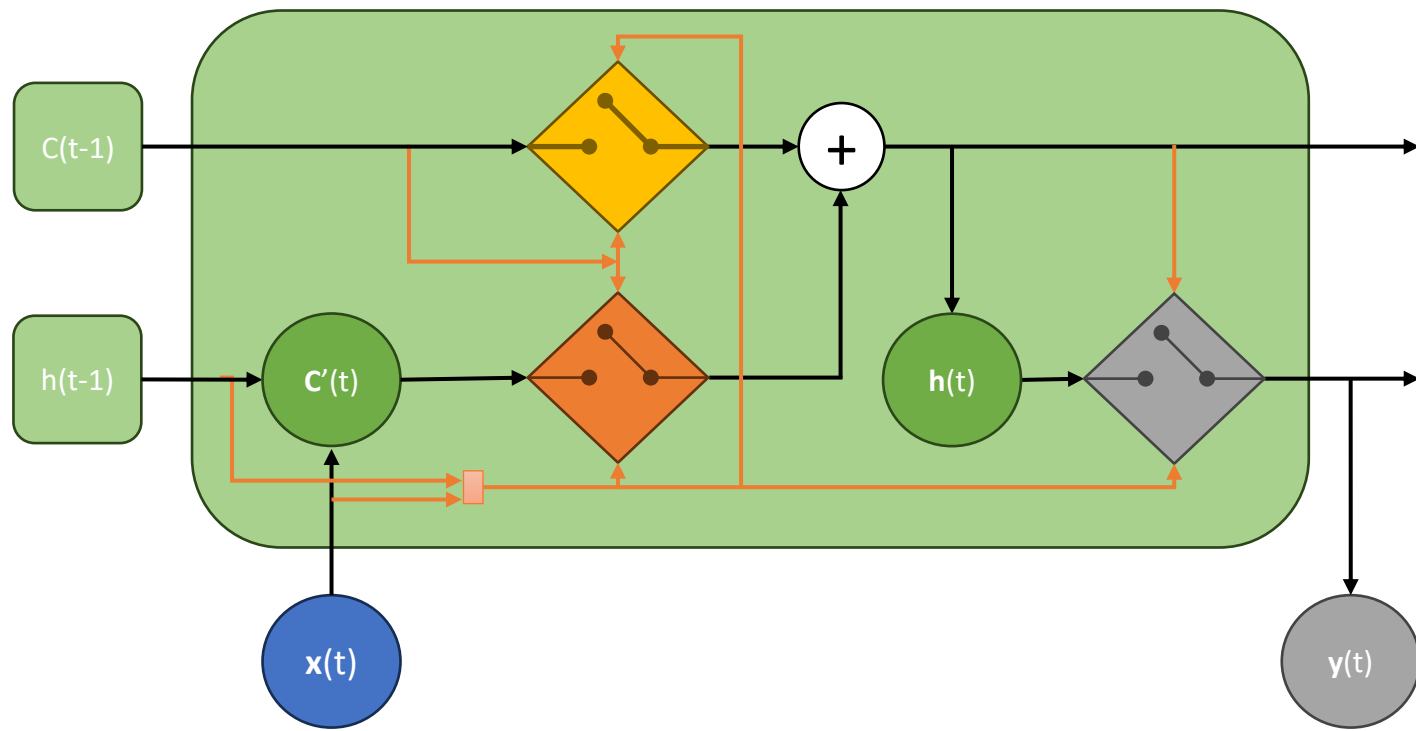
Adding Context for Long Term Memory

- GRU's update: weighted sum of long & short term states
 - Weights are close to 0 or 1 – retains either long or short term only
- LSTM: Learn appropriate function of long & short term memory
 - Long term memory is retained through cell memory or context $\mathbf{c}_t$
 - Update context $\mathbf{C}_t = \mathbf{f}_t \cdot \mathbf{C}_{t-1} + \mathbf{i}_t \cdot g_c(\mathbf{x}_t, \mathbf{h}_{t-1})$
 - Forget or retain with a gate $\mathbf{f}_t$
 - Usefulness of current input decided with input gate $\mathbf{i}_t$
 - Context (long term) is sum of long-term info + learned function of short-term info

LSTM

- Current state is updated as $\mathbf{h}_t = \mathbf{o}_t \bullet \mathbf{g}_h(\mathbf{C}_t)$
 - Output gate decides if this state is relevant to be carried forward
- Output is computed as $\mathbf{y}_t = \mathbf{f}(\mathbf{h}_t)$
- Forget gate $\mathbf{f}_t = \boldsymbol{\sigma}(\mathbf{W}_{fx}\mathbf{x}_t + \mathbf{W}_{fh}[\mathbf{C}_{t-1}\mathbf{h}_{t-1}] + \mathbf{b}_f)$
- Input gate $\mathbf{i}_t = \boldsymbol{\sigma}(\mathbf{W}_{ix}\mathbf{x}_t + \mathbf{W}_{ih}[\mathbf{C}_{t-1}\mathbf{h}_{t-1}] + \mathbf{b}_i)$
- Output gate $\mathbf{o}_t = \boldsymbol{\sigma}(\mathbf{W}_{ox}\mathbf{x}_t + \mathbf{W}_{oh}[\mathbf{C}_t\mathbf{h}_{t-1}] + \mathbf{b}_o)$

LSTM

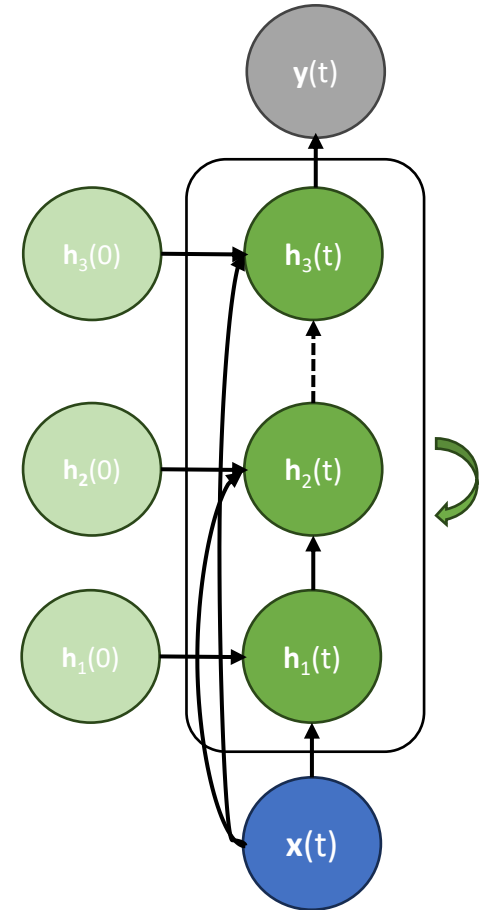


Teacher Forcing

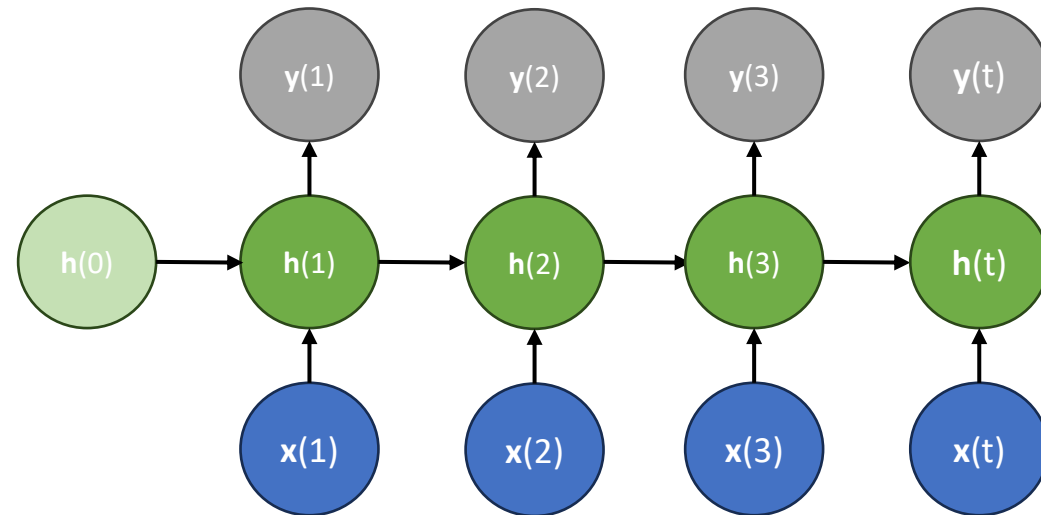
- Sequential training
 - Longer to train: Each NN unrolled T times – deeper network
 - Quicker inference: At each time step, forward pass only through $f()$ & $g()$
- Can it be parallelized? – Teacher forcing
 - During training feed ground truth $\mathbf{y}_{t-1}$ for computation of $\mathbf{y}_t$
 - If $\mathbf{h}_{t-1}$ to $\mathbf{h}_t$ connection does not exist, training can be parallel and faster
- Curriculum training in BPTT: mix of both teacher forcing and using predicted output
 - Gradually reduce teacher forcing over epochs

Stacking RNN Cells

- Multiple ‘layers’ of RNN cells stacked vertically
 - Each layer learns different features
 - Possibly of different time scales
 - Different context learned in each layer
- Deeper connection can be implemented
 - Dropout and skip connections implemented vertically
- Training is through the usual BPTT

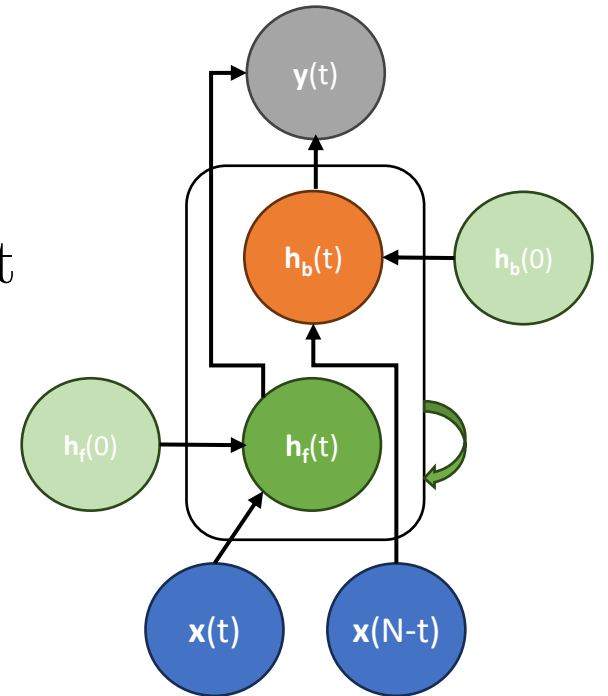


Unrolled Stacked RNN



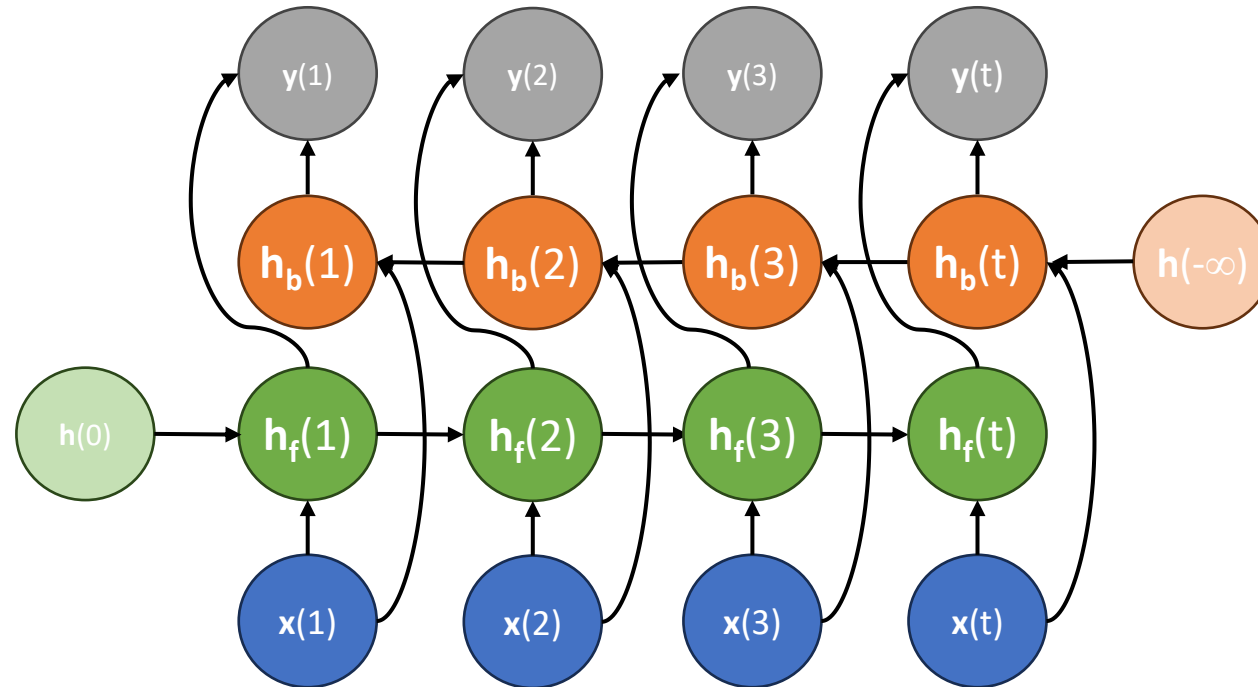
Bidirectional RNN – Breaking Causality

- In certain problems, the whole sequence may be available
 - E.g.: Translation, image captioning, summarization, etc.
 - Output length may still be unknown or input-dependent
- Output at time instant t may depend on inputs from past and future – captures more information
- Two RNN state units: $g_f()$ & $g_b()$
 - **Forward state** $\mathbf{h}_{ft}$: Inputs in forward sequence $t = 1, 2, \dots, N$
 - **Backward state** $\mathbf{h}_{bt}$: Inputs in reverse sequence $t = N, N-1, \dots, 1$
 - No weights shared between forward and backward



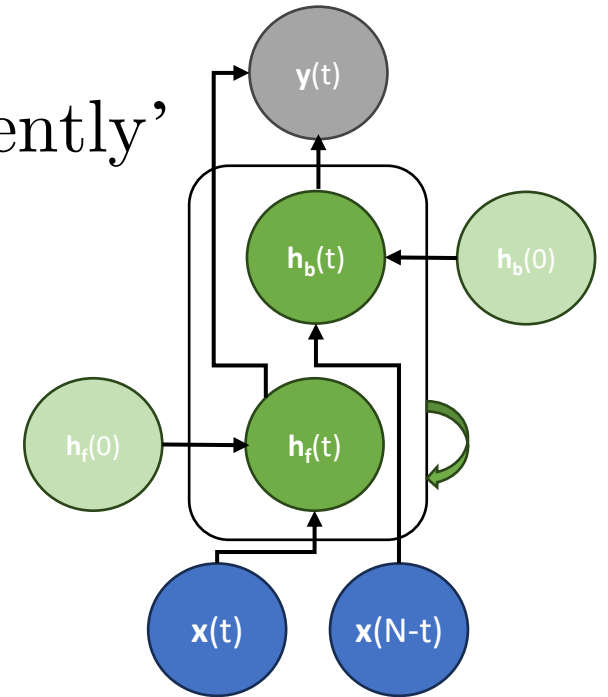
Unrolled BRNN

- All RNNs can be made BRNN
 - R. Pascanu et al, How to Construct Deep Recurrent Neural Networks
- BiLSTM
- BiGRU



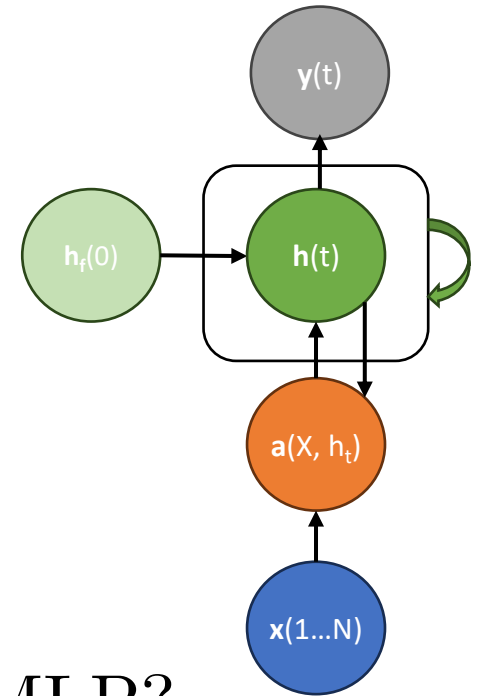
Training BRNN

- The two unrolled networks are training ‘independently’
 - Forward pass as usual
- Backward pass
 - Input to network $\mathbf{y}_t = f()$ is $[\mathbf{h}_{ft} \ \mathbf{h}_{bt}]$
 - Gradient output from $f() = [\nabla \mathbf{h}_{ft} \ \nabla \mathbf{h}_{bt}]$
 - Extract $\nabla \mathbf{h}_{ft}$ and $\nabla \mathbf{h}_{bt}$ perform backprop in $g_f()$ and $g_b()$ separately
 - If $\nabla \mathbf{x}_t$ is required at the input node, then add $\nabla \mathbf{x}_t$ from $g_f()$ and $g_b()$



Attention-Mechanism

- If the whole sequence is available, can we access it at all times?
 - $\mathbf{y}_t = f(\mathbf{x}_{1\dots t})$ instead of $\mathbf{y}_t = f(\mathbf{h}_{ft}, \mathbf{h}_{bt})$
- For the current output/state, how relevant is a sample in the input sequence?
 - Attention $\mathbf{a}_i = s(\mathbf{h}_t, \mathbf{x}_i)$, $i = 1\dots t$
 - Update the state as $\mathbf{h}_t = g(\mathbf{h}_{t-1}, \sum_i \mathbf{a}_i g_v(\mathbf{x}_i))$
- Can the **attention score** $\mathbf{a}_i$ be implemented with an MLP?



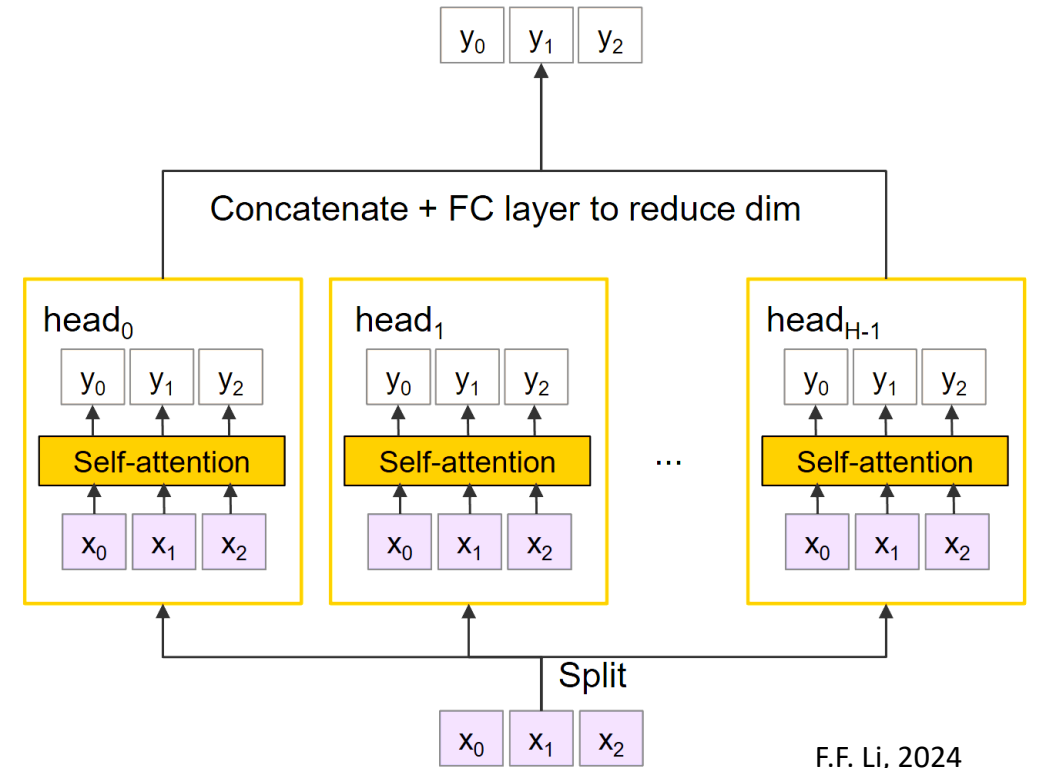
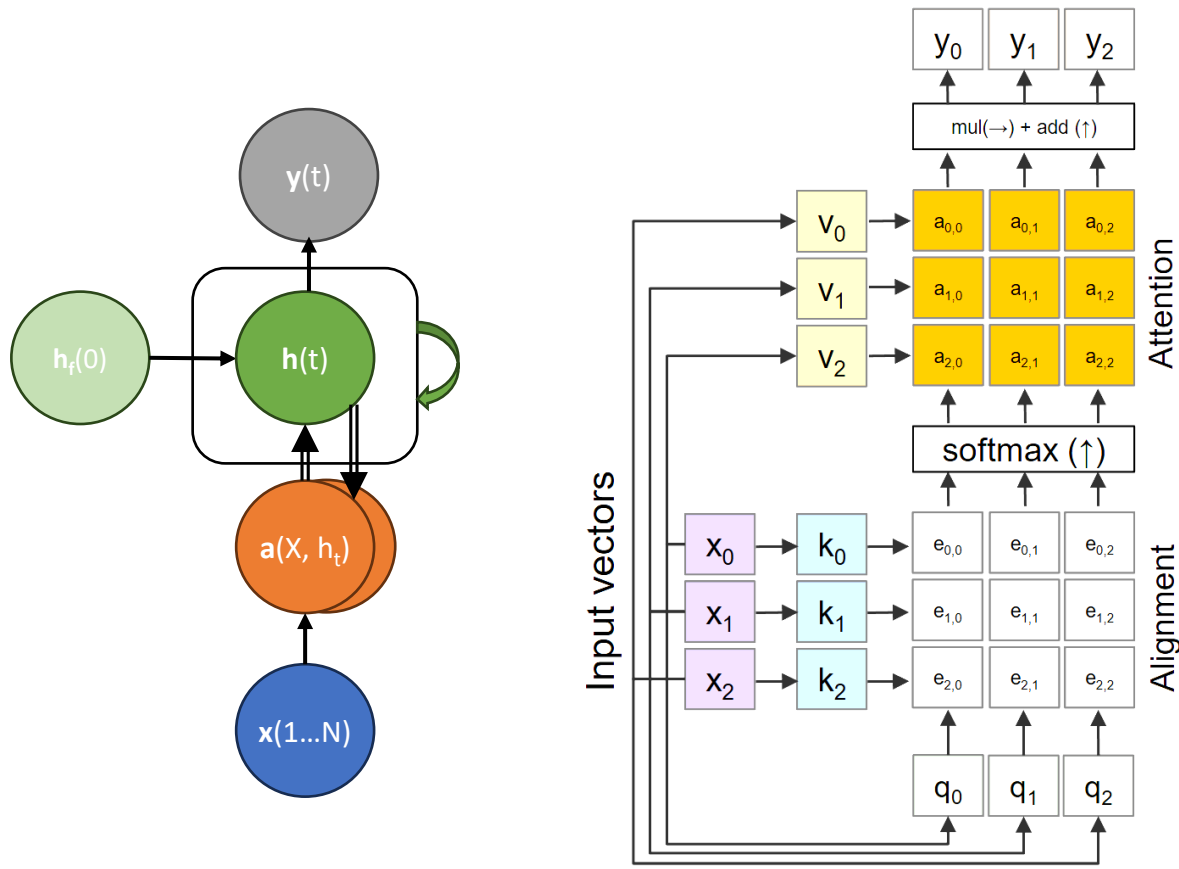
Attention Score

- For input $\mathbf{x}_i$, learn representations: key $\mathbf{k}_i$ and value $\mathbf{v}_i$
 - **Key** $\mathbf{k}_i = g_k(\mathbf{x}_i) = \mathbf{W}_k \mathbf{x}_i$ is a feature representing the input – normalized to have unit norm
 - **Value** $\mathbf{v}_i = g_v(\mathbf{x}_i) = \mathbf{W}_v \mathbf{x}_i$ is a feature of $\mathbf{x}_i$ useful for the current task
- For the state $\mathbf{h}_t$, learn a representation: **query** $\mathbf{q}_t = g_q(\mathbf{h}_t) = \mathbf{W}_q \mathbf{h}_t$ is a feature representing the current state
- Compute the attention score as $\mathbf{a}_t = s(\mathbf{q}_t, \mathbf{k}_i) = \sigma_N(\mathbf{q}_t^T \mathbf{k}_i)$
 - Projection gives **similarity score**; softmax (over all N inputs) gives a PMF of relevance – probability of how relevant is i^{th} input to the current state/output
- Attention = softmax(Query^T Normalized-Key) Value
- The weights $\mathbf{W}_k$, $\mathbf{W}_v$, $\mathbf{W}_q$ are shared for all i

Multi-Head Attention

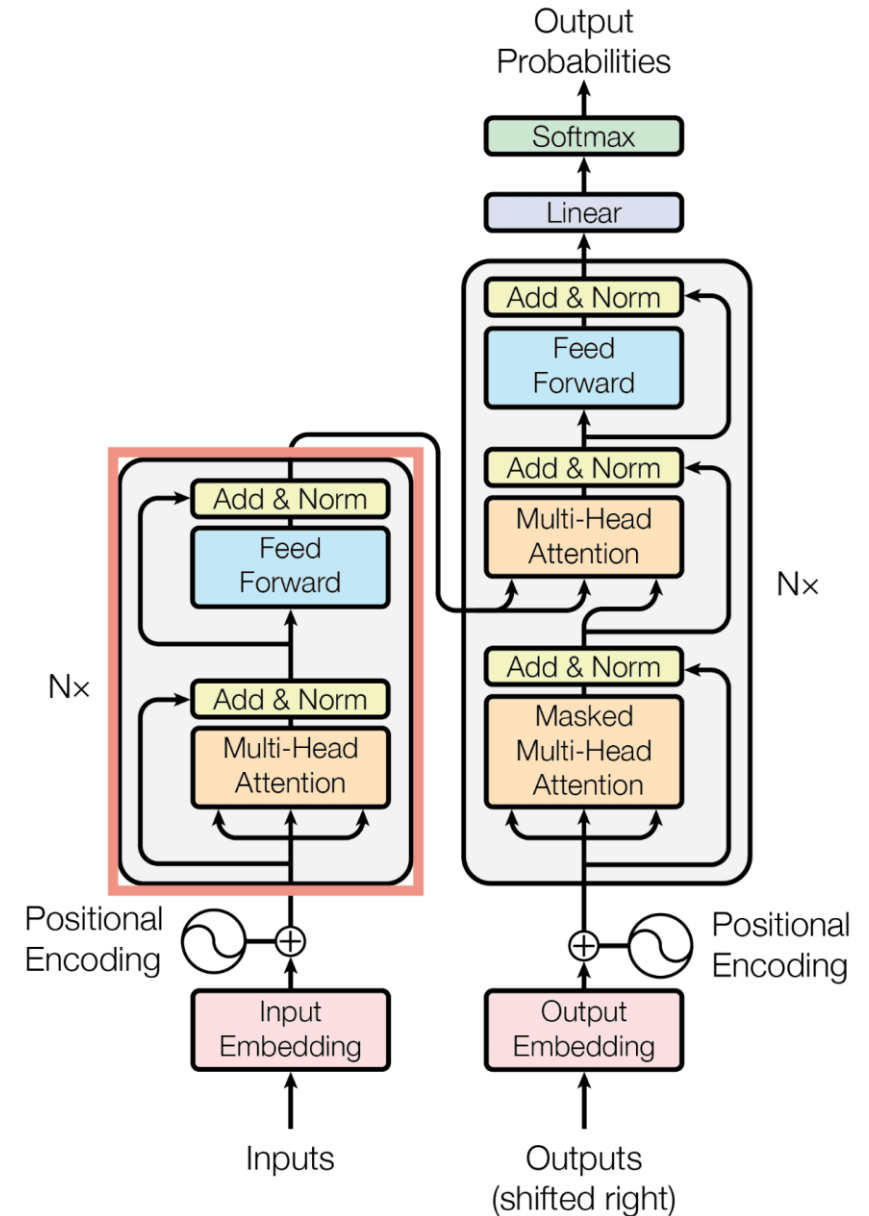
- Often, more than one input might be relevant; depending on the task performed by the NN, the input/state can have multiple relevant feature representations
 - For j^{th} feature of i^{th} input: $\mathbf{k}_{ij}$, $\mathbf{v}_{ij}$, $\mathbf{q}_j$, $j = 1$ to L
- **Multi-head** attention $\mathbf{h}_t = g(\mathbf{h}_{t-1}, g_m(\sum_i \mathbf{a}_{i1} g_{v1}(\mathbf{x}_i), \dots, \sum_i \mathbf{a}_{iL} g_{vL}(\mathbf{x}_i)))$
- This is **cross-attention**: computed between data and state
- **Self-attention**: if query and key are computed for the same sequence
 - Useful to learn a context-aware representation for an input sample
 - An input sample can represent/mean different things depending on surrounding samples

Self and Multi-Head Attention



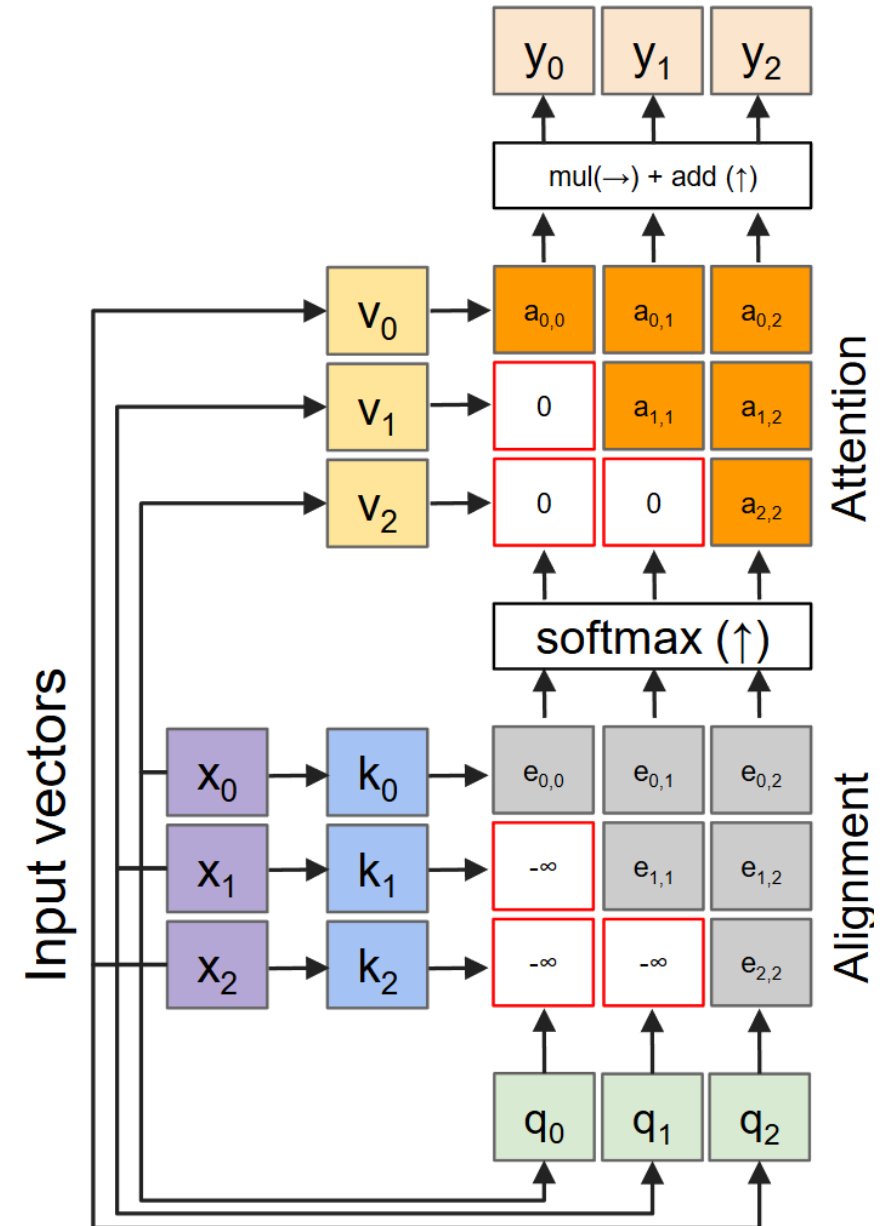
Transformer

- Fixed length input (context size): Input sequence is broken to **tokens** and mapped to **embeddings**
- Since attention is position invariant, a **positional encoding** vector is added to the input, $\mathbf{x}_i = \mathbf{x}_i + \mathbf{p}_i$
- Self-attention for inputs and outputs, and cross attention between encoder & decoder
- Parallel training – faster than RNN



Causal Attention

- Masked self-attention
 - Mask out future values in the context window
- Keys and values of past context windows are stored – KV cache
- Retrieval augmentation (RAG)
 - Database of embeddings stored elsewhere
 - Retrieved data used along with input keys



Unsupervised Learning

Autoencoders

- Learn principle (compact latent) representation through information bottleneck
- Applications
 - PCA
 - Denoising
 - Compression
 - Bootstrapping

